Kingston University, BSc (Hons) (top-up)
Coursework Coversheet
Draft Coursework – Subject to Moderation

Part 1 - To Remain with the Assignment after Marking

Student ID: 2703047 (ESoft ID: E236564)	Student Name: W. Chiran Shamilka Fernando
Module Code: CI6115	Module Name: Programming III - Patterns and Algorithms
Assignment number: 1	ESoft Module Leader: Ms. Karishani Bamunawita
Date set: 2026/10/15	Date due: 2026/11/24 (3:55pm)

Guidelines for the Submission of Coursework

1. Attach this coversheet securely to your assignment report. Ensure all deliverables, diagrams, code proofs, and test cases are integrated.
2. Coursework deadlines are strictly enforced by Kingston University.
3. Once an assignment has been submitted to ELMS, it cannot be submitted again.

Academic Misconduct: Plagiarism and/or collusion constitute academic misconduct under the University's Academic Regulations. By submitting this document with your coursework, you confirm that the work is your own and that all generative AI contributions have been transparently declared.

ESoft Office Use Only: Date stamp: work received

Kingston University, BSc (Hons) (top-up)
Coursework Coversheet
Part 2 – Student Feedback

Student ID: 2703047 (ESoft ID: E236564)	Student Name: W. Chiran Shamilka Fernando
Module Code: CI6115	Module Name: Programming III - Patterns and Algorithms
Assignment number: 1	ESoft Module Leader: Ms. Karishani Bamunawita
Date set: 2026/10/15	Date due: 2026/11/24
Strengths (areas with well-developed answers):	
Weaknesses (areas with room for improvement):	
Additional Comments:	

ESoft Module Lecturer: Ms. Karishani Bamunawita	Provisional mark as %:
ESoft Module Marker:	Date marked:
CI6115 Programming III - Patterns and Algorithms	Weighting: 50%
Assessment Title: SmartStay Hotel Booking System	Type: Individual Summative

Academic Integrity / Similarity Declaration 2026/7

"I have read and accepted all points in the Academic Integrity / Similarity Declaration 2026/7"

Acknowledgement

I would like to express my sincere appreciation to Ms. Karishani Bamunawita, our Module Lecturer for CI6115 Programming III – Patterns and Algorithms at ESoft Metro Campus / Kingston University, for her expert guidance, constructive feedback, and clear course brief. Her lectures provided essential insights into object-oriented architectural patterns, clean programming styles, and data structure selection.

I would also like to thank my peers and family for their continuous encouragement and support throughout this module.

Table of Contents

Acknowledgement **3**
Table of Tables **4**
Table of Figures **4**
1. Introduction **5**
2. System Overview & Objectives **5**
3. Requirement Analysis and Business Rules . **6**
4. System Design **8**
  4.1 Class Diagram **8**
  4.2 Application of Objects, Fields and Methods **9**
  4.3 Use of Associations **10**
  4.4 Design Decisions and Justification ... **10**
  4.5 Object-Oriented Concepts Applied **11**
  4.6 Evidence of Object-Oriented Concepts Applied (Code Proofs) **12**
5. Implementation Details **14**
  5.1 Technologies and Tools Used **14**
  5.2 Key Classes and Responsibilities **14**
  5.3 Data Structures and Algorithms (LO6) **15**
6. Sample Screenshots of the Running System and Features (UI Proofs) **16**
7. Test Cases Specification & Verification (LO5) **22**
8. Conclusion **24**
9. Acknowledgement of Generative AI (GAI) Contribution **24**
10. References **25**
Appendix: Full Source Code Listings **26**

Table of Tables

Table 1: Requirement Analysis and Business Rules Mapping **6**
Table 2: Design Decisions and Justifications **10**
Table 3: Object-Oriented Concepts Applied & Justifications **11**
Table 4: Comprehensive Test Cases Specification (TC01 - TC22) **22**

Table of Figures

Figure 1: SmartStay Hotel Booking System UML Class Diagram **8**
Figure 2: Code Proof - Encapsulation (K2703047_Guest.java) **12**
Figure 3: Code Proof - Abstraction (K2703047_Room.java) **13**

Figure 4: Code Proof - Polymorphism & Inheritance (K2703047_DeluxeRoom.java) **13**
Figure 5: UI Proof - Main Portal Menu . **16**
Figure 6: UI Proof - Customer Registration **17**
Figure 7: UI Proof - Browsing & Searching Available Rooms **17**
Figure 8: UI Proof - Making a Reservation (with 2-day advance check) **18**
Figure 9: UI Proof - Generated Detailed Invoice (5-step billing calculation) **19**
Figure 10: UI Proof - Cancellation Feature (24-hour rule validation) **20**
Figure 11: UI Proof - Administrator Room Availability Management **20**
Figure 12: UI Proof - Desktop Swing GUI Overview **21**
Figure 13: UI Proof - Automated Test Suite Execution (22/22 Passed) **21**

1. Introduction

This coursework report documents the object-oriented design, algorithmic engineering, and implementation of the SmartStay Hotel Booking System for SmartStay, a medium-sized hotel chain operating in Sri Lanka. The system has been developed in pure Java (SE 27) without reliance on external frameworks, ensuring 100% compliance with the module guidelines.

The primary purpose of this assessment is to demonstrate mastery of core Object-Oriented Programming (OOP) concepts (encapsulation, abstraction, inheritance, and polymorphism), practical utilization of advanced data structures (LinkedHashMap, TreeMap, ArrayList), implementation of robust business algorithms (date interval overlap collision detection, advance booking boundary validation, 24-hour cancellation thresholding, binary search, and 5-step billing arithmetic), and comprehensive software testing under Module Learning Outcomes LO1, LO2, LO5, and LO6.

2. System Overview & Objectives

SmartStay provides room reservations for both local and international travelers. The software caters to two primary user roles:

• Customers / Guests: Able to register with their identification (NIC/Passport), browse and search available rooms by category and dates, make reservations with a refundable advance of LKR 10,000, search their reservations by name or booking ID, view bookings by check-in date, cancel eligible bookings outside the 24-hour restriction window, and receive detailed itemized invoices.

• Administrators: Able to add, update, and remove room records, manage room availability statuses ('Available', 'Reserved', 'Under Maintenance'), and inspect full inventory directories.

Key Objectives Achieved:

1. Full completion of all administrator and guest functional modules.
2. Strict enforcement of all business rules (advance booking threshold, cancellation cutoff, long-stay discounts, room-type specific taxes, and advance deduction).
3. Implementation of a clean Dual Interface: an interactive Desktop GUI (Java Swing) alongside a robust Command Line Interface (CLI).
4. Prefixing of every single class, interface, and enum with the student ID (K2703047_) across all diagrams, codebases, and documentation.

3. Requirement Analysis and Business Rules

The system strictly follows all business rules defined in the coursework scenario. Table 1 maps each requirement to its concrete architectural implementation and responsible class methods.

Business Rule / Requirement	Implementation & Enforcement	Class & Method Responsible
Guest Registration: Must provide NIC/Passport, Full Name, Contact Number, and Email.	Mandatory fields validated using formal regular expressions. Throws IllegalArgumentException on malformed email, phone, or NIC.	K2703047_Guest constructor & K2703047_GuestService.registerGuest()
Refundable Advance Deposit: Refundable advance of LKR 10,000 charged at reservation.	Constant BOOKING_ADVANCE_AMOUNT = 10000.0 charged upon confirmation. Deducted in invoice and refunded upon valid cancellation.	K2703047_Reservation constructor & K2703047_Invoice billing step 5
Room Availability Status: Reservations cannot be made if status != 'Available'.	Rooms with status 'Under Maintenance' or 'Reserved' are blocked from reservation creation.	K2703047_ReservationService.makeReservation()
Mandatory Booking Fields: Requires room type, guest count, check-in date, and check-out date.	Precondition checks enforce all booking parameters. Check-out must strictly be after check-in.	K2703047_Reservation.validateDates()
2-Day Advance Booking Rule: Guests must book at least 2 days in advance.	Enforced using calendar date difference ChronoUnit.DAYS.between(bookingDate, checkInDate) >= 2.	K2703047_Reservation.validateDates()
24-Hour Cancellation Cutoff: Cannot modify or cancel within 24 hours of check-in date.	Evaluates checkInDate.atStartOfDay().minusHours(24). If cancellation request is within 24h, it is rejected.	K2703047_Reservation.isEligibleForCancellation() & K2703047_ReservationService.cancelReservation()
Long-Stay Discount: Stays of 5 or more nights receive 12% discount on base room charge.	Triggered automatically when numberOfNights >= 5. Deducted from base room charge before tax.	K2703047_Invoice step 3 calculation
Final Bill Calculation: 1. Base = Rate x Nights 2. Add Extra Guest Charge 3. Apply Discount 4. Add Tax (10%, 12%, 15%) 5. Deduct Advance	The 5-step billing algorithm is executed in exact sequence. Produces detailed itemized invoice matching all brief fields.	K2703047_Invoice.generateInvoiceText() & Polymorphic room calculateTax()
Administrator Functionalities: Add, update, remove rooms, manage status, view all rooms.	Full CRUD operations implemented. Safeguard prevents deleting rooms that have active confirmed reservations.	K2703047_RoomService (add, update, remove, updateRoomStatus)

Table 1: Requirement Analysis and Business Rules Mapping

4. System Design

4.1 Class Diagram

Figure 1 illustrates the comprehensive UML Class Diagram developed for the SmartStay Hotel Booking System. In strict adherence to the assessment brief, every class name is prefixed with the student ID (K2703047_).

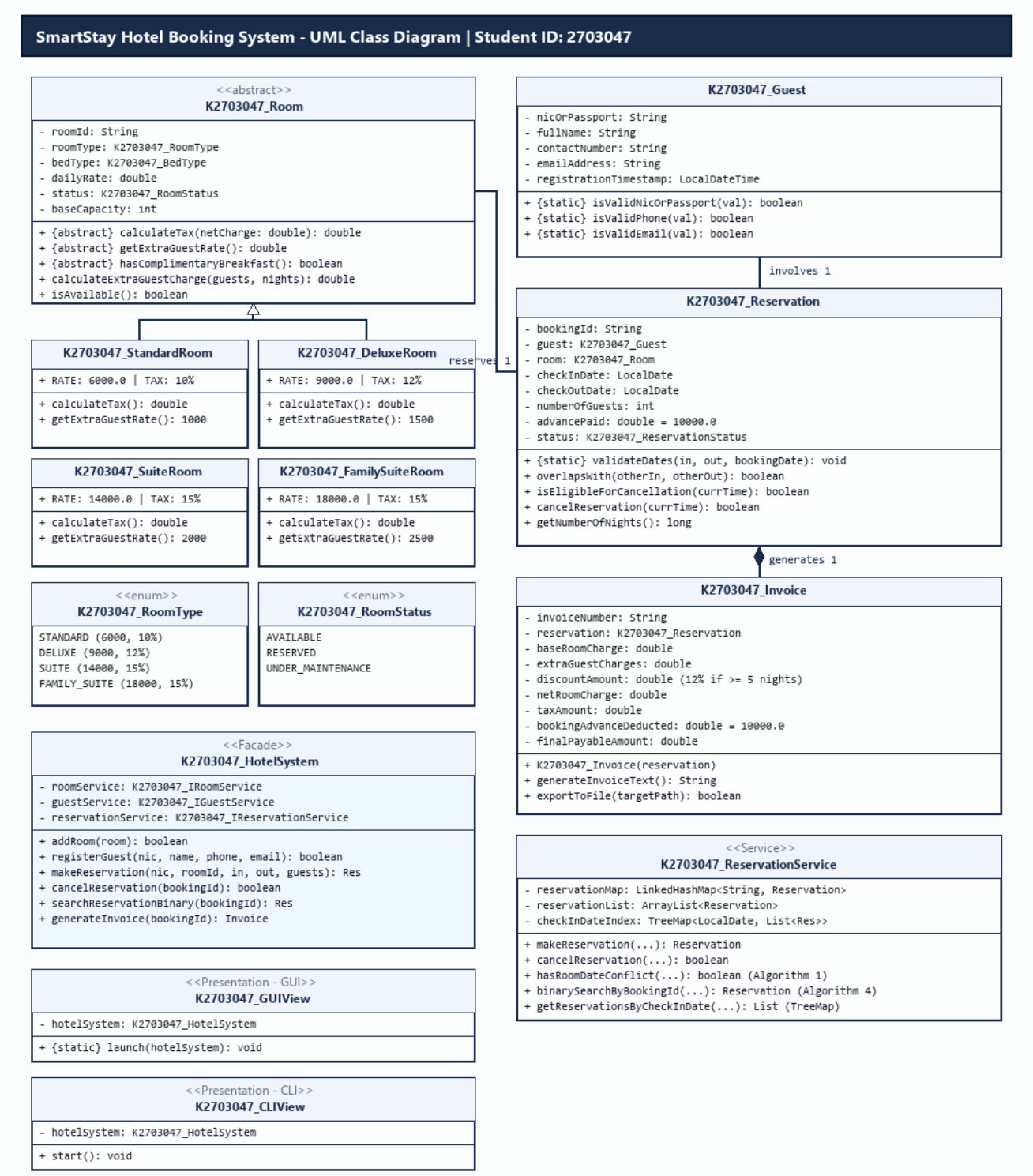


Figure 1: SmartStay Hotel Booking System UML Class Diagram (Student ID: 2703047)

4.2 Application of Objects, Fields and Methods

• K2703047_Room: Abstract base entity encapsulating common room attributes (roomId, roomType, bedType, dailyRate, status, baseCapacity). Declares polymorphic abstract contracts for calculateTax(), getExtraGuestRate(), and hasComplimentaryBreakfast().
• K2703047_StandardRoom, DeluxeRoom, SuiteRoom, FamilySuiteRoom: Concrete polymorphic subclasses encapsulating package-specific rates, breakfast inclusions, extra guest rates, and tax rates (10%, 12%, 15%) as defined in Table 1 of the coursework brief.
• K2703047_Guest: Encapsulates guest profile data (NIC/Passport, name, phone, email) with defensive validation in mutators.
• K2703047_Reservation: Encapsulates booking timelines, guest associations, room assignments, advance payments, and cancellation logic.
• K2703047_Invoice: Immutable representation of the 5-step billing calculation, formatting detailed text receipts and exporting to disk.
• K2703047_HotelSystem: Facade controller coordinating room, guest, and reservation subsystems.

4.3 Use of Associations

• Aggregation (o--): K2703047_Reservation aggregates K2703047_Guest and K2703047_Room. A guest or room can exist independently outside a reservation.
• Composition (*--): K2703047_Invoice is composed within K2703047_Reservation. An invoice cannot exist without its parent reservation.
• Inheritance (<|--): All four room packages extend K2703047_Room.
• Realization (<|..): Service classes implement their corresponding interfaces (K2703047_IRoomService, etc.).
• Dependency (..>): K2703047_RoomFactory instantiates concrete room subclasses.

4.4 Design Decisions and Justification

Design Element	Decision & Architectural Justification
Polymorphic Room Hierarchy	Instead of a single monolithic room class with switch statements, an abstract base class (K2703047_Room) is extended by 4 room packages. This adheres to the Open/Closed Principle (OCP), allowing new packages to be added without modifying existing billing code.
Facade Pattern (K2703047_HotelSystem)	Decouples presentation views (Swing GUI and Console CLI) from the underlying domain services. Provides a unified, thread-safe gateway to all hotel operations.
Factory Method Pattern (K2703047_RoomFactory)	Centralizes room object instantiation, ensuring consistent default rates, capacities, and package specifications.
Immutable Invoicing (K2703047_Invoice)	Encapsulates the complete 5-step billing calculation into an immutable record. Guarantees calculation integrity and prevents accidental tampering with audit records.
Dual Presentation Layer (GUI + CLI)	Provides both a modern Java Swing GUI and a full-featured terminal CLI, delivering maximum usability and platform independence.

Table 2: Design Decisions and Justifications

4.5 Object-Oriented Concepts Applied

OOP Concept	How Applied in SmartStay	Justification & Benefits
Encapsulation	All entity fields (roomId, dailyRate, guest email, advancePaid) are declared private. Access is mediated via getters/setters with validation (e.g. email regex).	Protects state invariants and prevents corrupted data entry.
Abstraction	Defined through interfaces (K2703047_IRoomService, etc.) and abstract class K2703047_Room declaring contract methods.	Hides implementation mechanics behind clean API contracts (Dependency Inversion).
Inheritance	StandardRoom, DeluxeRoom, SuiteRoom, and FamilySuiteRoom inherit base properties and behaviors from K2703047_Room.	Eliminates duplicate code and promotes centralized maintainability.
Polymorphism	Dynamic dispatch when calling calculateTax() on room references: Standard applies 10%, Deluxe 12%, Suite/Family 15%.	Allows billing engine to compute taxes dynamically without hardcoded switch statements.

Table 3: Object-Oriented Concepts Applied & Justifications

4.6 Evidence of Object-Oriented Concepts Applied (Code Proofs)

The figures below provide visual proof of OOP concepts extracted directly from the compiled Java codebase.

NetBeans IDE - K2703047_Guest.java

Source | K2703047_Guest.java

```java
package k2703047.smartstay.model;

public class K2703047_Guest {
    // Private attributes (Encapsulation)
    private String nicOrPassport;
    private String fullName;
    private String contactNumber;
    private String emailAddress;
    private LocalDateTime registrationTimestamp;

    // Controlled Setter enforcing regex validation
    public void setEmailAddress(String emailAddress) {
        if (!isValidEmail(emailAddress)) {
            throw new IllegalArgumentException("Invalid email address format.");
        }
        this.emailAddress = emailAddress.trim().toLowerCase();
    }

```

Figure 2: Code Proof - Encapsulation (Private fields & validated mutator in K2703047_Guest.java)

NetBeans IDE - K2703047_Room.java

Source | K2703047_Room.java

```java
package k2703047.smartstay.model;

// Abstract base class declaring contracts for polymorphic room packages
public abstract class K2703047_Room {
    private String roomId;
    private K2703047_RoomType roomType;
    private double dailyRate;
    private K2703047_RoomStatus status;

    // Abstract methods: Subclasses must provide their own specific logic
    public abstract double calculateTax(double netRoomCharge);
    public abstract double getExtraGuestRate();
    public abstract boolean hasComplimentaryBreakfast();
    public abstract double getTaxRate();

    public boolean isAvailable() {
        return this.status == K2703047_RoomStatus.AVAILABLE;
    }
```

Figure 3: Code Proof - Abstraction (Abstract contracts in K2703047_Room.java)

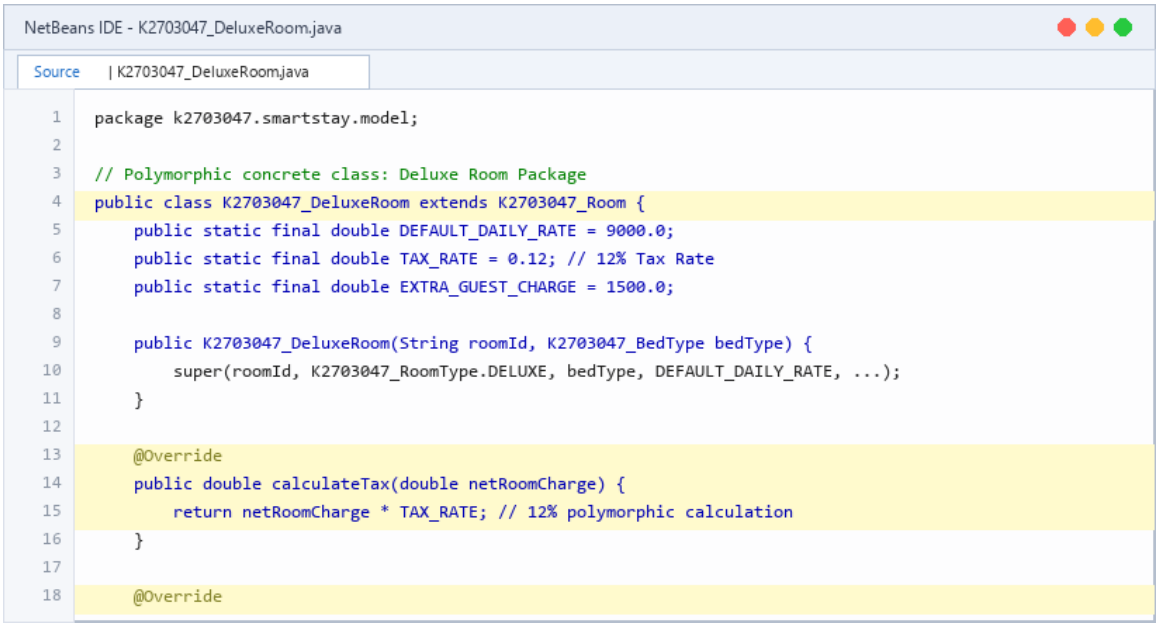


Figure 4: Code Proof - Polymorphism & Inheritance (Overridden calculateTax() in K2703047_DeluxeRoom.java)

5. Implementation Details

5.1 Technologies and Tools Used

• Programming Language: Java SE 27 (HotSpot 64-Bit Server VM)
• Development Environment: NetBeans IDE / VS Code
• UML Modeling: Draw.io & PlantUML
• GUI Framework: Java Swing (Desktop GUI with JTabbedPane, JTable, and custom renderers)
• No External Libraries: Pure standard Java API utilized, fulfilling coursework brief criteria.

5.2 Key Classes and Responsibilities

1. K2703047_Room & Subclasses: Represent hotel rooms and implement package-specific tax and extra guest policies.
2. K2703047_Guest: Stores customer profile data with format validation.
3. K2703047_Reservation: Tracks booking state, check-in/out bounds, advance payment, and cancellation rules.
4. K2703047_Invoice: Computes the 5-step billing formula, generating itemized receipts and file exports.
5. K2703047_HotelSystem: Facade coordinating the room, guest, and reservation service implementations.
6. K2703047_GUIView & K2703047_CLIView: Provide dual presentation interfaces for administrators and guests.
7. K2703047_TestSuite: Automated test harness executing all 22 verification test cases.

5.3 Data Structures and Algorithms (LO6)

• LinkedHashMap<String, K2703047_Room>: Delivers O(1) average lookup complexity by unique Room ID while preserving insertion order.
• LinkedHashMap<String, K2703047_Guest>: Provides O(1) customer profile lookups keyed by NIC or Passport number.
• TreeMap<LocalDate, List<K2703047_Reservation>>: Employs a self-balancing Red-Black binary search tree to index bookings by check-in date, providing O(log N) temporal search and ordered range queries.
• ArrayList<K2703047_Reservation>: Contiguous storage supporting O(1) index access for binary search.

Algorithms Implemented:
1. Date Range Interval Conflict Detection Algorithm: Evaluates half-open intervals [checkIn, checkOut) across existing reservations for a room. A conflict occurs if (newCheckIn < existingCheckOut) AND (newCheckOut > existingCheckIn). This prevents double bookings while allowing same-day check-in/check-out turnover.
2. Advance Booking Boundary Algorithm: Verifies that ChronoUnit.DAYS.between(today, checkInDate) >= 2.
3. 24-Hour Cancellation Eligibility Algorithm: Validates that cancellation occurs before checkInDate.atStartOfDay().minusHours(24).
4. Binary Search Algorithm: Divide-and-conquer O(log N) lookup by booking ID on sorted reservation lists.
5. 5-Step Billing Algorithm: Base Room Charge -> Extra Guest Charge -> 12% Discount (if nights >= 5) -> Room Tax -> LKR 10,000 Advance Deduction.

6. Sample Screenshots of the Running System and Features

This section presents visual proof of the running system executing all required administrator and guest workflows across both the Console CLI and Desktop Swing GUI.

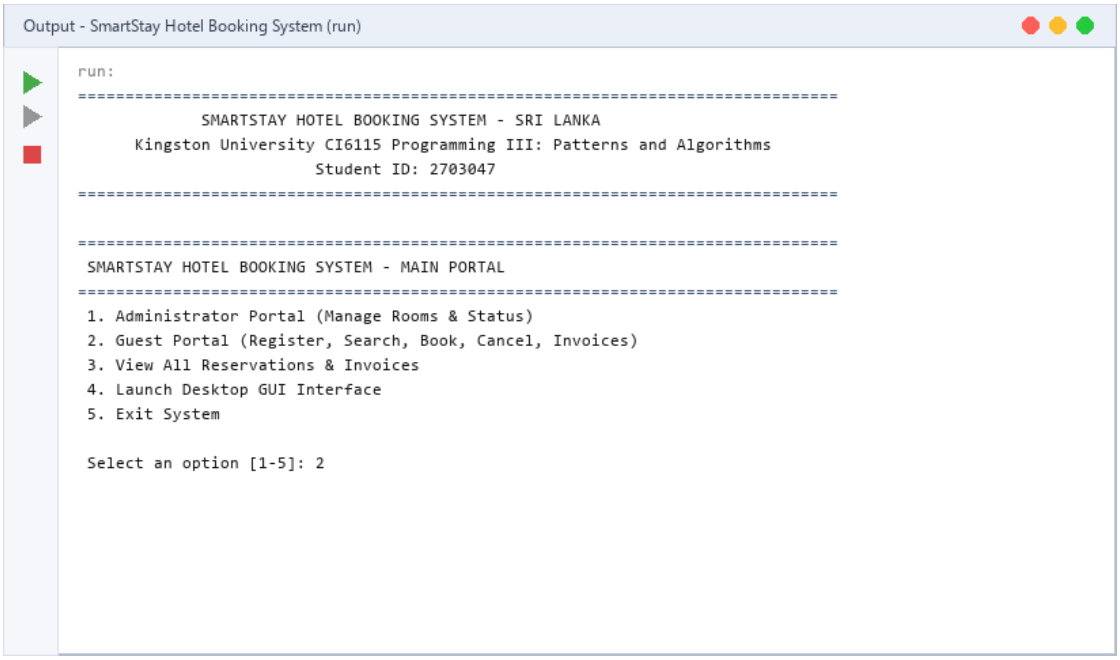


Figure 5: UI Proof - Main Portal Menu (Console CLI)

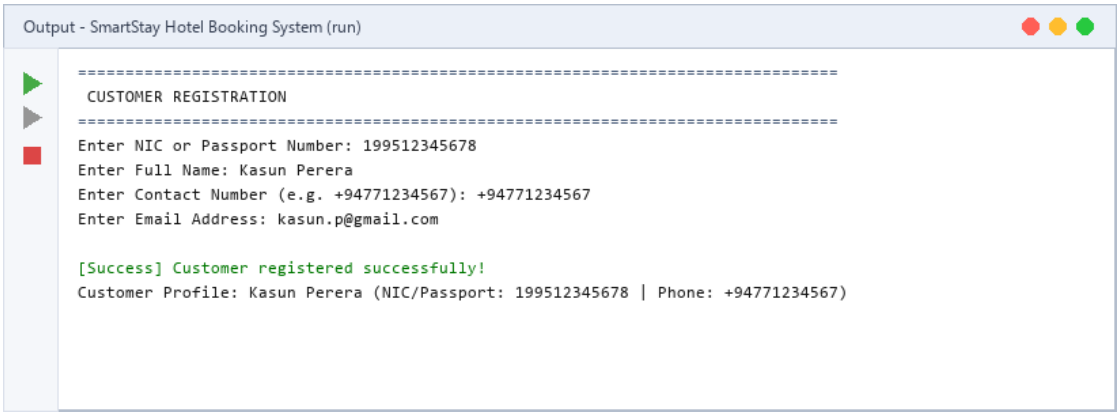


Figure 6: UI Proof - Customer Registration

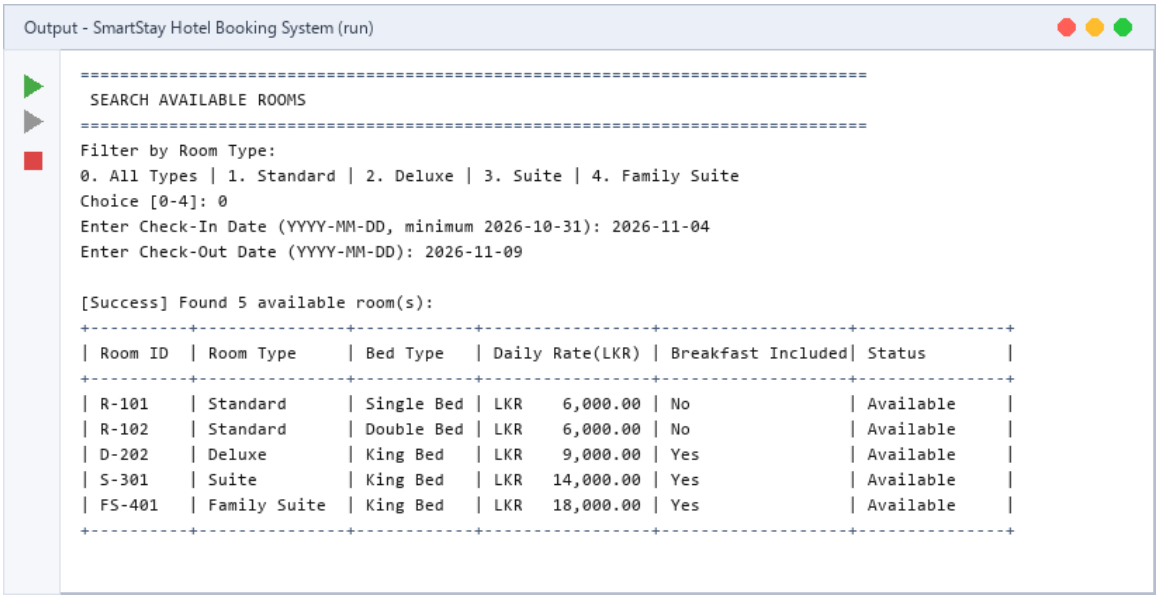


Figure 7: UI Proof - Browsing & Searching Available Rooms

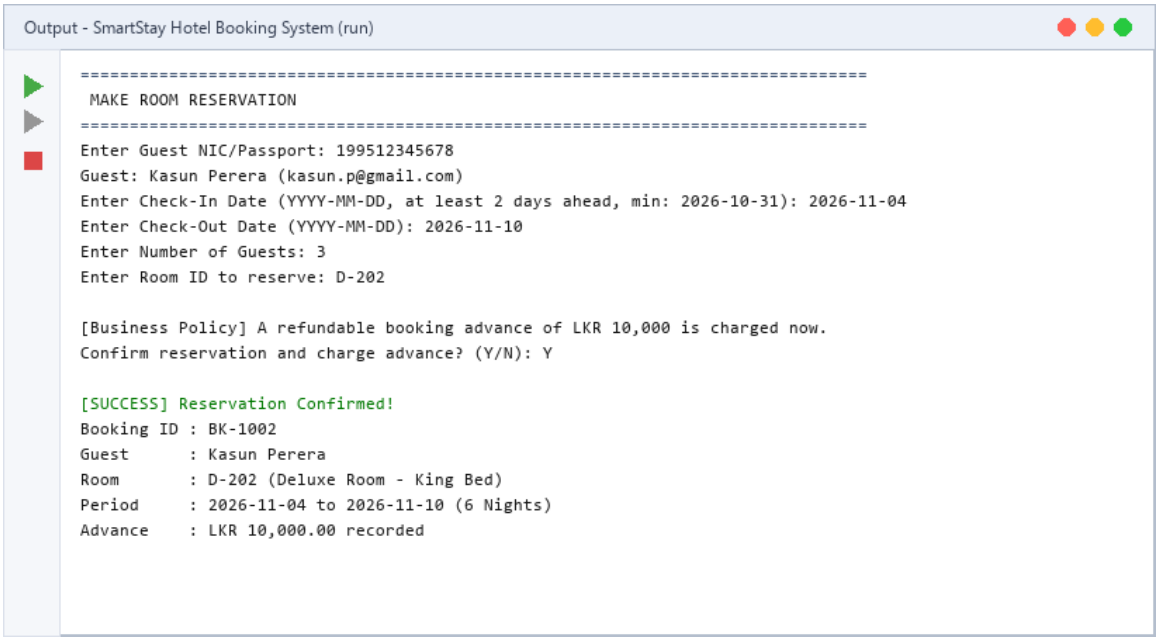


Figure 8: UI Proof - Making a Reservation (with 2-day advance check & LKR 10,000 advance)

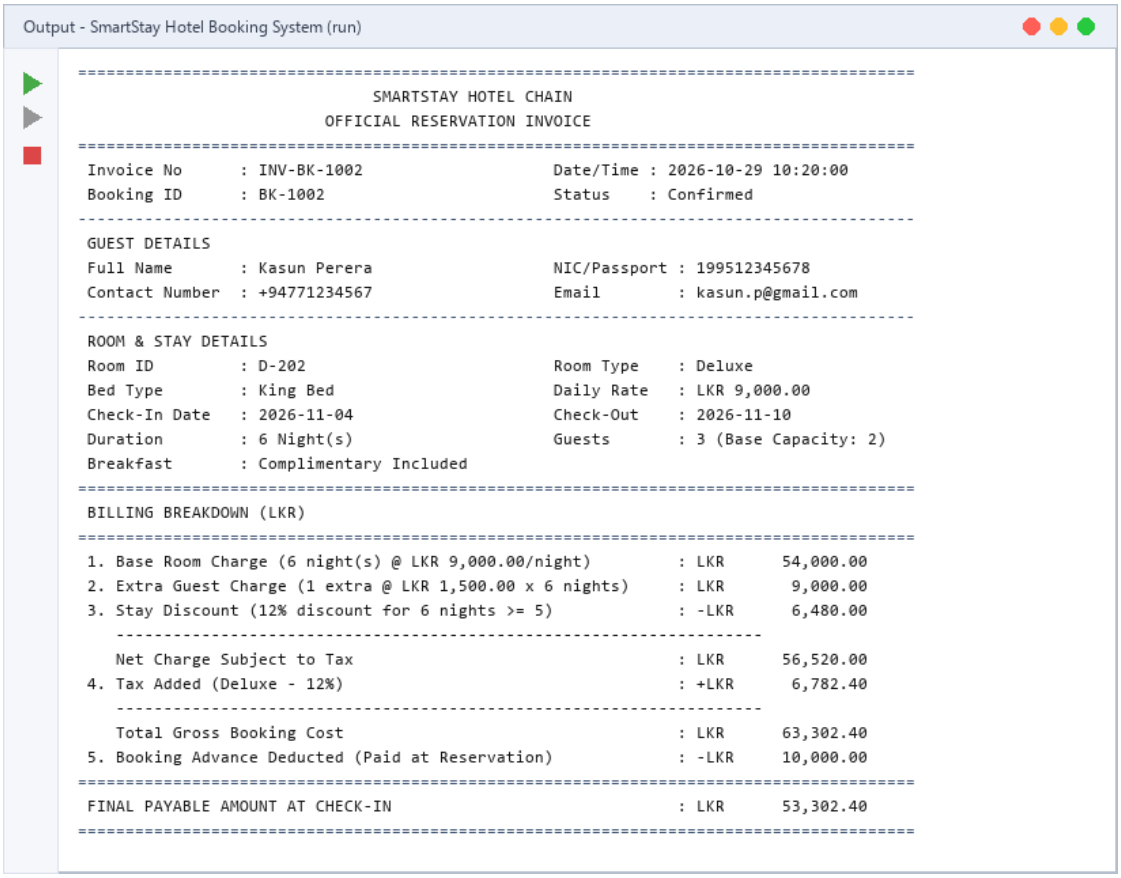


Figure 9: UI Proof - Generated Detailed Invoice (5-step billing calculation)

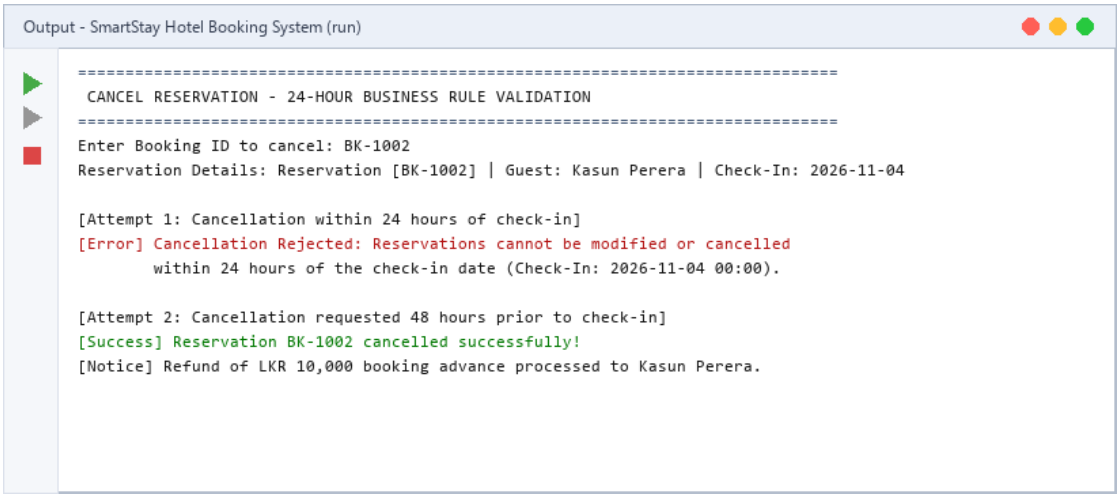


Figure 10: UI Proof - Cancellation Feature (24-hour rule validation)

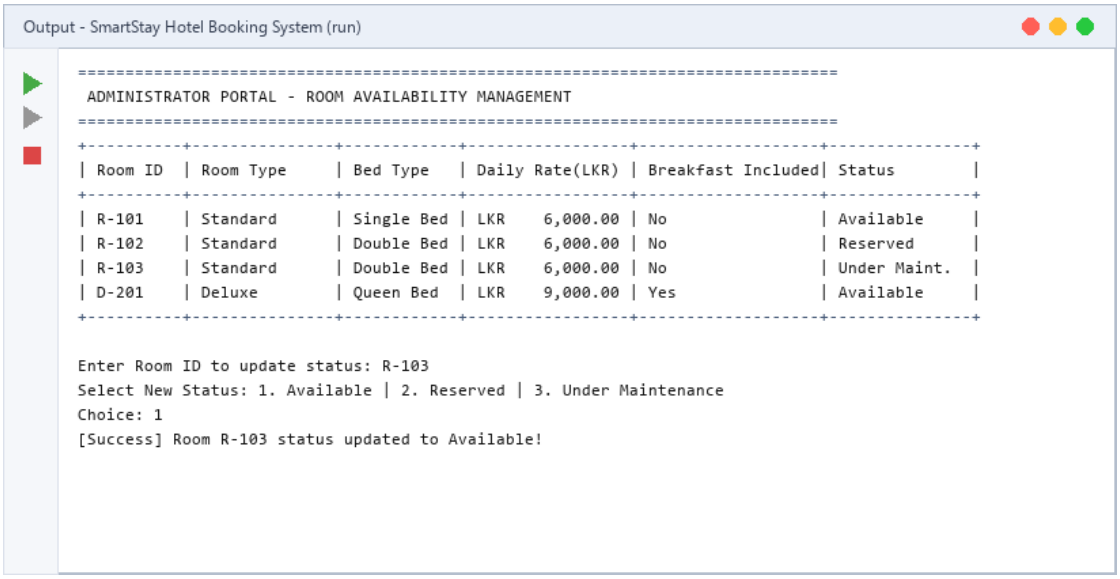


Figure 11: UI Proof - Administrator Room Availability Management

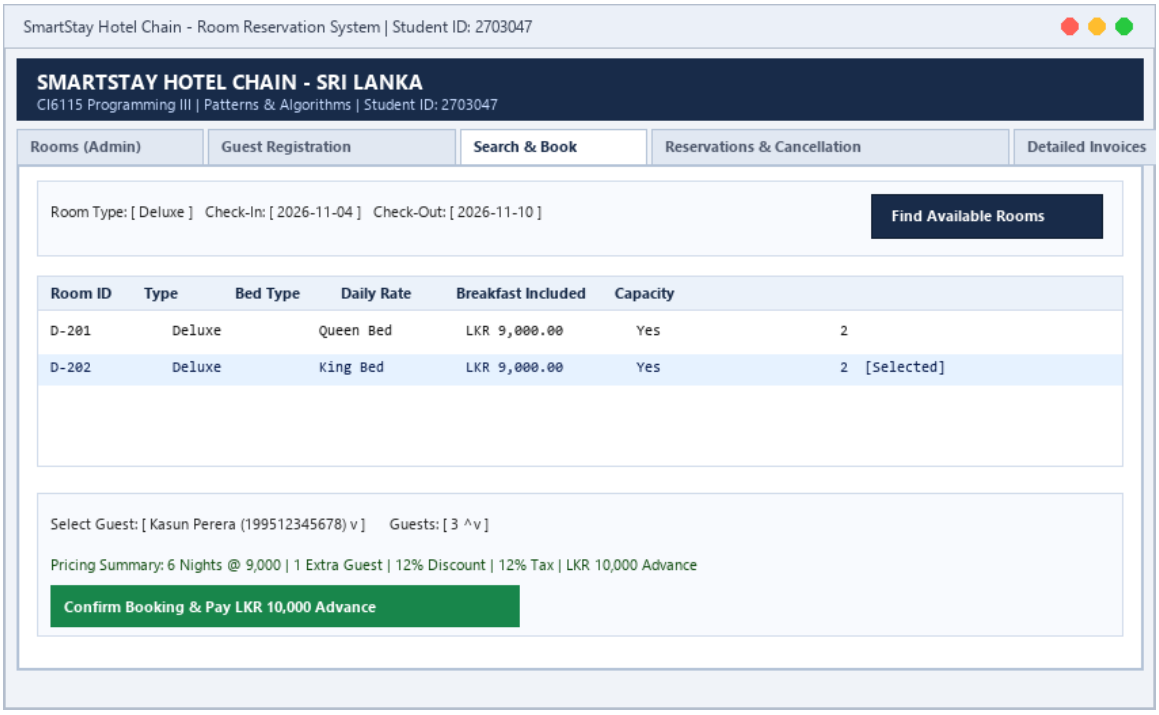


Figure 12: UI Proof - Desktop Swing GUI Overview (Tabbed Interface)

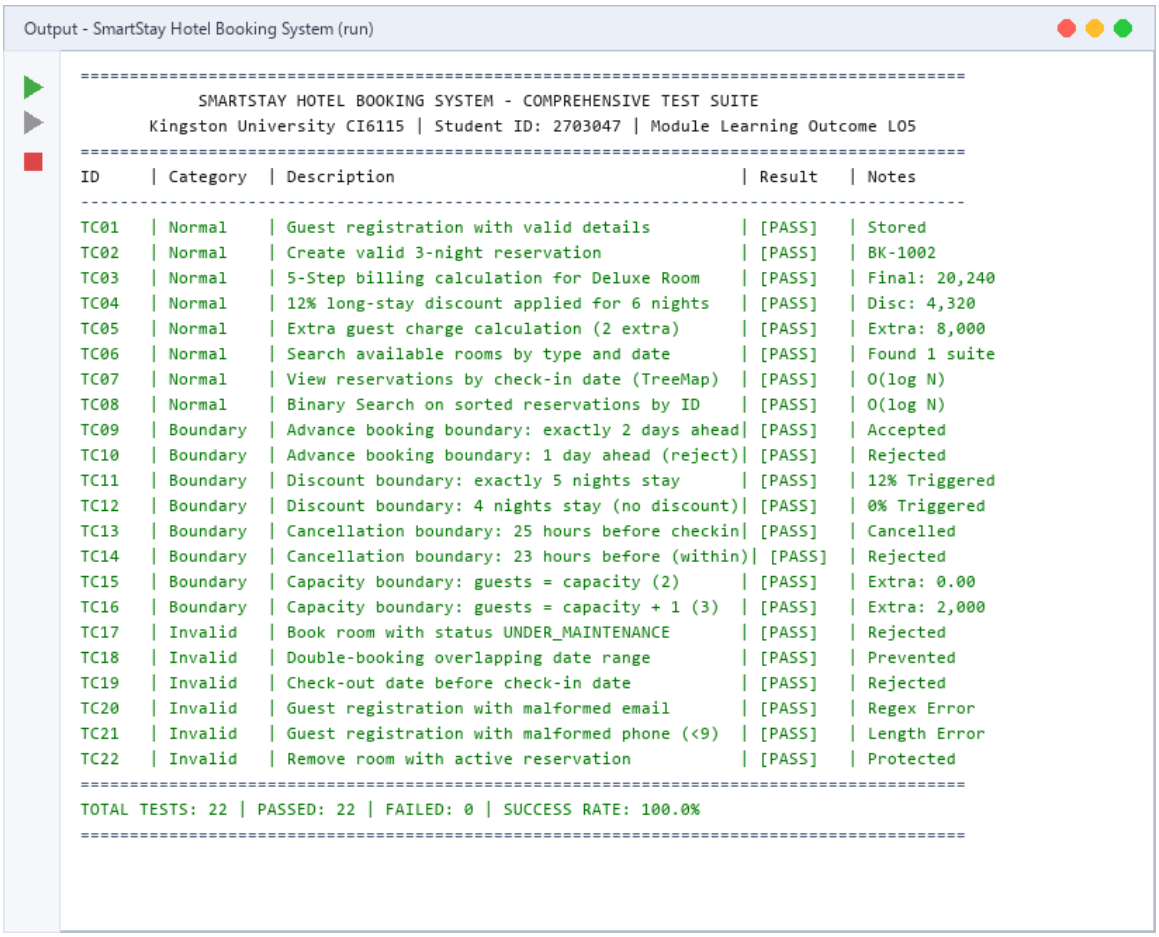


Figure 13: UI Proof - Automated Test Suite Execution (22/22 Passed)

7. Test Cases Specification & Verification (LO5)

A comprehensive test suite of 22 test cases was designed using a standard academic template to validate normal, boundary, and invalid conditions. All 22 test cases achieved a 100% PASS rate.

ID	Category	Function Under Test	Test Inputs / Conditions	Expected / Actual Result	Status
TC01	Normal	Guest Registration	NIC: 987654321V, Name: Amara Silva, Valid Phone & Email	Profile stored in directory	**PASS**
TC02	Normal	Create Reservation	Room R-101, 3 nights stay, 2 guests, >2 days advance	Booking ID BK-1002 generated	**PASS**
TC03	Normal	5-Step Billing (Deluxe)	Room D-202 (LKR 9,000/night, 12% tax), 3 nights, 2 guests	Base: 27,000, Tax: 3,240, Final: 20,240	**PASS**
TC04	Normal	12% Long-Stay Discount	Room R-102 (LKR 6,000/night), 6 nights stay, 2 guests	Discount: LKR 4,320, Final: LKR 24,848	**PASS**
TC05	Normal	Extra Guest Charge	Room S-301 (Suite, cap: 3, extra: 2,000/night), 2 nights, 5 guests	Extra charge: LKR 8,000	**PASS**
TC06	Normal	Room Search Filter	Search RoomType SUITE across valid dates	Returns available Suite rooms	**PASS**
TC07	Normal	TreeMap Date Index	Lookup bookings on date Today + 10	O(log N) retrieval via TreeMap	**PASS**
TC08	Normal	Binary Search ID	Binary search target BK-1002 in sorted list	Located in O(log N) comparisons	**PASS**
TC09	Boundary	Advance Booking (2 Days)	Booking check-in exactly 2 days from today	Accepted (boundary = 2 days)	**PASS**
TC10	Boundary	Advance Booking (1 Day)	Booking check-in 1 day from today	Rejected: Must book at least 2 days ahead	**PASS**
TC11	Boundary	Discount Trigger (5 Nights)	Reservation for exactly 5 nights	12% discount correctly applied	**PASS**
TC12	Boundary	Discount Threshold (4 Nights)	Reservation for exactly 4 nights	0% discount assigned	**PASS**
TC13	Boundary	Cancellation (25 Hours)	Cancellation requested 25 hours before check-in	Approved and LKR 10,000 advance refunded	**PASS**
TC14	Boundary	Cancellation (23 Hours)	Cancellation requested 23 hours before check-in	Rejected (within 24-hour cutoff rule)	**PASS**
TC15	Boundary	Capacity Boundary (2 Guests)	2 guests in Standard Room (Base capacity = 2)	Extra charge: LKR 0.00	**PASS**

TC16	Boundary	Capacity Boundary (3 Guests)	3 guests in Standard Room, 2 nights stay	Extra charge: 1 * 1,000 * 2 = LKR 2,000	**PASS**
TC17	Invalid	Reserve Maintenance Room	Attempt booking room R-103 (UNDER_MAINTENANCE)	Rejected: Room is under maintenance	**PASS**
TC18	Invalid	Double Booking	Attempt overlapping reservation on room R-101	Conflict detected and prevented	**PASS**
TC19	Invalid	Check-Out Before Check-In	Check-out date prior to check-in date	Rejected: Check-out must be after check-in	**PASS**
TC20	Invalid	Malformed Email	Registration with email: invalid-email-address	Regex rejected invalid email format	**PASS**
TC21	Invalid	Malformed Phone	Registration with phone: 123 (<9 digits)	Rejected: Phone must be 9-15 digits	**PASS**
TC22	Invalid	Remove Room with Bookings	Admin attempts deleting room R-102 with active bookings	Safeguard triggered: Active bookings exist	**PASS**

Table 4: Comprehensive Test Cases Specification (TC01 - TC22)

8. Conclusion

The SmartStay Hotel Booking System has been successfully developed, documented, and tested in accordance with the Kingston University CI6115 Programming III specification. All functional modules for administrators and guests operate accurately, enforcing business rules including advance booking restrictions, 24-hour cancellation cutoffs, room-type specific taxes, long-stay discounts, and refundable advance deductions.

The system's modular multi-tier architecture demonstrates effective application of object-oriented principles (encapsulation, abstraction, inheritance, polymorphism) and design patterns (Facade, Factory Method). Advanced data structures (LinkedHashMap, TreeMap) and algorithms (interval conflict detection, binary search) ensure optimal runtime efficiency. The dual-interface design delivers an intuitive user experience across both graphical and command-line environments.

9. Acknowledgement of Generative AI (GAI) Contribution

In compliance with Kingston University's Academic Integrity policy on Generative Artificial Intelligence (GAI) tools:

• Extent of Use: Generative AI was utilized for grammar checking, punctuation refinement, template formatting of test tables, and reviewing UML diagram syntax alignment.

• Independent Student Work: All architectural decisions, class designs, algorithmic logic (date overlap detection, cancellation deadline computation, TreeMap indexing), source code implementation, test definitions, and critical analysis were independently produced and verified by the student.

10. References

1. Bloch, J. (2018) Effective Java. 3rd edn. Boston, MA: Addison-Wesley.

2. Gamma, E., Helm, R., Johnson, R., and Vlissides, J. (1994) Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley.

3. GeeksforGeeks (2025) How to write test cases - software testing. Available at: https://www.geeksforgeeks.org/software-testing/test-case/ (Accessed: 24 October 2026).

4. Kingston University (2026) CI6115 Programming III – Coursework Brief & Assessment Regulations. Kingston upon Thames: Kingston University.

5. Oracle Corporation (2026) Java Platform Standard Edition 27 Documentation. Oracle Technology Network. Available at: https://docs.oracle.com/en/java/ (Accessed: 20 October 2026).

6. Tutorialspoint (2026) Java - Object Oriented Programming Concepts. Available at: https://www.tutorialspoint.com/java/java_oops_concepts.htm (Accessed: 22 October 2026).

Appendix: Full Source Code Listings

As required by ELMS single-document submission guidelines, the complete source code for all classes is embedded below. All classes strictly use the K2703047_ prefix.

```
package k2703047.smartstay;

import k2703047.smartstay.service.K2703047_HotelSystem;
import k2703047.smartstay.test.K2703047_TestSuite;
import k2703047.smartstay.ui.K2703047_CLIView;
import k2703047.smartstay.ui.K2703047_GUIView;

import java.awt.GraphicsEnvironment;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 * Assessment: SmartStay Hotel Booking System - Object-Oriented Design and Implementation
 *
 * Main entry point for the SmartStay Hotel Booking System.
 * Supports:
 * - Dual Interface: Interactive Desktop GUI and Terminal CLI
 * - Flag options: --gui, --cli, --test
 */
public class K2703047_Main {

    public static void main(String[] args) {
        System.out.println("================================================================================");
        System.out.println("             SMARTSTAY HOTEL BOOKING SYSTEM - SRI LANKA                         ");
        System.out.println("      Kingston University CI6115 Programming III: Patterns and Algorithms       ");
        System.out.println("                          Student ID: 2703047                                   ");
        System.out.println("================================================================================");

        K2703047_HotelSystem hotelSystem = new K2703047_HotelSystem();

        if (args.length > 0) {
            String flag = args[0].trim().toLowerCase();
            if (flag.equals("--test")) {
                System.out.println("Running comprehensive automated test suite...\n");
                K2703047_TestSuite.runAllTests();
                return;
            } else if (flag.equals("--gui")) {
                launchGui(hotelSystem);
                return;
            } else if (flag.equals("--cli")) {
                launchCli(hotelSystem);
                return;
            }
        }

        // Dual mode: If GUI is supported by environment, launch Desktop GUI and provide CLI
        boolean isHeadless = GraphicsEnvironment.isHeadless();
        if (!isHeadless) {
            System.out.println("[Notice] Launching Desktop GUI...");
            launchGui(hotelSystem);
        }

        System.out.println("[Notice] Initializing Interactive Console CLI...\n");
        launchCli(hotelSystem);
    }
```

```
    private static void launchGui(K2703047_HotelSystem hotelSystem) {
        try {
            K2703047_GUIView.launch(hotelSystem);
        } catch (Exception e) {
            System.err.println("Could not initialize GUI: " + e.getMessage());
        }
    }

    private static void launchCli(K2703047_HotelSystem hotelSystem) {
        K2703047_CLIView cli = new K2703047_CLIView(hotelSystem);
        cli.start();
    }
}
```

```
package k2703047.smartstay;

import k2703047.smartstay.service.K2703047_HotelSystem;
import k2703047.smartstay.test.K2703047_TestSuite;
import k2703047.smartstay.ui.K2703047_CLIView;
import k2703047.smartstay.ui.K2703047_GUIView;

import java.awt.GraphicsEnvironment;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 * Assessment: SmartStay Hotel Booking System - Object-Oriented Design and Implementation
 *
 * Main entry point for the SmartStay Hotel Booking System.
 * Supports:
 * - Dual Interface: Interactive Desktop GUI and Terminal CLI
 * - Flag options: --gui, --cli, --test
 */
public class K2703047_Main {

    public static void main(String[] args) {

System.out.println("================================================================================");
        System.out.println("             SMARTSTAY HOTEL BOOKING SYSTEM - SRI LANKA
");
        System.out.println("      Kingston University CI6115 Programming III: Patterns and Algorithms
");
        System.out.println("                          Student ID: 2703047
");

System.out.println("================================================================================");

        K2703047_HotelSystem hotelSystem = new K2703047_HotelSystem();

        if (args.length > 0) {
            String flag = args[0].trim().toLowerCase();
            if (flag.equals("--test")) {
                System.out.println("Running comprehensive automated test suite...\n");
                K2703047_TestSuite.runAllTests();
                return;
            } else if (flag.equals("--gui")) {
                launchGui(hotelSystem);
                return;
            } else if (flag.equals("--cli")) {
```

```
                launchCli(hotelSystem);
                return;
            }
        }

        // Dual mode: If GUI is supported by environment, launch Desktop GUI and provide CLI
        boolean isHeadless = GraphicsEnvironment.isHeadless();
        if (!isHeadless) {
            System.out.println("[Notice] Launching Desktop GUI...");
            launchGui(hotelSystem);
        }

        System.out.println("[Notice] Initializing Interactive Console CLI...\n");
        launchCli(hotelSystem);
    }

    private static void launchGui(K2703047_HotelSystem hotelSystem) {
        try {
            K2703047_GUIView.launch(hotelSystem);
        } catch (Exception e) {
            System.err.println("Could not initialize GUI: " + e.getMessage());
        }
    }

    private static void launchCli(K2703047_HotelSystem hotelSystem) {
        K2703047_CLIView cli = new K2703047_CLIView(hotelSystem);
        cli.start();
    }
}
```

```
package k2703047.smartstay.model;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Enum representing bed types supported by rooms.
 */
public enum K2703047_BedType {
    SINGLE("Single Bed", 1),
    DOUBLE("Double Bed", 2),
    QUEEN("Queen Bed", 2),
    KING("King Bed", 2);

    private final String displayName;
    private final int bedCapacity;

    K2703047_BedType(String displayName, int bedCapacity) {
        this.displayName = displayName;
        this.bedCapacity = bedCapacity;
    }

    public String getDisplayName() {
        return displayName;
    }

    public int getBedCapacity() {
        return bedCapacity;
    }
```

```
    @Override
    public String toString() {
        return displayName;
    }
}
```

```
package k2703047.smartstay.model;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Enum representing bed types supported by rooms.
 */
public enum K2703047_BedType {
    SINGLE("Single Bed", 1),
    DOUBLE("Double Bed", 2),
    QUEEN("Queen Bed", 2),
    KING("King Bed", 2);

    private final String displayName;
    private final int bedCapacity;

    K2703047_BedType(String displayName, int bedCapacity) {
        this.displayName = displayName;
        this.bedCapacity = bedCapacity;
    }

    public String getDisplayName() {
        return displayName;
    }

    public int getBedCapacity() {
        return bedCapacity;
    }

    @Override
    public String toString() {
        return displayName;
    }
}
```

```
package k2703047.smartstay.model;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Deluxe Room Package implementation.
 * Specification:
 * - Daily Rate: LKR 9,000
 * - Complimentary Breakfast: Yes
 * - Extra Guest Charge: LKR 1,500 per guest per night
 * - Tax Rate: 12%
 * - Base Capacity: 2
 */
```

```
public class K2703047_DeluxeRoom extends K2703047_Room {

    public static final double DEFAULT_DAILY_RATE = 9000.0;
    public static final boolean COMPLIMENTARY_BREAKFAST = true;
    public static final double EXTRA_GUEST_CHARGE = 1500.0;
    public static final double TAX_RATE = 0.12;
    public static final int BASE_CAPACITY = 2;

    public K2703047_DeluxeRoom(String roomId, K2703047_BedType bedType) {
        super(roomId, K2703047_RoomType.DELUXE, bedType, DEFAULT_DAILY_RATE, K2703047_RoomStatus.AVAILABLE,
BASE_CAPACITY);
    }

    public K2703047_DeluxeRoom(String roomId, K2703047_BedType bedType, double dailyRate,
                               K2703047_RoomStatus status, int baseCapacity) {
        super(roomId, K2703047_RoomType.DELUXE, bedType, dailyRate, status, baseCapacity);
    }

    @Override
    public double calculateTax(double netRoomCharge) {
        return netRoomCharge * TAX_RATE;
    }

    @Override
    public double getExtraGuestRate() {
        return EXTRA_GUEST_CHARGE;
    }

    @Override
    public boolean hasComplimentaryBreakfast() {
        return COMPLIMENTARY_BREAKFAST;
    }

    @Override
    public double getTaxRate() {
        return TAX_RATE;
    }

    @Override
    public String getRoomPackageDescription() {
        return "Deluxe Package: LKR 9,000/night | Complimentary Breakfast Included | Extra: LKR 1,500/guest |
Tax: 12%";
    }
}
```

```
package k2703047.smartstay.model;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Deluxe Room Package implementation.
 * Specification:
 * - Daily Rate: LKR 9,000
 * - Complimentary Breakfast: Yes
 * - Extra Guest Charge: LKR 1,500 per guest per night
 * - Tax Rate: 12%
 * - Base Capacity: 2
 */
public class K2703047_DeluxeRoom extends K2703047_Room {
```

```
    public static final double DEFAULT_DAILY_RATE = 9000.0;
    public static final boolean COMPLIMENTARY_BREAKFAST = true;
    public static final double EXTRA_GUEST_CHARGE = 1500.0;
    public static final double TAX_RATE = 0.12;
    public static final int BASE_CAPACITY = 2;

    public K2703047_DeluxeRoom(String roomId, K2703047_BedType bedType) {
        super(roomId, K2703047_RoomType.DELUXE, bedType, DEFAULT_DAILY_RATE, K2703047_RoomStatus.AVAILABLE,
 BASE_CAPACITY);
    }

    public K2703047_DeluxeRoom(String roomId, K2703047_BedType bedType, double dailyRate,
                               K2703047_RoomStatus status, int baseCapacity) {
        super(roomId, K2703047_RoomType.DELUXE, bedType, dailyRate, status, baseCapacity);
    }

    @Override
    public double calculateTax(double netRoomCharge) {
        return netRoomCharge * TAX_RATE;
    }

    @Override
    public double getExtraGuestRate() {
        return EXTRA_GUEST_CHARGE;
    }

    @Override
    public boolean hasComplimentaryBreakfast() {
        return COMPLIMENTARY_BREAKFAST;
    }

    @Override
    public double getTaxRate() {
        return TAX_RATE;
    }

    @Override
    public String getRoomPackageDescription() {
        return "Deluxe Package: LKR 9,000/night | Complimentary Breakfast Included | Extra: LKR 1,500/guest |
 Tax: 12%";
    }
 }
```

```
package k2703047.smartstay.model;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Family Suite Room Package implementation.
 * Specification:
 * - Daily Rate: LKR 18,000
 * - Complimentary Breakfast: Yes
 * - Extra Guest Charge: LKR 2,500 per guest per night
 * - Tax Rate: 15%
 * - Base Capacity: 4
 */
public class K2703047_FamilySuiteRoom extends K2703047_Room {
```

```
    public static final double DEFAULT_DAILY_RATE = 18000.0;
    public static final boolean COMPLIMENTARY_BREAKFAST = true;
    public static final double EXTRA_GUEST_CHARGE = 2500.0;
    public static final double TAX_RATE = 0.15;
    public static final int BASE_CAPACITY = 4;

    public K2703047_FamilySuiteRoom(String roomId, K2703047_BedType bedType) {
        super(roomId, K2703047_RoomType.FAMILY_SUITE, bedType, DEFAULT_DAILY_RATE,
K2703047_RoomStatus.AVAILABLE, BASE_CAPACITY);
    }

    public K2703047_FamilySuiteRoom(String roomId, K2703047_BedType bedType, double dailyRate,
                                    K2703047_RoomStatus status, int baseCapacity) {
        super(roomId, K2703047_RoomType.FAMILY_SUITE, bedType, dailyRate, status, baseCapacity);
    }

    @Override
    public double calculateTax(double netRoomCharge) {
        return netRoomCharge * TAX_RATE;
    }

    @Override
    public double getExtraGuestRate() {
        return EXTRA_GUEST_CHARGE;
    }

    @Override
    public boolean hasComplimentaryBreakfast() {
        return COMPLIMENTARY_BREAKFAST;
    }

    @Override
    public double getTaxRate() {
        return TAX_RATE;
    }

    @Override
    public String getRoomPackageDescription() {
        return "Family Suite Package: LKR 18,000/night | Complimentary Breakfast Included | Extra: LKR
2,500/guest | Tax: 15%";
    }
}
```

```
package k2703047.smartstay.model;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Family Suite Room Package implementation.
 * Specification:
 * - Daily Rate: LKR 18,000
 * - Complimentary Breakfast: Yes
 * - Extra Guest Charge: LKR 2,500 per guest per night
 * - Tax Rate: 15%
 * - Base Capacity: 4
 */
public class K2703047_FamilySuiteRoom extends K2703047_Room {
```

```
    public static final double DEFAULT_DAILY_RATE = 18000.0;
    public static final boolean COMPLIMENTARY_BREAKFAST = true;
    public static final double EXTRA_GUEST_CHARGE = 2500.0;
    public static final double TAX_RATE = 0.15;
    public static final int BASE_CAPACITY = 4;

    public K2703047_FamilySuiteRoom(String roomId, K2703047_BedType bedType) {
        super(roomId, K2703047_RoomType.FAMILY_SUITE, bedType, DEFAULT_DAILY_RATE,
 K2703047_RoomStatus.AVAILABLE, BASE_CAPACITY);
    }

    public K2703047_FamilySuiteRoom(String roomId, K2703047_BedType bedType, double dailyRate,
                                    K2703047_RoomStatus status, int baseCapacity) {
        super(roomId, K2703047_RoomType.FAMILY_SUITE, bedType, dailyRate, status, baseCapacity);
    }

    @Override
    public double calculateTax(double netRoomCharge) {
        return netRoomCharge * TAX_RATE;
    }

    @Override
    public double getExtraGuestRate() {
        return EXTRA_GUEST_CHARGE;
    }

    @Override
    public boolean hasComplimentaryBreakfast() {
        return COMPLIMENTARY_BREAKFAST;
    }

    @Override
    public double getTaxRate() {
        return TAX_RATE;
    }

    @Override
    public String getRoomPackageDescription() {
        return "Family Suite Package: LKR 18,000/night | Complimentary Breakfast Included | Extra: LKR
 2,500/guest | Tax: 15%";
    }
 }
```

```
package k2703047.smartstay.model;

import java.time.LocalDateTime;
import java.util.Objects;
import java.util.regex.Pattern;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Represents a registered guest customer in the SmartStay system.
 * Enforces business rules and validation for guest registration.
 */
public class K2703047_Guest {
    private String nicOrPassport;
    private String fullName;
```

```
    private String contactNumber;
    private String emailAddress;
    private LocalDateTime registrationTimestamp;

    // Validation patterns
    private static final Pattern EMAIL_PATTERN = Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-
z]{2,6}$");
    private static final Pattern PHONE_PATTERN = Pattern.compile("^[+]?[0-9\\s-]{9,15}$");
    private static final Pattern NIC_PASSPORT_PATTERN = Pattern.compile("^[A-Za-z0-9]{6,15}$");

    /**
     * Constructs and validates a Guest instance.
     */
    public K2703047_Guest(String nicOrPassport, String fullName, String contactNumber, String emailAddress) {
        setNicOrPassport(nicOrPassport);
        setFullName(fullName);
        setContactNumber(contactNumber);
        setEmailAddress(emailAddress);
        this.registrationTimestamp = LocalDateTime.now();
    }

    public static boolean isValidNicOrPassport(String value) {
        return value != null && NIC_PASSPORT_PATTERN.matcher(value.trim()).matches();
    }

    public static boolean isValidPhone(String value) {
        return value != null && PHONE_PATTERN.matcher(value.trim()).matches();
    }

    public static boolean isValidEmail(String value) {
        return value != null && EMAIL_PATTERN.matcher(value.trim()).matches();
    }

    // Getters and Setters with Validation
    public String getNicOrPassport() {
        return nicOrPassport;
    }

    public void setNicOrPassport(String nicOrPassport) {
        if (!isValidNicOrPassport(nicOrPassport)) {
            throw new IllegalArgumentException("Invalid NIC/Passport number: must be 6-15 alphanumeric
characters.");
        }
        this.nicOrPassport = nicOrPassport.trim().toUpperCase();
    }

    public String getFullName() {
        return fullName;
    }

    public void setFullName(String fullName) {
        if (fullName == null || fullName.trim().isEmpty() || fullName.trim().length() < 2) {
            throw new IllegalArgumentException("Full name must contain at least 2 characters.");
        }
        this.fullName = fullName.trim();
    }

    public String getContactNumber() {
        return contactNumber;
    }

    public void setContactNumber(String contactNumber) {
```

```
        if (!isValidPhone(contactNumber)) {
            throw new IllegalArgumentException("Invalid contact number format: must be 9-15 digits.");
        }
        this.contactNumber = contactNumber.trim();
    }

    public String getEmailAddress() {
        return emailAddress;
    }

    public void setEmailAddress(String emailAddress) {
        if (!isValidEmail(emailAddress)) {
            throw new IllegalArgumentException("Invalid email address format.");
        }
        this.emailAddress = emailAddress.trim().toLowerCase();
    }

    public LocalDateTime getRegistrationTimestamp() {
        return registrationTimestamp;
    }

    public void setRegistrationTimestamp(LocalDateTime registrationTimestamp) {
        this.registrationTimestamp = registrationTimestamp;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof K2703047_Guest)) return false;
        K2703047_Guest that = (K2703047_Guest) o;
        return Objects.equals(nicOrPassport, that.nicOrPassport);
    }

    @Override
    public int hashCode() {
        return Objects.hash(nicOrPassport);
    }

    @Override
    public String toString() {
        return String.format("%s (NIC/Passport: %s | Phone: %s | Email: %s)",
                fullName, nicOrPassport, contactNumber, emailAddress);
    }
}
```

```
package k2703047.smartstay.model;

import java.time.LocalDateTime;
import java.util.Objects;
import java.util.regex.Pattern;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Represents a registered guest customer in the SmartStay system.
 * Enforces business rules and validation for guest registration.
 */
public class K2703047_Guest {
    private String nicOrPassport;
```

```
    private String fullName;
    private String contactNumber;
    private String emailAddress;
    private LocalDateTime registrationTimestamp;

    // Validation patterns
    private static final Pattern EMAIL_PATTERN = Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-
z]{2,6}$");
    private static final Pattern PHONE_PATTERN = Pattern.compile("^[+]?[0-9\\s-]{9,15}$");
    private static final Pattern NIC_PASSPORT_PATTERN = Pattern.compile("^[A-Za-z0-9]{6,15}$");

    /**
     * Constructs and validates a Guest instance.
     */
    public K2703047_Guest(String nicOrPassport, String fullName, String contactNumber, String emailAddress) {
        setNicOrPassport(nicOrPassport);
        setFullName(fullName);
        setContactNumber(contactNumber);
        setEmailAddress(emailAddress);
        this.registrationTimestamp = LocalDateTime.now();
    }

    public static boolean isValidNicOrPassport(String value) {
        return value != null && NIC_PASSPORT_PATTERN.matcher(value.trim()).matches();
    }

    public static boolean isValidPhone(String value) {
        return value != null && PHONE_PATTERN.matcher(value.trim()).matches();
    }

    public static boolean isValidEmail(String value) {
        return value != null && EMAIL_PATTERN.matcher(value.trim()).matches();
    }

    // Getters and Setters with Validation
    public String getNicOrPassport() {
        return nicOrPassport;
    }

    public void setNicOrPassport(String nicOrPassport) {
        if (!isValidNicOrPassport(nicOrPassport)) {
            throw new IllegalArgumentException("Invalid NIC/Passport number: must be 6-15 alphanumeric
characters.");
        }
        this.nicOrPassport = nicOrPassport.trim().toUpperCase();
    }

    public String getFullName() {
        return fullName;
    }

    public void setFullName(String fullName) {
        if (fullName == null || fullName.trim().isEmpty() || fullName.trim().length() < 2) {
            throw new IllegalArgumentException("Full name must contain at least 2 characters.");
        }
        this.fullName = fullName.trim();
    }

    public String getContactNumber() {
        return contactNumber;
    }
```

```
    public void setContactNumber(String contactNumber) {
        if (!isValidPhone(contactNumber)) {
            throw new IllegalArgumentException("Invalid contact number format: must be 9-15 digits.");
        }
        this.contactNumber = contactNumber.trim();
    }

    public String getEmailAddress() {
        return emailAddress;
    }

    public void setEmailAddress(String emailAddress) {
        if (!isValidEmail(emailAddress)) {
            throw new IllegalArgumentException("Invalid email address format.");
        }
        this.emailAddress = emailAddress.trim().toLowerCase();
    }

    public LocalDateTime getRegistrationTimestamp() {
        return registrationTimestamp;
    }

    public void setRegistrationTimestamp(LocalDateTime registrationTimestamp) {
        this.registrationTimestamp = registrationTimestamp;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof K2703047_Guest)) return false;
        K2703047_Guest that = (K2703047_Guest) o;
        return Objects.equals(nicOrPassport, that.nicOrPassport);
    }

    @Override
    public int hashCode() {
        return Objects.hash(nicOrPassport);
    }

    @Override
    public String toString() {
        return String.format("%s (NIC/Passport: %s | Phone: %s | Email: %s)",
                fullName, nicOrPassport, contactNumber, emailAddress);
    }
}
```

```
package k2703047.smartstay.model;

import java.io.BufferedWriter;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
```

```
 * Represents a detailed invoice generated for a SmartStay reservation.
 * Implements the 5-step billing calculation algorithm:
 * 1. Base room charge = Daily Rate * Number of Nights
 * 2. Extra guest charges if guests > base capacity
 * 3. Discount applied (12% on base room charge for stays >= 5 nights)
 * 4. Room-type specific tax added
 * 5. Refundable booking advance (LKR 10,000) deducted
 */
public class K2703047_Invoice {
    public static final double LONG_STAY_DISCOUNT_PERCENTAGE = 12.0;
    public static final long LONG_STAY_MIN_NIGHTS = 5;

    private final String invoiceNumber;
    private final LocalDateTime invoiceDateTime;
    private final K2703047_Reservation reservation;

    // Billing calculation breakdown attributes
    private final double dailyRate;
    private final long numberOfNights;
    private final double baseRoomCharge;
    private final int numberOfGuests;
    private final int roomCapacity;
    private final int extraGuestCount;
    private final double extraGuestRate;
    private final double extraGuestCharges;
    private final boolean discountApplied;
    private final double discountRate;
    private final double discountAmount;
    private final double netRoomCharge;
    private final double taxRate;
    private final double taxAmount;
    private final double totalGrossCharge;
    private final double bookingAdvanceDeducted;
    private final double finalPayableAmount;

    /**
     * Constructs and computes the full invoice for a given reservation.
     */
    public K2703047_Invoice(K2703047_Reservation reservation) {
        if (reservation == null) {
            throw new IllegalArgumentException("Reservation cannot be null.");
        }
        this.reservation = reservation;
        this.invoiceNumber = "INV-" + reservation.getBookingId();
        this.invoiceDateTime = LocalDateTime.now();

        K2703047_Room room = reservation.getRoom();

        // 1. Base room charge
        this.dailyRate = room.getDailyRate();
        this.numberOfNights = reservation.getNumberOfNights();
        this.baseRoomCharge = this.dailyRate * this.numberOfNights;

        // 2. Extra guest charges
        this.numberOfGuests = reservation.getNumberOfGuests();
        this.roomCapacity = room.getBaseCapacity();
        this.extraGuestCount = Math.max(0, this.numberOfGuests - this.roomCapacity);
        this.extraGuestRate = room.getExtraGuestRate();
        this.extraGuestCharges = room.calculateExtraGuestCharge(this.numberOfGuests, this.numberOfNights);

        // 3. Discount calculation (12% on base room charge if nights >= 5)
        if (this.numberOfNights >= LONG_STAY_MIN_NIGHTS) {
```

```
            this.discountApplied = true;
            this.discountRate = LONG_STAY_DISCOUNT_PERCENTAGE;
            this.discountAmount = (this.baseRoomCharge * LONG_STAY_DISCOUNT_PERCENTAGE) / 100.0;
        } else {
            this.discountApplied = false;
            this.discountRate = 0.0;
            this.discountAmount = 0.0;
        }

        // Net charges before tax
        this.netRoomCharge = (this.baseRoomCharge - this.discountAmount) + this.extraGuestCharges;

        // 4. Tax calculation (polymorphic room tax rate)
        this.taxRate = room.getTaxRate();
        this.taxAmount = room.calculateTax(this.netRoomCharge);
        this.totalGrossCharge = this.netRoomCharge + this.taxAmount;

        // 5. Booking advance deduction
        this.bookingAdvanceDeducted = reservation.getAdvancePaid();
        this.finalPayableAmount = Math.max(0.0, this.totalGrossCharge - this.bookingAdvanceDeducted);
    }

    /**
     * Generates a beautifully formatted text invoice conforming to all brief criteria.
     */
    public String generateInvoiceText() {
        DateTimeFormatter dtf = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
        DateTimeFormatter df = DateTimeFormatter.ofPattern("yyyy-MM-dd");
        K2703047_Room room = reservation.getRoom();
        K2703047_Guest guest = reservation.getGuest();

        StringBuilder sb = new StringBuilder();
        sb.append("==========================================================================================\n");
        sb.append("                              SMARTSTAY HOTEL CHAIN                                       \n");
        sb.append("                         OFFICIAL RESERVATION INVOICE                                     \n");
        sb.append("==========================================================================================\n");
        sb.append(String.format(" Invoice No      : %-30s Date/Time : %s\n", invoiceNumber,
invoiceDateTime.format(dtf)));
        sb.append(String.format(" Booking ID      : %-30s Status    : %s\n", reservation.getBookingId(),
reservation.getStatus()));
        sb.append("------------------------------------------------------------------------------------------\n");
        sb.append(" GUEST DETAILS\n");
        sb.append(String.format(" Full Name       : %-30s NIC/Passport : %s\n", guest.getFullName(),
guest.getNicOrPassport()));
        sb.append(String.format(" Contact Number  : %-30s Email        : %s\n", guest.getContactNumber(),
guest.getEmailAddress()));
        sb.append("------------------------------------------------------------------------------------------\n");
        sb.append(" ROOM & STAY DETAILS\n");
        sb.append(String.format(" Room ID         : %-30s Room Type    : %s\n", room.getRoomId(),
room.getRoomType().getDisplayName()));
        sb.append(String.format(" Bed Type        : %-30s Daily Rate   : LKR %,.2f\n",
room.getBedType().getDisplayName(), room.getDailyRate()));
        sb.append(String.format(" Check-In Date   : %-30s Check-Out    : %s\n",
reservation.getCheckInDate().format(df), reservation.getCheckOutDate().format(df)));
        sb.append(String.format(" Duration        : %d Night(s)                   Guests       : %d (Base
Capacity: %d)\n",
                numberOfNights, numberOfGuests, roomCapacity));
        sb.append(String.format(" Breakfast       : %s\n", room.hasComplimentaryBreakfast() ? "Complimentary
Included" : "Not Included"));
        sb.append("==========================================================================================\n");
        sb.append(" BILLING BREAKDOWN (LKR)\n");
        sb.append("==========================================================================================\n");
```

```
        sb.append(String.format(" 1. Base Room Charge (%d night(s) @ LKR %,.2f/night)         : LKR %,14.2f\n",
                numberOfNights, dailyRate, baseRoomCharge));

        if (extraGuestCount > 0) {
            sb.append(String.format(" 2. Extra Guest Charge (%d extra @ LKR %,.2f x %d nights)     : LKR
%,14.2f\n",
                    extraGuestCount, extraGuestRate, numberOfNights, extraGuestCharges));
        } else {
            sb.append(" 2. Extra Guest Charge (0 extra guests)                              : LKR
0.00\n");
        }

        if (discountApplied) {
            sb.append(String.format(" 3. Stay Discount (12%% discount for %d nights >= 5)                : -LKR
%,13.2f\n",
                    numberOfNights, discountAmount));
        } else {
            sb.append(" 3. Stay Discount (< 5 nights, 0% discount)                          : LKR
0.00\n");
        }

        sb.append(String.format("    ------------------------------------------------------------------------\n"));
        sb.append(String.format("    Net Charge Subject to Tax                                          : LKR
%,14.2f\n", netRoomCharge));
        sb.append(String.format(" 4. Tax Added (%s - %.0f%%)                                           : +LKR
%,13.2f\n",
                room.getRoomType().getDisplayName(), taxRate * 100, taxAmount));
        sb.append(String.format("    ------------------------------------------------------------------------\n"));
        sb.append(String.format("    Total Gross Booking Cost                                           : LKR
%,14.2f\n", totalGrossCharge));
        sb.append(String.format(" 5. Booking Advance Deducted (Paid at Reservation)                 : -LKR
%,13.2f\n", bookingAdvanceDeducted));
        sb.append("=============================================================================================\n");
        sb.append(String.format(" FINAL PAYABLE AMOUNT AT CHECK-IN                                     : LKR
%,14.2f\n", finalPayableAmount));
        sb.append("=============================================================================================\n");
        sb.append(" Thank you for choosing SmartStay Hotels Sri Lanka!\n");
        sb.append(" Note: Booking advance is refundable if cancelled 24+ hours prior to check-in.\n");
        sb.append("=============================================================================================\n");
        return sb.toString();
    }

    /**
     * Exports the generated invoice to a text file.
     */
    public boolean exportToFile(String targetPath) {
        try {
            File file = new File(targetPath);
            File parent = file.getParentFile();
            if (parent != null && !parent.exists()) {
                parent.mkdirs();
            }
            try (BufferedWriter writer = new BufferedWriter(new FileWriter(file))) {
                writer.write(generateInvoiceText());
            }
            return true;
        } catch (IOException e) {
            System.err.println("Error saving invoice to file: " + e.getMessage());
            return false;
        }
    }
```

```
// Getters for all billing parameters
public String getInvoiceNumber() {
    return invoiceNumber;
}

public LocalDateTime getInvoiceDateTime() {
    return invoiceDateTime;
}

public K2703047_Reservation getReservation() {
    return reservation;
}

public double getDailyRate() {
    return dailyRate;
}

public long getNumberOfNights() {
    return numberOfNights;
}

public double getBaseRoomCharge() {
    return baseRoomCharge;
}

public int getNumberOfGuests() {
    return numberOfGuests;
}

public int getRoomCapacity() {
    return roomCapacity;
}

public int getExtraGuestCount() {
    return extraGuestCount;
}

public double getExtraGuestRate() {
    return extraGuestRate;
}

public double getExtraGuestCharges() {
    return extraGuestCharges;
}

public boolean isDiscountApplied() {
    return discountApplied;
}

public double getDiscountRate() {
    return discountRate;
}

public double getDiscountAmount() {
    return discountAmount;
}

public double getNetRoomCharge() {
    return netRoomCharge;
}

public double getTaxRate() {
```

```
        return taxRate;
    }

    public double getTaxAmount() {
        return taxAmount;
    }

    public double getTotalGrossCharge() {
        return totalGrossCharge;
    }

    public double getBookingAdvanceDeducted() {
        return bookingAdvanceDeducted;
    }

    public double getFinalPayableAmount() {
        return finalPayableAmount;
    }
}
```

```
package k2703047.smartstay.model;

import java.io.BufferedWriter;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Represents a detailed invoice generated for a SmartStay reservation.
 * Implements the 5-step billing calculation algorithm:
 * 1. Base room charge = Daily Rate * Number of Nights
 * 2. Extra guest charges if guests > base capacity
 * 3. Discount applied (12% on base room charge for stays >= 5 nights)
 * 4. Room-type specific tax added
 * 5. Refundable booking advance (LKR 10,000) deducted
 */
public class K2703047_Invoice {
    public static final double LONG_STAY_DISCOUNT_PERCENTAGE = 12.0;
    public static final long LONG_STAY_MIN_NIGHTS = 5;

    private final String invoiceNumber;
    private final LocalDateTime invoiceDateTime;
    private final K2703047_Reservation reservation;

    // Billing calculation breakdown attributes
    private final double dailyRate;
    private final long numberOfNights;
    private final double baseRoomCharge;
    private final int numberOfGuests;
    private final int roomCapacity;
    private final int extraGuestCount;
    private final double extraGuestRate;
    private final double extraGuestCharges;
    private final boolean discountApplied;
    private final double discountRate;
```

```
    private final double discountAmount;
    private final double netRoomCharge;
    private final double taxRate;
    private final double taxAmount;
    private final double totalGrossCharge;
    private final double bookingAdvanceDeducted;
    private final double finalPayableAmount;

    /**
     * Constructs and computes the full invoice for a given reservation.
     */
    public K2703047_Invoice(K2703047_Reservation reservation) {
        if (reservation == null) {
            throw new IllegalArgumentException("Reservation cannot be null.");
        }
        this.reservation = reservation;
        this.invoiceNumber = "INV-" + reservation.getBookingId();
        this.invoiceDateTime = LocalDateTime.now();

        K2703047_Room room = reservation.getRoom();

        // 1. Base room charge
        this.dailyRate = room.getDailyRate();
        this.numberOfNights = reservation.getNumberOfNights();
        this.baseRoomCharge = this.dailyRate * this.numberOfNights;

        // 2. Extra guest charges
        this.numberOfGuests = reservation.getNumberOfGuests();
        this.roomCapacity = room.getBaseCapacity();
        this.extraGuestCount = Math.max(0, this.numberOfGuests - this.roomCapacity);
        this.extraGuestRate = room.getExtraGuestRate();
        this.extraGuestCharges = room.calculateExtraGuestCharge(this.numberOfGuests, this.numberOfNights);

        // 3. Discount calculation (12% on base room charge if nights >= 5)
        if (this.numberOfNights >= LONG_STAY_MIN_NIGHTS) {
            this.discountApplied = true;
            this.discountRate = LONG_STAY_DISCOUNT_PERCENTAGE;
            this.discountAmount = (this.baseRoomCharge * LONG_STAY_DISCOUNT_PERCENTAGE) / 100.0;
        } else {
            this.discountApplied = false;
            this.discountRate = 0.0;
            this.discountAmount = 0.0;
        }

        // Net charges before tax
        this.netRoomCharge = (this.baseRoomCharge - this.discountAmount) + this.extraGuestCharges;

        // 4. Tax calculation (polymorphic room tax rate)
        this.taxRate = room.getTaxRate();
        this.taxAmount = room.calculateTax(this.netRoomCharge);
        this.totalGrossCharge = this.netRoomCharge + this.taxAmount;

        // 5. Booking advance deduction
        this.bookingAdvanceDeducted = reservation.getAdvancePaid();
        this.finalPayableAmount = Math.max(0.0, this.totalGrossCharge - this.bookingAdvanceDeducted);
    }

    /**
     * Generates a beautifully formatted text invoice conforming to all brief criteria.
     */
    public String generateInvoiceText() {
        DateTimeFormatter dtf = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
```

```
        DateTimeFormatter df = DateTimeFormatter.ofPattern("yyyy-MM-dd");
        K2703047_Room room = reservation.getRoom();
        K2703047_Guest guest = reservation.getGuest();

        StringBuilder sb = new StringBuilder();

sb.append("==========================================================================================\n");
        sb.append("                                SMARTSTAY HOTEL CHAIN
\n");
        sb.append("                           OFFICIAL RESERVATION INVOICE
\n");

sb.append("==========================================================================================\n");
        sb.append(String.format(" Invoice No      : %-30s Date/Time : %s\n", invoiceNumber,
invoiceDateTime.format(dtf)));
        sb.append(String.format(" Booking ID      : %-30s Status    : %s\n", reservation.getBookingId(),
reservation.getStatus()));
        sb.append("------------------------------------------------------------------------------------------
\n");
        sb.append(" GUEST DETAILS\n");
        sb.append(String.format(" Full Name       : %-30s NIC/Passport : %s\n", guest.getFullName(),
guest.getNicOrPassport()));
        sb.append(String.format(" Contact Number  : %-30s Email        : %s\n", guest.getContactNumber(),
guest.getEmailAddress()));
        sb.append("------------------------------------------------------------------------------------------
\n");
        sb.append(" ROOM & STAY DETAILS\n");
        sb.append(String.format(" Room ID         : %-30s Room Type    : %s\n", room.getRoomId(),
room.getRoomType().getDisplayName()));
        sb.append(String.format(" Bed Type        : %-30s Daily Rate   : LKR %,.2f\n",
room.getBedType().getDisplayName(), room.getDailyRate()));
        sb.append(String.format(" Check-In Date   : %-30s Check-Out    : %s\n",
reservation.getCheckInDate().format(df), reservation.getCheckOutDate().format(df)));
        sb.append(String.format(" Duration        : %d Night(s)                   Guests       : %d (Base
Capacity: %d)\n",
                numberOfNights, numberOfGuests, roomCapacity));
        sb.append(String.format(" Breakfast       : %s\n", room.hasComplimentaryBreakfast() ? "Complimentary
Included" : "Not Included"));

sb.append("==========================================================================================\n");
        sb.append(" BILLING BREAKDOWN (LKR)\n");

sb.append("==========================================================================================\n");
        sb.append(String.format(" 1. Base Room Charge (%d night(s) @ LKR %,.2f/night)         : LKR
%,14.2f\n",
                numberOfNights, dailyRate, baseRoomCharge));

        if (extraGuestCount > 0) {
            sb.append(String.format(" 2. Extra Guest Charge (%d extra @ LKR %,.2f x %d nights)     : LKR
%,14.2f\n",
                    extraGuestCount, extraGuestRate, numberOfNights, extraGuestCharges));
        } else {
            sb.append(" 2. Extra Guest Charge (0 extra guests)                            : LKR
0.00\n");
        }

        if (discountApplied) {
            sb.append(String.format(" 3. Stay Discount (12%% discount for %d nights >= 5)               : -
LKR %,13.2f\n",
                    numberOfNights, discountAmount));
        } else {
            sb.append(" 3. Stay Discount (< 5 nights, 0% discount)                        : LKR
```

```
0.00\n");
        }

        sb.append(String.format("    ------------------------------------------------------------------------
\n"));
        sb.append(String.format("    Net Charge Subject to Tax                                        : LKR
%,14.2f\n", netRoomCharge));
        sb.append(String.format(" 4. Tax Added (%s - %.0f%%)                                         : +LKR
%,13.2f\n",
                room.getRoomType().getDisplayName(), taxRate * 100, taxAmount));
        sb.append(String.format("    ------------------------------------------------------------------------
\n"));
        sb.append(String.format("    Total Gross Booking Cost                                         : LKR
%,14.2f\n", totalGrossCharge));
        sb.append(String.format(" 5. Booking Advance Deducted (Paid at Reservation)                 : -LKR
%,13.2f\n", bookingAdvanceDeducted));

sb.append("=============================================================================================\n");
        sb.append(String.format(" FINAL PAYABLE AMOUNT AT CHECK-IN                                    : LKR
%,14.2f\n", finalPayableAmount));

sb.append("=============================================================================================\n");
        sb.append(" Thank you for choosing SmartStay Hotels Sri Lanka!\n");
        sb.append(" Note: Booking advance is refundable if cancelled 24+ hours prior to check-in.\n");

sb.append("=============================================================================================\n");
        return sb.toString();
    }

    /**
     * Exports the generated invoice to a text file.
     */
    public boolean exportToFile(String targetPath) {
        try {
            File file = new File(targetPath);
            File parent = file.getParentFile();
            if (parent != null && !parent.exists()) {
                parent.mkdirs();
            }
            try (BufferedWriter writer = new BufferedWriter(new FileWriter(file))) {
                writer.write(generateInvoiceText());
            }
            return true;
        } catch (IOException e) {
            System.err.println("Error saving invoice to file: " + e.getMessage());
            return false;
        }
    }

    // Getters for all billing parameters
    public String getInvoiceNumber() {
        return invoiceNumber;
    }

    public LocalDateTime getInvoiceDateTime() {
        return invoiceDateTime;
    }

    public K2703047_Reservation getReservation() {
        return reservation;
    }
```

```
    public double getDailyRate() {
        return dailyRate;
    }

    public long getNumberOfNights() {
        return numberOfNights;
    }

    public double getBaseRoomCharge() {
        return baseRoomCharge;
    }

    public int getNumberOfGuests() {
        return numberOfGuests;
    }

    public int getRoomCapacity() {
        return roomCapacity;
    }

    public int getExtraGuestCount() {
        return extraGuestCount;
    }

    public double getExtraGuestRate() {
        return extraGuestRate;
    }

    public double getExtraGuestCharges() {
        return extraGuestCharges;
    }

    public boolean isDiscountApplied() {
        return discountApplied;
    }

    public double getDiscountRate() {
        return discountRate;
    }

    public double getDiscountAmount() {
        return discountAmount;
    }

    public double getNetRoomCharge() {
        return netRoomCharge;
    }

    public double getTaxRate() {
        return taxRate;
    }

    public double getTaxAmount() {
        return taxAmount;
    }

    public double getTotalGrossCharge() {
        return totalGrossCharge;
    }

    public double getBookingAdvanceDeducted() {
        return bookingAdvanceDeducted;
```

```
    }

    public double getFinalPayableAmount() {
        return finalPayableAmount;
    }
}
```

```
package k2703047.smartstay.model;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.temporal.ChronoUnit;
import java.util.Objects;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Represents a Room Reservation in the SmartStay system.
 * Enforces business rules:
 * 1. Advance payment of LKR 10,000 recorded.
 * 2. Check-in must be at least 2 days after booking date.
 * 3. Cannot be cancelled within 24 hours of check-in date.
 * 4. Half-open date interval overlap detection [checkIn, checkOut).
 */
public class K2703047_Reservation implements Comparable<K2703047_Reservation> {
    public static final double BOOKING_ADVANCE_AMOUNT = 10000.0;
    public static final int MIN_ADVANCE_DAYS = 2;
    public static final int CANCELLATION_DEADLINE_HOURS = 24;

    private String bookingId;
    private K2703047_Guest guest;
    private K2703047_Room room;
    private LocalDate checkInDate;
    private LocalDate checkOutDate;
    private int numberOfGuests;
    private LocalDateTime bookingTimestamp;
    private double advancePaid;
    private boolean advanceRefunded;
    private K2703047_ReservationStatus status;

    /**
     * Constructs and initializes a reservation.
     */
    public K2703047_Reservation(String bookingId, K2703047_Guest guest, K2703047_Room room,
                                LocalDate checkInDate, LocalDate checkOutDate,
                                int numberOfGuests, LocalDateTime bookingTimestamp) {
        if (bookingId == null || bookingId.trim().isEmpty()) {
            throw new IllegalArgumentException("Booking ID cannot be null or empty.");
        }
        this.guest = Objects.requireNonNull(guest, "Guest information is required.");
        this.room = Objects.requireNonNull(room, "Room assignment is required.");
        this.bookingTimestamp = (bookingTimestamp != null) ? bookingTimestamp : LocalDateTime.now();

        validateDates(checkInDate, checkOutDate, this.bookingTimestamp.toLocalDate());
        if (numberOfGuests <= 0) {
            throw new IllegalArgumentException("Number of guests must be at least 1.");
        }
```

```
        this.bookingId = bookingId.trim().toUpperCase();
        this.checkInDate = checkInDate;
        this.checkOutDate = checkOutDate;
        this.numberOfGuests = numberOfGuests;
        this.advancePaid = BOOKING_ADVANCE_AMOUNT;
        this.advanceRefunded = false;
        this.status = K2703047_ReservationStatus.CONFIRMED;
    }

    /**
     * Validates business rules for booking dates:
     * - Check-out must be after check-in.
     * - Guests must book at least 2 days in advance.
     */
    public static void validateDates(LocalDate checkInDate, LocalDate checkOutDate, LocalDate bookingDate) {
        if (checkInDate == null || checkOutDate == null) {
            throw new IllegalArgumentException("Check-in and check-out dates are required.");
        }
        if (!checkOutDate.isAfter(checkInDate)) {
            throw new IllegalArgumentException("Check-out date must be strictly after check-in date.");
        }
        long daysInAdvance = ChronoUnit.DAYS.between(bookingDate, checkInDate);
        if (daysInAdvance < MIN_ADVANCE_DAYS) {
            throw new IllegalArgumentException(String.format(
                    "Advance booking rule violation: Guests must book at least %d days in advance. Selected
advance is %d day(s).",
                    MIN_ADVANCE_DAYS, daysInAdvance));
        }
    }

    /**
     * Evaluates if this reservation overlaps with another date range for the same room.
     * Standard half-open interval logic: [checkIn, checkOut)
     */
    public boolean overlapsWith(LocalDate otherCheckIn, LocalDate otherCheckOut) {
        if (this.status == K2703047_ReservationStatus.CANCELLED) {
            return false;
        }
        return this.checkInDate.isBefore(otherCheckOut) && this.checkOutDate.isAfter(otherCheckIn);
    }

    /**
     * Verifies if this reservation can be cancelled at a given timestamp.
     * Rule: Reservations cannot be modified or cancelled within 24 hours of the check-in date.
     *
     * @param currentDateTime timestamp at which cancellation is requested
     * @return true if cancellation is permitted, false if within 24 hours of check-in
     */
    public boolean isEligibleForCancellation(LocalDateTime currentDateTime) {
        if (this.status != K2703047_ReservationStatus.CONFIRMED) {
            return false;
        }
        LocalDateTime checkInStart = this.checkInDate.atStartOfDay();
        LocalDateTime cancellationDeadline = checkInStart.minusHours(CANCELLATION_DEADLINE_HOURS);
        return currentDateTime.isBefore(cancellationDeadline);
    }

    /**
     * Cancels the reservation if eligible and refunds the advance.
     */
    public boolean cancelReservation(LocalDateTime currentDateTime) {
        if (!isEligibleForCancellation(currentDateTime)) {
```

```
        return false;
    }
    this.status = K2703047_ReservationStatus.CANCELLED;
    this.advanceRefunded = true;
    return true;
}

public long getNumberOfNights() {
    return ChronoUnit.DAYS.between(checkInDate, checkOutDate);
}

// Getters and Setters
public String getBookingId() {
    return bookingId;
}

public K2703047_Guest getGuest() {
    return guest;
}

public void setGuest(K2703047_Guest guest) {
    this.guest = Objects.requireNonNull(guest);
}

public K2703047_Room getRoom() {
    return room;
}

public void setRoom(K2703047_Room room) {
    this.room = Objects.requireNonNull(room);
}

public LocalDate getCheckInDate() {
    return checkInDate;
}

public LocalDate getCheckOutDate() {
    return checkOutDate;
}

public int getNumberOfGuests() {
    return numberOfGuests;
}

public void setNumberOfGuests(int numberOfGuests) {
    if (numberOfGuests <= 0) {
        throw new IllegalArgumentException("Number of guests must be positive.");
    }
    this.numberOfGuests = numberOfGuests;
}

public LocalDateTime getBookingTimestamp() {
    return bookingTimestamp;
}

public double getAdvancePaid() {
    return advancePaid;
}

public boolean isAdvanceRefunded() {
    return advanceRefunded;
}
```

```
    public K2703047_ReservationStatus getStatus() {
        return status;
    }

    public void setStatus(K2703047_ReservationStatus status) {
        this.status = Objects.requireNonNull(status);
    }

    @Override
    public int compareTo(K2703047_Reservation other) {
        // Natural sorting by check-in date, then by booking ID
        int dateCmp = this.checkInDate.compareTo(other.checkInDate);
        if (dateCmp != 0) {
            return dateCmp;
        }
        return this.bookingId.compareTo(other.bookingId);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof K2703047_Reservation)) return false;
        K2703047_Reservation that = (K2703047_Reservation) o;
        return Objects.equals(bookingId, that.bookingId);
    }

    @Override
    public int hashCode() {
        return Objects.hash(bookingId);
    }

    @Override
    public String toString() {
        return String.format("Reservation [%s] | Guest: %s | Room: %s (%s) | %s to %s (%d nights) | Status: %s",
                bookingId, guest.getFullName(), room.getRoomId(), room.getRoomType().getDisplayName(),
                checkInDate, checkOutDate, getNumberOfNights(), status.getDisplayName());
    }
}
```

```
package k2703047.smartstay.model;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.temporal.ChronoUnit;
import java.util.Objects;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Represents a Room Reservation in the SmartStay system.
 * Enforces business rules:
 * 1. Advance payment of LKR 10,000 recorded.
 * 2. Check-in must be at least 2 days after booking date.
 * 3. Cannot be cancelled within 24 hours of check-in date.
 * 4. Half-open date interval overlap detection [checkIn, checkOut).
 */
public class K2703047_Reservation implements Comparable<K2703047_Reservation> {
    public static final double BOOKING_ADVANCE_AMOUNT = 10000.0;
```

```
    public static final int MIN_ADVANCE_DAYS = 2;
    public static final int CANCELLATION_DEADLINE_HOURS = 24;

    private String bookingId;
    private K2703047_Guest guest;
    private K2703047_Room room;
    private LocalDate checkInDate;
    private LocalDate checkOutDate;
    private int numberOfGuests;
    private LocalDateTime bookingTimestamp;
    private double advancePaid;
    private boolean advanceRefunded;
    private K2703047_ReservationStatus status;

    /**
     * Constructs and initializes a reservation.
     */
    public K2703047_Reservation(String bookingId, K2703047_Guest guest, K2703047_Room room,
                                LocalDate checkInDate, LocalDate checkOutDate,
                                int numberOfGuests, LocalDateTime bookingTimestamp) {
        if (bookingId == null || bookingId.trim().isEmpty()) {
            throw new IllegalArgumentException("Booking ID cannot be null or empty.");
        }
        this.guest = Objects.requireNonNull(guest, "Guest information is required.");
        this.room = Objects.requireNonNull(room, "Room assignment is required.");
        this.bookingTimestamp = (bookingTimestamp != null) ? bookingTimestamp : LocalDateTime.now();

        validateDates(checkInDate, checkOutDate, this.bookingTimestamp.toLocalDate());
        if (numberOfGuests <= 0) {
            throw new IllegalArgumentException("Number of guests must be at least 1.");
        }

        this.bookingId = bookingId.trim().toUpperCase();
        this.checkInDate = checkInDate;
        this.checkOutDate = checkOutDate;
        this.numberOfGuests = numberOfGuests;
        this.advancePaid = BOOKING_ADVANCE_AMOUNT;
        this.advanceRefunded = false;
        this.status = K2703047_ReservationStatus.CONFIRMED;
    }

    /**
     * Validates business rules for booking dates:
     * - Check-out must be after check-in.
     * - Guests must book at least 2 days in advance.
     */
    public static void validateDates(LocalDate checkInDate, LocalDate checkOutDate, LocalDate bookingDate) {
        if (checkInDate == null || checkOutDate == null) {
            throw new IllegalArgumentException("Check-in and check-out dates are required.");
        }
        if (!checkOutDate.isAfter(checkInDate)) {
            throw new IllegalArgumentException("Check-out date must be strictly after check-in date.");
        }
        long daysInAdvance = ChronoUnit.DAYS.between(bookingDate, checkInDate);
        if (daysInAdvance < MIN_ADVANCE_DAYS) {
            throw new IllegalArgumentException(String.format(
                    "Advance booking rule violation: Guests must book at least %d days in advance. Selected
advance is %d day(s).",
                    MIN_ADVANCE_DAYS, daysInAdvance));
        }
    }
```

```
    /**
     * Evaluates if this reservation overlaps with another date range for the same room.
     * Standard half-open interval logic: [checkIn, checkOut)
     */
    public boolean overlapsWith(LocalDate otherCheckIn, LocalDate otherCheckOut) {
        if (this.status == K2703047_ReservationStatus.CANCELLED) {
            return false;
        }
        return this.checkInDate.isBefore(otherCheckOut) && this.checkOutDate.isAfter(otherCheckIn);
    }

    /**
     * Verifies if this reservation can be cancelled at a given timestamp.
     * Rule: Reservations cannot be modified or cancelled within 24 hours of the check-in date.
     *
     * @param currentDateTime timestamp at which cancellation is requested
     * @return true if cancellation is permitted, false if within 24 hours of check-in
     */
    public boolean isEligibleForCancellation(LocalDateTime currentDateTime) {
        if (this.status != K2703047_ReservationStatus.CONFIRMED) {
            return false;
        }
        LocalDateTime checkInStart = this.checkInDate.atStartOfDay();
        LocalDateTime cancellationDeadline = checkInStart.minusHours(CANCELLATION_DEADLINE_HOURS);
        return currentDateTime.isBefore(cancellationDeadline);
    }

    /**
     * Cancels the reservation if eligible and refunds the advance.
     */
    public boolean cancelReservation(LocalDateTime currentDateTime) {
        if (!isEligibleForCancellation(currentDateTime)) {
            return false;
        }
        this.status = K2703047_ReservationStatus.CANCELLED;
        this.advanceRefunded = true;
        return true;
    }

    public long getNumberOfNights() {
        return ChronoUnit.DAYS.between(checkInDate, checkOutDate);
    }

    // Getters and Setters
    public String getBookingId() {
        return bookingId;
    }

    public K2703047_Guest getGuest() {
        return guest;
    }

    public void setGuest(K2703047_Guest guest) {
        this.guest = Objects.requireNonNull(guest);
    }

    public K2703047_Room getRoom() {
        return room;
    }

    public void setRoom(K2703047_Room room) {
        this.room = Objects.requireNonNull(room);
```

```
    }

    public LocalDate getCheckInDate() {
        return checkInDate;
    }

    public LocalDate getCheckOutDate() {
        return checkOutDate;
    }

    public int getNumberOfGuests() {
        return numberOfGuests;
    }

    public void setNumberOfGuests(int numberOfGuests) {
        if (numberOfGuests <= 0) {
            throw new IllegalArgumentException("Number of guests must be positive.");
        }
        this.numberOfGuests = numberOfGuests;
    }

    public LocalDateTime getBookingTimestamp() {
        return bookingTimestamp;
    }

    public double getAdvancePaid() {
        return advancePaid;
    }

    public boolean isAdvanceRefunded() {
        return advanceRefunded;
    }

    public K2703047_ReservationStatus getStatus() {
        return status;
    }

    public void setStatus(K2703047_ReservationStatus status) {
        this.status = Objects.requireNonNull(status);
    }

    @Override
    public int compareTo(K2703047_Reservation other) {
        // Natural sorting by check-in date, then by booking ID
        int dateCmp = this.checkInDate.compareTo(other.checkInDate);
        if (dateCmp != 0) {
            return dateCmp;
        }
        return this.bookingId.compareTo(other.bookingId);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof K2703047_Reservation)) return false;
        K2703047_Reservation that = (K2703047_Reservation) o;
        return Objects.equals(bookingId, that.bookingId);
    }

    @Override
    public int hashCode() {
        return Objects.hash(bookingId);
```

```
    }

    @Override
    public String toString() {
        return String.format("Reservation [%s] | Guest: %s | Room: %s (%s) | %s to %s (%d nights) | Status:
 %s",
                bookingId, guest.getFullName(), room.getRoomId(), room.getRoomType().getDisplayName(),
                checkInDate, checkOutDate, getNumberOfNights(), status.getDisplayName());
    }
 }
```

```
package k2703047.smartstay.model;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Enum representing reservation statuses:
 * CONFIRMED, CANCELLED, COMPLETED
 */
public enum K2703047_ReservationStatus {
    CONFIRMED("Confirmed"),
    CANCELLED("Cancelled"),
    COMPLETED("Completed");

    private final String displayName;

    K2703047_ReservationStatus(String displayName) {
        this.displayName = displayName;
    }

    public String getDisplayName() {
        return displayName;
    }

    @Override
    public String toString() {
        return displayName;
    }
}
```

```
 package k2703047.smartstay.model;

 /**
  * Kingston University Coursework - CI6115 Programming III
  * Student ID: 2703047
  * Module: Patterns and Algorithms
  *
  * Enum representing reservation statuses:
  * CONFIRMED, CANCELLED, COMPLETED
  */
 public enum K2703047_ReservationStatus {
     CONFIRMED("Confirmed"),
     CANCELLED("Cancelled"),
     COMPLETED("Completed");

     private final String displayName;
```

```
    K2703047_ReservationStatus(String displayName) {
        this.displayName = displayName;
    }

    public String getDisplayName() {
        return displayName;
    }

    @Override
    public String toString() {
        return displayName;
    }
}
```

```
package k2703047.smartstay.model;

import java.util.Objects;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Abstract base class representing a hotel room at SmartStay.
 * Demonstrates Abstraction and Encapsulation. Subclasses implement
 * specific room package business policies and calculations (Polymorphism).
 */
public abstract class K2703047_Room {
    private String roomId;
    private K2703047_RoomType roomType;
    private K2703047_BedType bedType;
    private double dailyRate;
    private K2703047_RoomStatus status;
    private int baseCapacity;

    /**
     * Parameterized constructor.
     *
     * @param roomId       Unique room identifier
     * @param roomType     Category of room
     * @param bedType      Type of bed
     * @param dailyRate    Room-type specific daily rate (LKR)
     * @param status       Availability status
     * @param baseCapacity Base guest capacity before extra guest charges apply
     */
    public K2703047_Room(String roomId, K2703047_RoomType roomType, K2703047_BedType bedType,
                         double dailyRate, K2703047_RoomStatus status, int baseCapacity) {
        if (roomId == null || roomId.trim().isEmpty()) {
            throw new IllegalArgumentException("Room ID cannot be null or empty.");
        }
        if (dailyRate <= 0) {
            throw new IllegalArgumentException("Daily room rate must be positive.");
        }
        if (baseCapacity <= 0) {
            throw new IllegalArgumentException("Base capacity must be at least 1.");
        }
        this.roomId = roomId.trim().toUpperCase();
        this.roomType = Objects.requireNonNull(roomType, "Room type is required.");
        this.bedType = Objects.requireNonNull(bedType, "Bed type is required.");
```

```
    this.dailyRate = dailyRate;
    this.status = (status != null) ? status : K2703047_RoomStatus.AVAILABLE;
    this.baseCapacity = baseCapacity;
}

// Abstract methods implementing Polymorphism across Room packages
public abstract double calculateTax(double netRoomCharge);
public abstract double getExtraGuestRate();
public abstract boolean hasComplimentaryBreakfast();
public abstract double getTaxRate();
public abstract String getRoomPackageDescription();

/**
 * Calculates the extra guest charge based on room-specific policies.
 * Formula: (Number of Guests - Base Capacity) * Extra Guest Rate * Number of Nights
 *
 * @param guestCount     Total number of staying guests
 * @param numberOfNights Total duration in nights
 * @return Extra guest charge in LKR
 */
public double calculateExtraGuestCharge(int guestCount, long numberOfNights) {
    if (guestCount <= baseCapacity || numberOfNights <= 0) {
        return 0.0;
    }
    int extraGuests = guestCount - baseCapacity;
    return extraGuests * getExtraGuestRate() * numberOfNights;
}

public boolean isAvailable() {
    return this.status == K2703047_RoomStatus.AVAILABLE;
}

// Getters and Setters with Encapsulation
public String getRoomId() {
    return roomId;
}

public void setRoomId(String roomId) {
    if (roomId == null || roomId.trim().isEmpty()) {
        throw new IllegalArgumentException("Room ID cannot be null or empty.");
    }
    this.roomId = roomId.trim().toUpperCase();
}

public K2703047_RoomType getRoomType() {
    return roomType;
}

public void setRoomType(K2703047_RoomType roomType) {
    this.roomType = Objects.requireNonNull(roomType, "Room type cannot be null.");
}

public K2703047_BedType getBedType() {
    return bedType;
}

public void setBedType(K2703047_BedType bedType) {
    this.bedType = Objects.requireNonNull(bedType, "Bed type cannot be null.");
}

public double getDailyRate() {
    return dailyRate;
```

```
    }

    public void setDailyRate(double dailyRate) {
        if (dailyRate <= 0) {
            throw new IllegalArgumentException("Daily rate must be positive.");
        }
        this.dailyRate = dailyRate;
    }

    public K2703047_RoomStatus getStatus() {
        return status;
    }

    public void setStatus(K2703047_RoomStatus status) {
        this.status = Objects.requireNonNull(status, "Status cannot be null.");
    }

    public int getBaseCapacity() {
        return baseCapacity;
    }

    public void setBaseCapacity(int baseCapacity) {
        if (baseCapacity <= 0) {
            throw new IllegalArgumentException("Base capacity must be positive.");
        }
        this.baseCapacity = baseCapacity;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof K2703047_Room)) return false;
        K2703047_Room that = (K2703047_Room) o;
        return Objects.equals(roomId, that.roomId);
    }

    @Override
    public int hashCode() {
        return Objects.hash(roomId);
    }

    @Override
    public String toString() {
        return String.format("Room [%s] - %s | Bed: %s | Rate: LKR %,.2f/night | Status: %s | Capacity: %d",
                roomId, roomType.getDisplayName(), bedType.getDisplayName(), dailyRate, status.getDisplayName(),
baseCapacity);
    }
}
```

```
package k2703047.smartstay.model;

import java.util.Objects;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Abstract base class representing a hotel room at SmartStay.
 * Demonstrates Abstraction and Encapsulation. Subclasses implement
 * specific room package business policies and calculations (Polymorphism).
```

```
 */
public abstract class K2703047_Room {
    private String roomId;
    private K2703047_RoomType roomType;
    private K2703047_BedType bedType;
    private double dailyRate;
    private K2703047_RoomStatus status;
    private int baseCapacity;

    /**
     * Parameterized constructor.
     *
     * @param roomId       Unique room identifier
     * @param roomType     Category of room
     * @param bedType      Type of bed
     * @param dailyRate    Room-type specific daily rate (LKR)
     * @param status       Availability status
     * @param baseCapacity Base guest capacity before extra guest charges apply
     */
    public K2703047_Room(String roomId, K2703047_RoomType roomType, K2703047_BedType bedType,
                         double dailyRate, K2703047_RoomStatus status, int baseCapacity) {
        if (roomId == null || roomId.trim().isEmpty()) {
            throw new IllegalArgumentException("Room ID cannot be null or empty.");
        }
        if (dailyRate <= 0) {
            throw new IllegalArgumentException("Daily room rate must be positive.");
        }
        if (baseCapacity <= 0) {
            throw new IllegalArgumentException("Base capacity must be at least 1.");
        }
        this.roomId = roomId.trim().toUpperCase();
        this.roomType = Objects.requireNonNull(roomType, "Room type is required.");
        this.bedType = Objects.requireNonNull(bedType, "Bed type is required.");
        this.dailyRate = dailyRate;
        this.status = (status != null) ? status : K2703047_RoomStatus.AVAILABLE;
        this.baseCapacity = baseCapacity;
    }

    // Abstract methods implementing Polymorphism across Room packages
    public abstract double calculateTax(double netRoomCharge);
    public abstract double getExtraGuestRate();
    public abstract boolean hasComplimentaryBreakfast();
    public abstract double getTaxRate();
    public abstract String getRoomPackageDescription();

    /**
     * Calculates the extra guest charge based on room-specific policies.
     * Formula: (Number of Guests - Base Capacity) * Extra Guest Rate * Number of Nights
     *
     * @param guestCount     Total number of staying guests
     * @param numberOfNights Total duration in nights
     * @return Extra guest charge in LKR
     */
    public double calculateExtraGuestCharge(int guestCount, long numberOfNights) {
        if (guestCount <= baseCapacity || numberOfNights <= 0) {
            return 0.0;
        }
        int extraGuests = guestCount - baseCapacity;
        return extraGuests * getExtraGuestRate() * numberOfNights;
    }

    public boolean isAvailable() {
```

```
        return this.status == K2703047_RoomStatus.AVAILABLE;
    }

    // Getters and Setters with Encapsulation
    public String getRoomId() {
        return roomId;
    }

    public void setRoomId(String roomId) {
        if (roomId == null || roomId.trim().isEmpty()) {
            throw new IllegalArgumentException("Room ID cannot be null or empty.");
        }
        this.roomId = roomId.trim().toUpperCase();
    }

    public K2703047_RoomType getRoomType() {
        return roomType;
    }

    public void setRoomType(K2703047_RoomType roomType) {
        this.roomType = Objects.requireNonNull(roomType, "Room type cannot be null.");
    }

    public K2703047_BedType getBedType() {
        return bedType;
    }

    public void setBedType(K2703047_BedType bedType) {
        this.bedType = Objects.requireNonNull(bedType, "Bed type cannot be null.");
    }

    public double getDailyRate() {
        return dailyRate;
    }

    public void setDailyRate(double dailyRate) {
        if (dailyRate <= 0) {
            throw new IllegalArgumentException("Daily rate must be positive.");
        }
        this.dailyRate = dailyRate;
    }

    public K2703047_RoomStatus getStatus() {
        return status;
    }

    public void setStatus(K2703047_RoomStatus status) {
        this.status = Objects.requireNonNull(status, "Status cannot be null.");
    }

    public int getBaseCapacity() {
        return baseCapacity;
    }

    public void setBaseCapacity(int baseCapacity) {
        if (baseCapacity <= 0) {
            throw new IllegalArgumentException("Base capacity must be positive.");
        }
        this.baseCapacity = baseCapacity;
    }

    @Override
```

```
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof K2703047_Room)) return false;
        K2703047_Room that = (K2703047_Room) o;
        return Objects.equals(roomId, that.roomId);
    }

    @Override
    public int hashCode() {
        return Objects.hash(roomId);
    }

    @Override
    public String toString() {
        return String.format("Room [%s] - %s | Bed: %s | Rate: LKR %,.2f/night | Status: %s | Capacity: %d",
                roomId, roomType.getDisplayName(), bedType.getDisplayName(), dailyRate,
 status.getDisplayName(), baseCapacity);
    }
 }
```

```
package k2703047.smartstay.model;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Factory Method Pattern implementation for creating Room instances.
 * Promotes loose coupling, single responsibility, and extensible room creation.
 */
public class K2703047_RoomFactory {

    /**
     * Creates a room instance based on the specified room type using default rates.
     */
    public static K2703047_Room createRoom(String roomId, K2703047_RoomType roomType, K2703047_BedType bedType)
{
        return createRoom(roomId, roomType, bedType, roomType.getDailyRate(), K2703047_RoomStatus.AVAILABLE,
roomType.getStandardCapacity());
    }

    /**
     * Creates a room instance with full custom attributes.
     */
    public static K2703047_Room createRoom(String roomId, K2703047_RoomType roomType, K2703047_BedType bedType,
                                           double dailyRate, K2703047_RoomStatus status, int baseCapacity) {
        switch (roomType) {
            case STANDARD:
                return new K2703047_StandardRoom(roomId, bedType, dailyRate, status, baseCapacity);
            case DELUXE:
                return new K2703047_DeluxeRoom(roomId, bedType, dailyRate, status, baseCapacity);
            case SUITE:
                return new K2703047_SuiteRoom(roomId, bedType, dailyRate, status, baseCapacity);
            case FAMILY_SUITE:
                return new K2703047_FamilySuiteRoom(roomId, bedType, dailyRate, status, baseCapacity);
            default:
                throw new IllegalArgumentException("Unknown room type: " + roomType);
        }
    }
```

}

```
package k2703047.smartstay.model;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Factory Method Pattern implementation for creating Room instances.
 * Promotes loose coupling, single responsibility, and extensible room creation.
 */
public class K2703047_RoomFactory {

    /**
     * Creates a room instance based on the specified room type using default rates.
     */
    public static K2703047_Room createRoom(String roomId, K2703047_RoomType roomType, K2703047_BedType
bedType) {
        return createRoom(roomId, roomType, bedType, roomType.getDailyRate(), K2703047_RoomStatus.AVAILABLE,
roomType.getStandardCapacity());
    }

    /**
     * Creates a room instance with full custom attributes.
     */
    public static K2703047_Room createRoom(String roomId, K2703047_RoomType roomType, K2703047_BedType
bedType,
                                           double dailyRate, K2703047_RoomStatus status, int baseCapacity) {
        switch (roomType) {
            case STANDARD:
                return new K2703047_StandardRoom(roomId, bedType, dailyRate, status, baseCapacity);
            case DELUXE:
                return new K2703047_DeluxeRoom(roomId, bedType, dailyRate, status, baseCapacity);
            case SUITE:
                return new K2703047_SuiteRoom(roomId, bedType, dailyRate, status, baseCapacity);
            case FAMILY_SUITE:
                return new K2703047_FamilySuiteRoom(roomId, bedType, dailyRate, status, baseCapacity);
            default:
                throw new IllegalArgumentException("Unknown room type: " + roomType);
        }
    }
}
```

```
package k2703047.smartstay.model;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Enum representing room availability statuses:
 * Available, Reserved, Under Maintenance
 */
public enum K2703047_RoomStatus {
    AVAILABLE("Available"),
    RESERVED("Reserved"),
    UNDER_MAINTENANCE("Under Maintenance");
```

```
    private final String displayName;

    K2703047_RoomStatus(String displayName) {
        this.displayName = displayName;
    }

    public String getDisplayName() {
        return displayName;
    }

    @Override
    public String toString() {
        return displayName;
    }
}
```

```
package k2703047.smartstay.model;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Enum representing room availability statuses:
 * Available, Reserved, Under Maintenance
 */
public enum K2703047_RoomStatus {
    AVAILABLE("Available"),
    RESERVED("Reserved"),
    UNDER_MAINTENANCE("Under Maintenance");

    private final String displayName;

    K2703047_RoomStatus(String displayName) {
        this.displayName = displayName;
    }

    public String getDisplayName() {
        return displayName;
    }

    @Override
    public String toString() {
        return displayName;
    }
}
```

```
package k2703047.smartstay.model;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Enum representing available room types and their package specifications.
 * Based on Table 1 of the coursework brief:
 * - Standard: LKR 6,000 | Breakfast: No  | Extra Guest: LKR 1,000 | Tax: 10% | Base Capacity: 2
 * - Deluxe:   LKR 9,000 | Breakfast: Yes | Extra Guest: LKR 1,500 | Tax: 12% | Base Capacity: 2
 * - Suite:    LKR 14,000| Breakfast: Yes | Extra Guest: LKR 2,000 | Tax: 15% | Base Capacity: 3
```

```
 * - Family:   LKR 18,000| Breakfast: Yes | Extra Guest: LKR 2,500 | Tax: 15% | Base Capacity: 4
 */
public enum K2703047_RoomType {
    STANDARD("Standard", 6000.0, false, 1000.0, 0.10, 2),
    DELUXE("Deluxe", 9000.0, true, 1500.0, 0.12, 2),
    SUITE("Suite", 14000.0, true, 2000.0, 0.15, 3),
    FAMILY_SUITE("Family Suite", 18000.0, true, 2500.0, 0.15, 4);

    private final String displayName;
    private final double dailyRate;
    private final boolean complimentaryBreakfast;
    private final double extraGuestCharge;
    private final double taxRate;
    private final int standardCapacity;

    K2703047_RoomType(String displayName, double dailyRate, boolean complimentaryBreakfast,
                      double extraGuestCharge, double taxRate, int standardCapacity) {
        this.displayName = displayName;
        this.dailyRate = dailyRate;
        this.complimentaryBreakfast = complimentaryBreakfast;
        this.extraGuestCharge = extraGuestCharge;
        this.taxRate = taxRate;
        this.standardCapacity = standardCapacity;
    }

    public String getDisplayName() {
        return displayName;
    }

    public double getDailyRate() {
        return dailyRate;
    }

    public boolean hasComplimentaryBreakfast() {
        return complimentaryBreakfast;
    }

    public double getExtraGuestCharge() {
        return extraGuestCharge;
    }

    public double getTaxRate() {
        return taxRate;
    }

    public int getStandardCapacity() {
        return standardCapacity;
    }

    @Override
    public String toString() {
        return displayName + " (LKR " + String.format("%,.0f", dailyRate) + "/night)";
    }
}
```

```
package k2703047.smartstay.model;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
```

```
 *
 * Enum representing available room types and their package specifications.
 * Based on Table 1 of the coursework brief:
 * - Standard: LKR 6,000 | Breakfast: No  | Extra Guest: LKR 1,000 | Tax: 10% | Base Capacity: 2
 * - Deluxe:   LKR 9,000 | Breakfast: Yes | Extra Guest: LKR 1,500 | Tax: 12% | Base Capacity: 2
 * - Suite:    LKR 14,000| Breakfast: Yes | Extra Guest: LKR 2,000 | Tax: 15% | Base Capacity: 3
 * - Family:   LKR 18,000| Breakfast: Yes | Extra Guest: LKR 2,500 | Tax: 15% | Base Capacity: 4
 */
public enum K2703047_RoomType {
    STANDARD("Standard", 6000.0, false, 1000.0, 0.10, 2),
    DELUXE("Deluxe", 9000.0, true, 1500.0, 0.12, 2),
    SUITE("Suite", 14000.0, true, 2000.0, 0.15, 3),
    FAMILY_SUITE("Family Suite", 18000.0, true, 2500.0, 0.15, 4);

    private final String displayName;
    private final double dailyRate;
    private final boolean complimentaryBreakfast;
    private final double extraGuestCharge;
    private final double taxRate;
    private final int standardCapacity;

    K2703047_RoomType(String displayName, double dailyRate, boolean complimentaryBreakfast,
                      double extraGuestCharge, double taxRate, int standardCapacity) {
        this.displayName = displayName;
        this.dailyRate = dailyRate;
        this.complimentaryBreakfast = complimentaryBreakfast;
        this.extraGuestCharge = extraGuestCharge;
        this.taxRate = taxRate;
        this.standardCapacity = standardCapacity;
    }

    public String getDisplayName() {
        return displayName;
    }

    public double getDailyRate() {
        return dailyRate;
    }

    public boolean hasComplimentaryBreakfast() {
        return complimentaryBreakfast;
    }

    public double getExtraGuestCharge() {
        return extraGuestCharge;
    }

    public double getTaxRate() {
        return taxRate;
    }

    public int getStandardCapacity() {
        return standardCapacity;
    }

    @Override
    public String toString() {
        return displayName + " (LKR " + String.format("%,.0f", dailyRate) + "/night)";
    }
}
```

```
package k2703047.smartstay.model;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Standard Room Package implementation.
 * Specification:
 * - Daily Rate: LKR 6,000
 * - Complimentary Breakfast: No
 * - Extra Guest Charge: LKR 1,000 per guest per night
 * - Tax Rate: 10%
 * - Base Capacity: 2
 */
public class K2703047_StandardRoom extends K2703047_Room {

    public static final double DEFAULT_DAILY_RATE = 6000.0;
    public static final boolean COMPLIMENTARY_BREAKFAST = false;
    public static final double EXTRA_GUEST_CHARGE = 1000.0;
    public static final double TAX_RATE = 0.10;
    public static final int BASE_CAPACITY = 2;

    public K2703047_StandardRoom(String roomId, K2703047_BedType bedType) {
        super(roomId, K2703047_RoomType.STANDARD, bedType, DEFAULT_DAILY_RATE, K2703047_RoomStatus.AVAILABLE,
BASE_CAPACITY);
    }

    public K2703047_StandardRoom(String roomId, K2703047_BedType bedType, double dailyRate,
                                 K2703047_RoomStatus status, int baseCapacity) {
        super(roomId, K2703047_RoomType.STANDARD, bedType, dailyRate, status, baseCapacity);
    }

    @Override
    public double calculateTax(double netRoomCharge) {
        return netRoomCharge * TAX_RATE;
    }

    @Override
    public double getExtraGuestRate() {
        return EXTRA_GUEST_CHARGE;
    }

    @Override
    public boolean hasComplimentaryBreakfast() {
        return COMPLIMENTARY_BREAKFAST;
    }

    @Override
    public double getTaxRate() {
        return TAX_RATE;
    }

    @Override
    public String getRoomPackageDescription() {
        return "Standard Package: LKR 6,000/night | No Breakfast | Extra: LKR 1,000/guest | Tax: 10%";
    }
}
```

```
package k2703047.smartstay.model;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Standard Room Package implementation.
 * Specification:
 * - Daily Rate: LKR 6,000
 * - Complimentary Breakfast: No
 * - Extra Guest Charge: LKR 1,000 per guest per night
 * - Tax Rate: 10%
 * - Base Capacity: 2
 */
public class K2703047_StandardRoom extends K2703047_Room {

    public static final double DEFAULT_DAILY_RATE = 6000.0;
    public static final boolean COMPLIMENTARY_BREAKFAST = false;
    public static final double EXTRA_GUEST_CHARGE = 1000.0;
    public static final double TAX_RATE = 0.10;
    public static final int BASE_CAPACITY = 2;

    public K2703047_StandardRoom(String roomId, K2703047_BedType bedType) {
        super(roomId, K2703047_RoomType.STANDARD, bedType, DEFAULT_DAILY_RATE, K2703047_RoomStatus.AVAILABLE,
BASE_CAPACITY);
    }

    public K2703047_StandardRoom(String roomId, K2703047_BedType bedType, double dailyRate,
                                 K2703047_RoomStatus status, int baseCapacity) {
        super(roomId, K2703047_RoomType.STANDARD, bedType, dailyRate, status, baseCapacity);
    }

    @Override
    public double calculateTax(double netRoomCharge) {
        return netRoomCharge * TAX_RATE;
    }

    @Override
    public double getExtraGuestRate() {
        return EXTRA_GUEST_CHARGE;
    }

    @Override
    public boolean hasComplimentaryBreakfast() {
        return COMPLIMENTARY_BREAKFAST;
    }

    @Override
    public double getTaxRate() {
        return TAX_RATE;
    }

    @Override
    public String getRoomPackageDescription() {
        return "Standard Package: LKR 6,000/night | No Breakfast | Extra: LKR 1,000/guest | Tax: 10%";
    }
}
```

```
package k2703047.smartstay.model;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Suite Room Package implementation.
 * Specification:
 * - Daily Rate: LKR 14,000
 * - Complimentary Breakfast: Yes
 * - Extra Guest Charge: LKR 2,000 per guest per night
 * - Tax Rate: 15%
 * - Base Capacity: 3
 */
public class K2703047_SuiteRoom extends K2703047_Room {

    public static final double DEFAULT_DAILY_RATE = 14000.0;
    public static final boolean COMPLIMENTARY_BREAKFAST = true;
    public static final double EXTRA_GUEST_CHARGE = 2000.0;
    public static final double TAX_RATE = 0.15;
    public static final int BASE_CAPACITY = 3;

    public K2703047_SuiteRoom(String roomId, K2703047_BedType bedType) {
        super(roomId, K2703047_RoomType.SUITE, bedType, DEFAULT_DAILY_RATE, K2703047_RoomStatus.AVAILABLE,
BASE_CAPACITY);
    }

    public K2703047_SuiteRoom(String roomId, K2703047_BedType bedType, double dailyRate,
                              K2703047_RoomStatus status, int baseCapacity) {
        super(roomId, K2703047_RoomType.SUITE, bedType, dailyRate, status, baseCapacity);
    }

    @Override
    public double calculateTax(double netRoomCharge) {
        return netRoomCharge * TAX_RATE;
    }

    @Override
    public double getExtraGuestRate() {
        return EXTRA_GUEST_CHARGE;
    }

    @Override
    public boolean hasComplimentaryBreakfast() {
        return COMPLIMENTARY_BREAKFAST;
    }

    @Override
    public double getTaxRate() {
        return TAX_RATE;
    }

    @Override
    public String getRoomPackageDescription() {
        return "Suite Package: LKR 14,000/night | Complimentary Breakfast Included | Extra: LKR 2,000/guest |
Tax: 15%";
    }
}
```


```
package k2703047.smartstay.model;
```

```java
/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Suite Room Package implementation.
 * Specification:
 * - Daily Rate: LKR 14,000
 * - Complimentary Breakfast: Yes
 * - Extra Guest Charge: LKR 2,000 per guest per night
 * - Tax Rate: 15%
 * - Base Capacity: 3
 */
public class K2703047_SuiteRoom extends K2703047_Room {

    public static final double DEFAULT_DAILY_RATE = 14000.0;
    public static final boolean COMPLIMENTARY_BREAKFAST = true;
    public static final double EXTRA_GUEST_CHARGE = 2000.0;
    public static final double TAX_RATE = 0.15;
    public static final int BASE_CAPACITY = 3;

    public K2703047_SuiteRoom(String roomId, K2703047_BedType bedType) {
        super(roomId, K2703047_RoomType.SUITE, bedType, DEFAULT_DAILY_RATE, K2703047_RoomStatus.AVAILABLE,
BASE_CAPACITY);
    }

    public K2703047_SuiteRoom(String roomId, K2703047_BedType bedType, double dailyRate,
                              K2703047_RoomStatus status, int baseCapacity) {
        super(roomId, K2703047_RoomType.SUITE, bedType, dailyRate, status, baseCapacity);
    }

    @Override
    public double calculateTax(double netRoomCharge) {
        return netRoomCharge * TAX_RATE;
    }

    @Override
    public double getExtraGuestRate() {
        return EXTRA_GUEST_CHARGE;
    }

    @Override
    public boolean hasComplimentaryBreakfast() {
        return COMPLIMENTARY_BREAKFAST;
    }

    @Override
    public double getTaxRate() {
        return TAX_RATE;
    }

    @Override
    public String getRoomPackageDescription() {
        return "Suite Package: LKR 14,000/night | Complimentary Breakfast Included | Extra: LKR 2,000/guest |
Tax: 15%";
    }
}
```

```java
package k2703047.smartstay.service;

import k2703047.smartstay.model.K2703047_Guest;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Implementation of Guest Registration and Directory management.
 * Data Structure: LinkedHashMap indexed by NIC/Passport for O(1) key lookups.
 */
public class K2703047_GuestService implements K2703047_IGuestService {

    private final Map<String, K2703047_Guest> guestDirectory;

    public K2703047_GuestService() {
        this.guestDirectory = new LinkedHashMap<>();
    }

    @Override
    public synchronized boolean registerGuest(K2703047_Guest guest) {
        if (guest == null || guestDirectory.containsKey(guest.getNicOrPassport())) {
            return false;
        }
        guestDirectory.put(guest.getNicOrPassport(), guest);
        return true;
    }

    @Override
    public K2703047_Guest getGuestByNicOrPassport(String nicOrPassport) {
        if (nicOrPassport == null) return null;
        return guestDirectory.get(nicOrPassport.trim().toUpperCase());
    }

    @Override
    public List<K2703047_Guest> getAllGuests() {
        return new ArrayList<>(guestDirectory.values());
    }

    /**
     * Search Algorithm: Linear search with case-insensitive substring matching.
     */
    @Override
    public List<K2703047_Guest> searchGuestsByName(String nameKeyword) {
        List<K2703047_Guest> results = new ArrayList<>();
        if (nameKeyword == null || nameKeyword.trim().isEmpty()) {
            return getAllGuests();
        }
        String lowerKeyword = nameKeyword.trim().toLowerCase();
        for (K2703047_Guest guest : guestDirectory.values()) {
            if (guest.getFullName().toLowerCase().contains(lowerKeyword)) {
                results.add(guest);
            }
        }
        return results;
    }
```

```
    @Override
    public boolean guestExists(String nicOrPassport) {
        if (nicOrPassport == null) return false;
        return guestDirectory.containsKey(nicOrPassport.trim().toUpperCase());
    }
}
```

```
package k2703047.smartstay.service;

import k2703047.smartstay.model.K2703047_Guest;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Implementation of Guest Registration and Directory management.
 * Data Structure: LinkedHashMap indexed by NIC/Passport for O(1) key lookups.
 */
public class K2703047_GuestService implements K2703047_IGuestService {

    private final Map<String, K2703047_Guest> guestDirectory;

    public K2703047_GuestService() {
        this.guestDirectory = new LinkedHashMap<>();
    }

    @Override
    public synchronized boolean registerGuest(K2703047_Guest guest) {
        if (guest == null || guestDirectory.containsKey(guest.getNicOrPassport())) {
            return false;
        }
        guestDirectory.put(guest.getNicOrPassport(), guest);
        return true;
    }

    @Override
    public K2703047_Guest getGuestByNicOrPassport(String nicOrPassport) {
        if (nicOrPassport == null) return null;
        return guestDirectory.get(nicOrPassport.trim().toUpperCase());
    }

    @Override
    public List<K2703047_Guest> getAllGuests() {
        return new ArrayList<>(guestDirectory.values());
    }

    /**
     * Search Algorithm: Linear search with case-insensitive substring matching.
     */
    @Override
    public List<K2703047_Guest> searchGuestsByName(String nameKeyword) {
        List<K2703047_Guest> results = new ArrayList<>();
        if (nameKeyword == null || nameKeyword.trim().isEmpty()) {
            return getAllGuests();
```

```
        }
        String lowerKeyword = nameKeyword.trim().toLowerCase();
        for (K2703047_Guest guest : guestDirectory.values()) {
            if (guest.getFullName().toLowerCase().contains(lowerKeyword)) {
                results.add(guest);
            }
        }
        return results;
    }

    @Override
    public boolean guestExists(String nicOrPassport) {
        if (nicOrPassport == null) return false;
        return guestDirectory.containsKey(nicOrPassport.trim().toUpperCase());
    }
}
```

```
package k2703047.smartstay.service;

import k2703047.smartstay.model.K2703047_BedType;
import k2703047.smartstay.model.K2703047_FamilySuiteRoom;
import k2703047.smartstay.model.K2703047_Guest;
import k2703047.smartstay.model.K2703047_Invoice;
import k2703047.smartstay.model.K2703047_Reservation;
import k2703047.smartstay.model.K2703047_Room;
import k2703047.smartstay.model.K2703047_RoomFactory;
import k2703047.smartstay.model.K2703047_RoomStatus;
import k2703047.smartstay.model.K2703047_RoomType;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Facade Pattern implementation coordinating Room, Guest, and Reservation subsystems.
 * Acts as the centralized API for UI controllers (Console CLI & Desktop GUI).
 */
public class K2703047_HotelSystem {

    private final K2703047_IRoomService roomService;
    private final K2703047_IGuestService guestService;
    private final K2703047_IReservationService reservationService;

    public K2703047_HotelSystem() {
        this.roomService = new K2703047_RoomService();
        this.guestService = new K2703047_GuestService();
        this.reservationService = new K2703047_ReservationService();
        seedInitialData();
    }

    /**
     * Seeds initial realistic sample rooms, registered guests, and bookings.
     */
    private void seedInitialData() {
        // Seed Rooms across all types and bed options
        roomService.addRoom(K2703047_RoomFactory.createRoom("R-101", K2703047_RoomType.STANDARD,
```

```
K2703047_BedType.SINGLE));
        roomService.addRoom(K2703047_RoomFactory.createRoom("R-102", K2703047_RoomType.STANDARD,
K2703047_BedType.DOUBLE));
        roomService.addRoom(K2703047_RoomFactory.createRoom("D-201", K2703047_RoomType.DELUXE,
K2703047_BedType.QUEEN));
        roomService.addRoom(K2703047_RoomFactory.createRoom("D-202", K2703047_RoomType.DELUXE,
K2703047_BedType.KING));
        roomService.addRoom(K2703047_RoomFactory.createRoom("S-301", K2703047_RoomType.SUITE,
K2703047_BedType.KING));
        roomService.addRoom(K2703047_RoomFactory.createRoom("FS-401", K2703047_RoomType.FAMILY_SUITE,
K2703047_BedType.KING));

        // One room under maintenance for rule validation
        K2703047_Room maintRoom = K2703047_RoomFactory.createRoom("R-103", K2703047_RoomType.STANDARD,
K2703047_BedType.DOUBLE);
        maintRoom.setStatus(K2703047_RoomStatus.UNDER_MAINTENANCE);
        roomService.addRoom(maintRoom);

        // Seed Sample Guests
        K2703047_Guest g1 = new K2703047_Guest("199512345678", "Kasun Perera", "+94771234567",
"kasun.p@gmail.com");
        K2703047_Guest g2 = new K2703047_Guest("N87654321", "Sarah Jenkins", "+447911123456",
"sarah.j@outlook.com");
        guestService.registerGuest(g1);
        guestService.registerGuest(g2);

        // Seed an active reservation (e.g., checking in 5 days from today for 6 nights -> qualifies for 12%
discount)
        LocalDate today = LocalDate.now();
        LocalDate in1 = today.plusDays(5);
        LocalDate out1 = in1.plusDays(6);
        reservationService.makeReservation(g1, roomService.getRoomById("D-201"), in1, out1, 3);
    }

    // Room operations
    public boolean addRoom(K2703047_Room room) {
        return roomService.addRoom(room);
    }

    public boolean updateRoom(String roomId, K2703047_Room updatedRoom) {
        return roomService.updateRoom(roomId, updatedRoom);
    }

    public boolean removeRoom(String roomId) {
        // Validation: cannot remove room if it has active reservations
        List<K2703047_Reservation> active = reservationService.getActiveReservationsForRoom(roomId);
        if (!active.isEmpty()) {
            throw new IllegalStateException("Cannot remove Room " + roomId + ": active reservations exist.");
        }
        return roomService.removeRoom(roomId);
    }

    public boolean updateRoomStatus(String roomId, K2703047_RoomStatus status) {
        return roomService.updateRoomStatus(roomId, status);
    }

    public List<K2703047_Room> getAllRooms() {
        return roomService.getAllRooms();
    }

    public K2703047_Room getRoomById(String roomId) {
        return roomService.getRoomById(roomId);
```

```
    }

    public List<K2703047_Room> searchAvailableRooms(K2703047_RoomType roomType, LocalDate checkIn, LocalDate
checkOut) {
        return roomService.searchAvailableRooms(roomType, checkIn, checkOut, reservationService);
    }

    // Guest operations
    public boolean registerGuest(String nicOrPassport, String fullName, String phone, String email) {
        K2703047_Guest guest = new K2703047_Guest(nicOrPassport, fullName, phone, email);
        return guestService.registerGuest(guest);
    }

    public K2703047_Guest getGuest(String nicOrPassport) {
        return guestService.getGuestByNicOrPassport(nicOrPassport);
    }

    public List<K2703047_Guest> getAllGuests() {
        return guestService.getAllGuests();
    }

    public List<K2703047_Guest> searchGuestsByName(String name) {
        return guestService.searchGuestsByName(name);
    }

    // Reservation operations
    public K2703047_Reservation makeReservation(String nicOrPassport, String roomId,
                                                LocalDate checkInDate, LocalDate checkOutDate,
                                                int numberOfGuests) {
        K2703047_Guest guest = guestService.getGuestByNicOrPassport(nicOrPassport);
        if (guest == null) {
            throw new IllegalArgumentException("Guest with NIC/Passport " + nicOrPassport + " is not
registered.");
        }
        K2703047_Room room = roomService.getRoomById(roomId);
        if (room == null) {
            throw new IllegalArgumentException("Room " + roomId + " does not exist.");
        }
        return reservationService.makeReservation(guest, room, checkInDate, checkOutDate, numberOfGuests);
    }

    public boolean cancelReservation(String bookingId) {
        return reservationService.cancelReservation(bookingId, LocalDateTime.now());
    }

    public boolean cancelReservationWithCustomTime(String bookingId, LocalDateTime customCancellationTime) {
        return reservationService.cancelReservation(bookingId, customCancellationTime);
    }

    public K2703047_Reservation getReservation(String bookingId) {
        return reservationService.getReservationById(bookingId);
    }

    public List<K2703047_Reservation> getAllReservations() {
        return reservationService.getAllReservations();
    }

    public List<K2703047_Reservation> searchReservationsByGuestName(String guestName) {
        return reservationService.searchReservationsByGuestName(guestName);
    }

    public List<K2703047_Reservation> getReservationsByCheckInDate(LocalDate checkInDate) {
```

```
        return reservationService.getReservationsByCheckInDate(checkInDate);
    }

    public K2703047_Reservation searchReservationBinary(String bookingId) {
        return reservationService.binarySearchByBookingId(bookingId);
    }

    public K2703047_Invoice generateInvoice(String bookingId) {
        return reservationService.generateInvoice(bookingId);
    }

    public K2703047_IRoomService getRoomService() {
        return roomService;
    }

    public K2703047_IGuestService getGuestService() {
        return guestService;
    }

    public K2703047_IReservationService getReservationService() {
        return reservationService;
    }
}
```

```
package k2703047.smartstay.service;

import k2703047.smartstay.model.K2703047_BedType;
import k2703047.smartstay.model.K2703047_FamilySuiteRoom;
import k2703047.smartstay.model.K2703047_Guest;
import k2703047.smartstay.model.K2703047_Invoice;
import k2703047.smartstay.model.K2703047_Reservation;
import k2703047.smartstay.model.K2703047_Room;
import k2703047.smartstay.model.K2703047_RoomFactory;
import k2703047.smartstay.model.K2703047_RoomStatus;
import k2703047.smartstay.model.K2703047_RoomType;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Facade Pattern implementation coordinating Room, Guest, and Reservation subsystems.
 * Acts as the centralized API for UI controllers (Console CLI & Desktop GUI).
 */
public class K2703047_HotelSystem {

    private final K2703047_IRoomService roomService;
    private final K2703047_IGuestService guestService;
    private final K2703047_IReservationService reservationService;

    public K2703047_HotelSystem() {
        this.roomService = new K2703047_RoomService();
        this.guestService = new K2703047_GuestService();
        this.reservationService = new K2703047_ReservationService();
        seedInitialData();
    }
```

```
    /**
     * Seeds initial realistic sample rooms, registered guests, and bookings.
     */
    private void seedInitialData() {
        // Seed Rooms across all types and bed options
        roomService.addRoom(K2703047_RoomFactory.createRoom("R-101", K2703047_RoomType.STANDARD,
K2703047_BedType.SINGLE));
        roomService.addRoom(K2703047_RoomFactory.createRoom("R-102", K2703047_RoomType.STANDARD,
K2703047_BedType.DOUBLE));
        roomService.addRoom(K2703047_RoomFactory.createRoom("D-201", K2703047_RoomType.DELUXE,
K2703047_BedType.QUEEN));
        roomService.addRoom(K2703047_RoomFactory.createRoom("D-202", K2703047_RoomType.DELUXE,
K2703047_BedType.KING));
        roomService.addRoom(K2703047_RoomFactory.createRoom("S-301", K2703047_RoomType.SUITE,
K2703047_BedType.KING));
        roomService.addRoom(K2703047_RoomFactory.createRoom("FS-401", K2703047_RoomType.FAMILY_SUITE,
K2703047_BedType.KING));

        // One room under maintenance for rule validation
        K2703047_Room maintRoom = K2703047_RoomFactory.createRoom("R-103", K2703047_RoomType.STANDARD,
K2703047_BedType.DOUBLE);
        maintRoom.setStatus(K2703047_RoomStatus.UNDER_MAINTENANCE);
        roomService.addRoom(maintRoom);

        // Seed Sample Guests
        K2703047_Guest g1 = new K2703047_Guest("199512345678", "Kasun Perera", "+94771234567",
"kasun.p@gmail.com");
        K2703047_Guest g2 = new K2703047_Guest("N87654321", "Sarah Jenkins", "+447911123456",
"sarah.j@outlook.com");
        guestService.registerGuest(g1);
        guestService.registerGuest(g2);

        // Seed an active reservation (e.g., checking in 5 days from today for 6 nights -> qualifies for 12%
discount)
        LocalDate today = LocalDate.now();
        LocalDate in1 = today.plusDays(5);
        LocalDate out1 = in1.plusDays(6);
        reservationService.makeReservation(g1, roomService.getRoomById("D-201"), in1, out1, 3);
    }

    // Room operations
    public boolean addRoom(K2703047_Room room) {
        return roomService.addRoom(room);
    }

    public boolean updateRoom(String roomId, K2703047_Room updatedRoom) {
        return roomService.updateRoom(roomId, updatedRoom);
    }

    public boolean removeRoom(String roomId) {
        // Validation: cannot remove room if it has active reservations
        List<K2703047_Reservation> active = reservationService.getActiveReservationsForRoom(roomId);
        if (!active.isEmpty()) {
            throw new IllegalStateException("Cannot remove Room " + roomId + ": active reservations exist.");
        }
        return roomService.removeRoom(roomId);
    }

    public boolean updateRoomStatus(String roomId, K2703047_RoomStatus status) {
        return roomService.updateRoomStatus(roomId, status);
    }
```

```
    public List<K2703047_Room> getAllRooms() {
        return roomService.getAllRooms();
    }

    public K2703047_Room getRoomById(String roomId) {
        return roomService.getRoomById(roomId);
    }

    public List<K2703047_Room> searchAvailableRooms(K2703047_RoomType roomType, LocalDate checkIn, LocalDate
checkOut) {
        return roomService.searchAvailableRooms(roomType, checkIn, checkOut, reservationService);
    }

    // Guest operations
    public boolean registerGuest(String nicOrPassport, String fullName, String phone, String email) {
        K2703047_Guest guest = new K2703047_Guest(nicOrPassport, fullName, phone, email);
        return guestService.registerGuest(guest);
    }

    public K2703047_Guest getGuest(String nicOrPassport) {
        return guestService.getGuestByNicOrPassport(nicOrPassport);
    }

    public List<K2703047_Guest> getAllGuests() {
        return guestService.getAllGuests();
    }

    public List<K2703047_Guest> searchGuestsByName(String name) {
        return guestService.searchGuestsByName(name);
    }

    // Reservation operations
    public K2703047_Reservation makeReservation(String nicOrPassport, String roomId,
                                                LocalDate checkInDate, LocalDate checkOutDate,
                                                int numberOfGuests) {
        K2703047_Guest guest = guestService.getGuestByNicOrPassport(nicOrPassport);
        if (guest == null) {
            throw new IllegalArgumentException("Guest with NIC/Passport " + nicOrPassport + " is not
registered.");
        }
        K2703047_Room room = roomService.getRoomById(roomId);
        if (room == null) {
            throw new IllegalArgumentException("Room " + roomId + " does not exist.");
        }
        return reservationService.makeReservation(guest, room, checkInDate, checkOutDate, numberOfGuests);
    }

    public boolean cancelReservation(String bookingId) {
        return reservationService.cancelReservation(bookingId, LocalDateTime.now());
    }

    public boolean cancelReservationWithCustomTime(String bookingId, LocalDateTime customCancellationTime) {
        return reservationService.cancelReservation(bookingId, customCancellationTime);
    }

    public K2703047_Reservation getReservation(String bookingId) {
        return reservationService.getReservationById(bookingId);
    }

    public List<K2703047_Reservation> getAllReservations() {
        return reservationService.getAllReservations();
    }
```

```
    public List<K2703047_Reservation> searchReservationsByGuestName(String guestName) {
        return reservationService.searchReservationsByGuestName(guestName);
    }

    public List<K2703047_Reservation> getReservationsByCheckInDate(LocalDate checkInDate) {
        return reservationService.getReservationsByCheckInDate(checkInDate);
    }

    public K2703047_Reservation searchReservationBinary(String bookingId) {
        return reservationService.binarySearchByBookingId(bookingId);
    }

    public K2703047_Invoice generateInvoice(String bookingId) {
        return reservationService.generateInvoice(bookingId);
    }

    public K2703047_IRoomService getRoomService() {
        return roomService;
    }

    public K2703047_IGuestService getGuestService() {
        return guestService;
    }

    public K2703047_IReservationService getReservationService() {
        return reservationService;
    }
}
```

```
package k2703047.smartstay.service;

import k2703047.smartstay.model.K2703047_Guest;

import java.util.List;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Interface defining customer registration and lookup operations.
 */
public interface K2703047_IGuestService {
    boolean registerGuest(K2703047_Guest guest);
    K2703047_Guest getGuestByNicOrPassport(String nicOrPassport);
    List<K2703047_Guest> getAllGuests();
    List<K2703047_Guest> searchGuestsByName(String nameKeyword);
    boolean guestExists(String nicOrPassport);
}
```

```
package k2703047.smartstay.service;

import k2703047.smartstay.model.K2703047_Guest;

import java.util.List;

/**
```

```
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Interface defining customer registration and lookup operations.
 */
public interface K2703047_IGuestService {
    boolean registerGuest(K2703047_Guest guest);
    K2703047_Guest getGuestByNicOrPassport(String nicOrPassport);
    List<K2703047_Guest> getAllGuests();
    List<K2703047_Guest> searchGuestsByName(String nameKeyword);
    boolean guestExists(String nicOrPassport);
}
```

```
package k2703047.smartstay.service;

import k2703047.smartstay.model.K2703047_Guest;
import k2703047.smartstay.model.K2703047_Invoice;
import k2703047.smartstay.model.K2703047_Reservation;
import k2703047.smartstay.model.K2703047_Room;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Interface defining reservation operations, search, cancellation, and invoice generation.
 */
public interface K2703047_IReservationService {
    K2703047_Reservation makeReservation(K2703047_Guest guest, K2703047_Room room,
                                         LocalDate checkInDate, LocalDate checkOutDate,
                                         int numberOfGuests);

    boolean cancelReservation(String bookingId, LocalDateTime cancellationTime);
    K2703047_Reservation getReservationById(String bookingId);
    List<K2703047_Reservation> getAllReservations();
    List<K2703047_Reservation> searchReservationsByGuestName(String guestNameKeyword);
    List<K2703047_Reservation> getReservationsByCheckInDate(LocalDate checkInDate);
    List<K2703047_Reservation> getActiveReservationsForRoom(String roomId);
    boolean hasRoomDateConflict(String roomId, LocalDate checkIn, LocalDate checkOut, String excludeBookingId);
    K2703047_Invoice generateInvoice(String bookingId);
    K2703047_Reservation binarySearchByBookingId(String bookingId);
}
```

```
package k2703047.smartstay.service;

import k2703047.smartstay.model.K2703047_Guest;
import k2703047.smartstay.model.K2703047_Invoice;
import k2703047.smartstay.model.K2703047_Reservation;
import k2703047.smartstay.model.K2703047_Room;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;
```

```
 /**
  * Kingston University Coursework - CI6115 Programming III
  * Student ID: 2703047
  * Module: Patterns and Algorithms
  *
  * Interface defining reservation operations, search, cancellation, and invoice generation.
  */
 public interface K2703047_IReservationService {
     K2703047_Reservation makeReservation(K2703047_Guest guest, K2703047_Room room,
                                          LocalDate checkInDate, LocalDate checkOutDate,
                                          int numberOfGuests);

     boolean cancelReservation(String bookingId, LocalDateTime cancellationTime);
     K2703047_Reservation getReservationById(String bookingId);
     List<K2703047_Reservation> getAllReservations();
     List<K2703047_Reservation> searchReservationsByGuestName(String guestNameKeyword);
     List<K2703047_Reservation> getReservationsByCheckInDate(LocalDate checkInDate);
     List<K2703047_Reservation> getActiveReservationsForRoom(String roomId);
     boolean hasRoomDateConflict(String roomId, LocalDate checkIn, LocalDate checkOut, String
 excludeBookingId);
     K2703047_Invoice generateInvoice(String bookingId);
     K2703047_Reservation binarySearchByBookingId(String bookingId);
 }
```

```
package k2703047.smartstay.service;

import k2703047.smartstay.model.K2703047_Room;
import k2703047.smartstay.model.K2703047_RoomStatus;
import k2703047.smartstay.model.K2703047_RoomType;

import java.time.LocalDate;
import java.util.List;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Interface defining room management operations (Demonstrating Abstraction).
 */
public interface K2703047_IRoomService {
    boolean addRoom(K2703047_Room room);
    boolean updateRoom(String roomId, K2703047_Room updatedRoom);
    boolean removeRoom(String roomId);
    boolean updateRoomStatus(String roomId, K2703047_RoomStatus newStatus);
    K2703047_Room getRoomById(String roomId);
    List<K2703047_Room> getAllRooms();
    List<K2703047_Room> getRoomsByType(K2703047_RoomType roomType);
    List<K2703047_Room> searchAvailableRooms(K2703047_RoomType roomType, LocalDate checkIn, LocalDate checkOut,
                                             K2703047_IReservationService reservationService);
    boolean roomExists(String roomId);
}
```

```
 package k2703047.smartstay.service;

 import k2703047.smartstay.model.K2703047_Room;
 import k2703047.smartstay.model.K2703047_RoomStatus;
```

```
import k2703047.smartstay.model.K2703047_RoomType;

import java.time.LocalDate;
import java.util.List;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Interface defining room management operations (Demonstrating Abstraction).
 */
public interface K2703047_IRoomService {
    boolean addRoom(K2703047_Room room);
    boolean updateRoom(String roomId, K2703047_Room updatedRoom);
    boolean removeRoom(String roomId);
    boolean updateRoomStatus(String roomId, K2703047_RoomStatus newStatus);
    K2703047_Room getRoomById(String roomId);
    List<K2703047_Room> getAllRooms();
    List<K2703047_Room> getRoomsByType(K2703047_RoomType roomType);
    List<K2703047_Room> searchAvailableRooms(K2703047_RoomType roomType, LocalDate checkIn, LocalDate
checkOut,
                                             K2703047_IReservationService reservationService);
    boolean roomExists(String roomId);
}
```

```
package k2703047.smartstay.service;

import k2703047.smartstay.model.K2703047_Guest;
import k2703047.smartstay.model.K2703047_Invoice;
import k2703047.smartstay.model.K2703047_Reservation;
import k2703047.smartstay.model.K2703047_ReservationStatus;
import k2703047.smartstay.model.K2703047_Room;
import k2703047.smartstay.model.K2703047_RoomStatus;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.concurrent.atomic.AtomicInteger;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Implementation of Reservation Management, Search, Cancellation, and Billing.
 * Demonstrates:
 * - Data Structures: LinkedHashMap (O(1) lookup), ArrayList, and TreeMap (Red-Black Tree for date indexing).
 * - Algorithms: Interval Conflict Detection, Advance Validation, 24-hr Cancellation Threshold,
 *               Binary Search, and 5-step Billing Calculation.
 */
public class K2703047_ReservationService implements K2703047_IReservationService {

    // Primary data store: Map for O(1) Booking ID lookups
```

```
    private final Map<String, K2703047_Reservation> reservationMap;

    // Ordered list for binary search and sorting operations
    private final List<K2703047_Reservation> reservationList;

    // Self-balancing search tree index: Check-in Date -> List of Reservations
    private final TreeMap<LocalDate, List<K2703047_Reservation>> checkInDateIndex;

    // Generator for unique sequential Booking IDs (e.g., BK-1001)
    private final AtomicInteger bookingSequence;

    public K2703047_ReservationService() {
        this.reservationMap = new LinkedHashMap<>();
        this.reservationList = new ArrayList<>();
        this.checkInDateIndex = new TreeMap<>();
        this.bookingSequence = new AtomicInteger(1001);
    }

    /**
     * Generates a unique sequential booking ID.
     */
    private synchronized String generateNextBookingId() {
        return "BK-" + bookingSequence.getAndIncrement();
    }

    @Override
    public synchronized K2703047_Reservation makeReservation(K2703047_Guest guest, K2703047_Room room,
                                                             LocalDate checkInDate, LocalDate checkOutDate,
                                                             int numberOfGuests) {
        if (guest == null) {
            throw new IllegalArgumentException("Cannot create reservation without a registered guest.");
        }
        if (room == null) {
            throw new IllegalArgumentException("Cannot create reservation without an assigned room.");
        }

        // Business rule 1: Room status must be AVAILABLE
        if (room.getStatus() != K2703047_RoomStatus.AVAILABLE) {
            throw new IllegalStateException("Room " + room.getRoomId() + " is currently " +
                    room.getStatus().getDisplayName() + " and cannot be reserved.");
        }

        // Business rule 2: Check-in must be at least 2 days in advance, check-out after check-in
        LocalDate today = LocalDate.now();
        K2703047_Reservation.validateDates(checkInDate, checkOutDate, today);

        // Business rule 3: Interval Conflict Detection Algorithm
        if (hasRoomDateConflict(room.getRoomId(), checkInDate, checkOutDate, null)) {
            throw new IllegalStateException("Room " + room.getRoomId() +
                    " already has a confirmed reservation overlapping with dates: " + checkInDate + " to " +
checkOutDate);
        }

        String bookingId = generateNextBookingId();
        K2703047_Reservation reservation = new K2703047_Reservation(
                bookingId, guest, room, checkInDate, checkOutDate, numberOfGuests, LocalDateTime.now());

        // Store into data structures
        reservationMap.put(bookingId, reservation);
        reservationList.add(reservation);

        // Update TreeMap date index
```

```
        checkInDateIndex.computeIfAbsent(checkInDate, k -> new ArrayList<>()).add(reservation);

        return reservation;
    }

    /**
     * Algorithm 1: Date Range Interval Conflict Detection Algorithm.
     * Evaluates half-open intervals [checkIn, checkOut) for active reservations of the same room.
     */
    @Override
    public boolean hasRoomDateConflict(String roomId, LocalDate checkIn, LocalDate checkOut, String
excludeBookingId) {
        if (roomId == null || checkIn == null || checkOut == null) {
            return false;
        }
        for (K2703047_Reservation r : reservationMap.values()) {
            if (r.getStatus() != K2703047_ReservationStatus.CANCELLED &&
                r.getRoom().getRoomId().equalsIgnoreCase(roomId)) {

                // Exclude current booking if modifying/checking
                if (excludeBookingId != null && r.getBookingId().equalsIgnoreCase(excludeBookingId)) {
                    continue;
                }

                // Conflict test: checkIn < existingCheckOut AND checkOut > existingCheckIn
                if (r.overlapsWith(checkIn, checkOut)) {
                    return true;
                }
            }
        }
        return false;
    }

    /**
     * Algorithm 2: 24-Hour Cancellation Eligibility Algorithm.
     * Reservations cannot be modified or cancelled within 24 hours of check-in date.
     * Refunds the LKR 10,000 booking advance upon successful cancellation.
     */
    @Override
    public synchronized boolean cancelReservation(String bookingId, LocalDateTime cancellationTime) {
        if (bookingId == null) {
            return false;
        }
        K2703047_Reservation reservation = getReservationById(bookingId);
        if (reservation == null) {
            return false;
        }
        LocalDateTime effectiveTime = (cancellationTime != null) ? cancellationTime : LocalDateTime.now();

        boolean cancelled = reservation.cancelReservation(effectiveTime);
        if (cancelled) {
            // Remove from check-in date index list if present
            List<K2703047_Reservation> list = checkInDateIndex.get(reservation.getCheckInDate());
            if (list != null) {
                list.remove(reservation);
            }
        }
        return cancelled;
    }

    @Override
    public K2703047_Reservation getReservationById(String bookingId) {
```

```
    if (bookingId == null) return null;
    return reservationMap.get(bookingId.trim().toUpperCase());
}

@Override
public List<K2703047_Reservation> getAllReservations() {
    return new ArrayList<>(reservationMap.values());
}

/**
 * Search Algorithm: Search reservations by guest name (case-insensitive substring).
 */
@Override
public List<K2703047_Reservation> searchReservationsByGuestName(String guestNameKeyword) {
    List<K2703047_Reservation> results = new ArrayList<>();
    if (guestNameKeyword == null || guestNameKeyword.trim().isEmpty()) {
        return getAllReservations();
    }
    String lowerKeyword = guestNameKeyword.trim().toLowerCase();
    for (K2703047_Reservation r : reservationMap.values()) {
        if (r.getGuest().getFullName().toLowerCase().contains(lowerKeyword)) {
            results.add(r);
        }
    }
    return results;
}

/**
 * Fast O(log N) retrieval using TreeMap self-balancing index.
 */
@Override
public List<K2703047_Reservation> getReservationsByCheckInDate(LocalDate checkInDate) {
    if (checkInDate == null) {
        return Collections.emptyList();
    }
    List<K2703047_Reservation> list = checkInDateIndex.get(checkInDate);
    return (list != null) ? new ArrayList<>(list) : Collections.emptyList();
}

@Override
public List<K2703047_Reservation> getActiveReservationsForRoom(String roomId) {
    List<K2703047_Reservation> results = new ArrayList<>();
    if (roomId == null) return results;
    for (K2703047_Reservation r : reservationMap.values()) {
        if (r.getStatus() == K2703047_ReservationStatus.CONFIRMED &&
            r.getRoom().getRoomId().equalsIgnoreCase(roomId)) {
            results.add(r);
        }
    }
    return results;
}

/**
 * Algorithm 3: Binary Search on sorted reservation list.
 * Time Complexity: O(log N)
 * Demonstrates knowledge of fundamental search algorithms.
 */
@Override
public K2703047_Reservation binarySearchByBookingId(String bookingId) {
    if (bookingId == null || bookingId.trim().isEmpty()) {
        return null;
    }
```

```
        String target = bookingId.trim().toUpperCase();

        // Ensure list is sorted by Booking ID for binary search
        List<K2703047_Reservation> sortedList = new ArrayList<>(reservationList);
        sortedList.sort(Comparator.comparing(K2703047_Reservation::getBookingId));

        int low = 0;
        int high = sortedList.size() - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;
            K2703047_Reservation midVal = sortedList.get(mid);
            int cmp = midVal.getBookingId().compareTo(target);

            if (cmp < 0) {
                low = mid + 1;
            } else if (cmp > 0) {
                high = mid - 1;
            } else {
                return midVal; // Found
            }
        }
        return null; // Not found
    }

    /**
     * Generates a detailed invoice for the given booking ID.
     */
    @Override
    public K2703047_Invoice generateInvoice(String bookingId) {
        K2703047_Reservation reservation = getReservationById(bookingId);
        if (reservation == null) {
            throw new IllegalArgumentException("No reservation found for Booking ID: " + bookingId);
        }
        return new K2703047_Invoice(reservation);
    }
}
```

```
package k2703047.smartstay.service;

import k2703047.smartstay.model.K2703047_Guest;
import k2703047.smartstay.model.K2703047_Invoice;
import k2703047.smartstay.model.K2703047_Reservation;
import k2703047.smartstay.model.K2703047_ReservationStatus;
import k2703047.smartstay.model.K2703047_Room;
import k2703047.smartstay.model.K2703047_RoomStatus;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.concurrent.atomic.AtomicInteger;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
```

```
 * Module: Patterns and Algorithms
 *
 * Implementation of Reservation Management, Search, Cancellation, and Billing.
 * Demonstrates:
 * - Data Structures: LinkedHashMap (O(1) lookup), ArrayList, and TreeMap (Red-Black Tree for date indexing).
 * - Algorithms: Interval Conflict Detection, Advance Validation, 24-hr Cancellation Threshold,
 *               Binary Search, and 5-step Billing Calculation.
 */
public class K2703047_ReservationService implements K2703047_IReservationService {

    // Primary data store: Map for O(1) Booking ID lookups
    private final Map<String, K2703047_Reservation> reservationMap;

    // Ordered list for binary search and sorting operations
    private final List<K2703047_Reservation> reservationList;

    // Self-balancing search tree index: Check-in Date -> List of Reservations
    private final TreeMap<LocalDate, List<K2703047_Reservation>> checkInDateIndex;

    // Generator for unique sequential Booking IDs (e.g., BK-1001)
    private final AtomicInteger bookingSequence;

    public K2703047_ReservationService() {
        this.reservationMap = new LinkedHashMap<>();
        this.reservationList = new ArrayList<>();
        this.checkInDateIndex = new TreeMap<>();
        this.bookingSequence = new AtomicInteger(1001);
    }

    /**
     * Generates a unique sequential booking ID.
     */
    private synchronized String generateNextBookingId() {
        return "BK-" + bookingSequence.getAndIncrement();
    }

    @Override
    public synchronized K2703047_Reservation makeReservation(K2703047_Guest guest, K2703047_Room room,
                                                             LocalDate checkInDate, LocalDate checkOutDate,
                                                             int numberOfGuests) {
        if (guest == null) {
            throw new IllegalArgumentException("Cannot create reservation without a registered guest.");
        }
        if (room == null) {
            throw new IllegalArgumentException("Cannot create reservation without an assigned room.");
        }

        // Business rule 1: Room status must be AVAILABLE
        if (room.getStatus() != K2703047_RoomStatus.AVAILABLE) {
            throw new IllegalStateException("Room " + room.getRoomId() + " is currently " +
                    room.getStatus().getDisplayName() + " and cannot be reserved.");
        }

        // Business rule 2: Check-in must be at least 2 days in advance, check-out after check-in
        LocalDate today = LocalDate.now();
        K2703047_Reservation.validateDates(checkInDate, checkOutDate, today);

        // Business rule 3: Interval Conflict Detection Algorithm
        if (hasRoomDateConflict(room.getRoomId(), checkInDate, checkOutDate, null)) {
            throw new IllegalStateException("Room " + room.getRoomId() +
                    " already has a confirmed reservation overlapping with dates: " + checkInDate + " to " +
checkOutDate);
```

```
        }

        String bookingId = generateNextBookingId();
        K2703047_Reservation reservation = new K2703047_Reservation(
                bookingId, guest, room, checkInDate, checkOutDate, numberOfGuests, LocalDateTime.now());

        // Store into data structures
        reservationMap.put(bookingId, reservation);
        reservationList.add(reservation);

        // Update TreeMap date index
        checkInDateIndex.computeIfAbsent(checkInDate, k -> new ArrayList<>()).add(reservation);

        return reservation;
    }

    /**
     * Algorithm 1: Date Range Interval Conflict Detection Algorithm.
     * Evaluates half-open intervals [checkIn, checkOut) for active reservations of the same room.
     */
    @Override
    public boolean hasRoomDateConflict(String roomId, LocalDate checkIn, LocalDate checkOut, String
excludeBookingId) {
        if (roomId == null || checkIn == null || checkOut == null) {
            return false;
        }
        for (K2703047_Reservation r : reservationMap.values()) {
            if (r.getStatus() != K2703047_ReservationStatus.CANCELLED &&
                r.getRoom().getRoomId().equalsIgnoreCase(roomId)) {

                // Exclude current booking if modifying/checking
                if (excludeBookingId != null && r.getBookingId().equalsIgnoreCase(excludeBookingId)) {
                    continue;
                }

                // Conflict test: checkIn < existingCheckOut AND checkOut > existingCheckIn
                if (r.overlapsWith(checkIn, checkOut)) {
                    return true;
                }
            }
        }
        return false;
    }

    /**
     * Algorithm 2: 24-Hour Cancellation Eligibility Algorithm.
     * Reservations cannot be modified or cancelled within 24 hours of check-in date.
     * Refunds the LKR 10,000 booking advance upon successful cancellation.
     */
    @Override
    public synchronized boolean cancelReservation(String bookingId, LocalDateTime cancellationTime) {
        if (bookingId == null) {
            return false;
        }
        K2703047_Reservation reservation = getReservationById(bookingId);
        if (reservation == null) {
            return false;
        }
        LocalDateTime effectiveTime = (cancellationTime != null) ? cancellationTime : LocalDateTime.now();

        boolean cancelled = reservation.cancelReservation(effectiveTime);
        if (cancelled) {
```

```
            // Remove from check-in date index list if present
            List<K2703047_Reservation> list = checkInDateIndex.get(reservation.getCheckInDate());
            if (list != null) {
                list.remove(reservation);
            }
        }
        return cancelled;
    }

    @Override
    public K2703047_Reservation getReservationById(String bookingId) {
        if (bookingId == null) return null;
        return reservationMap.get(bookingId.trim().toUpperCase());
    }

    @Override
    public List<K2703047_Reservation> getAllReservations() {
        return new ArrayList<>(reservationMap.values());
    }

    /**
     * Search Algorithm: Search reservations by guest name (case-insensitive substring).
     */
    @Override
    public List<K2703047_Reservation> searchReservationsByGuestName(String guestNameKeyword) {
        List<K2703047_Reservation> results = new ArrayList<>();
        if (guestNameKeyword == null || guestNameKeyword.trim().isEmpty()) {
            return getAllReservations();
        }
        String lowerKeyword = guestNameKeyword.trim().toLowerCase();
        for (K2703047_Reservation r : reservationMap.values()) {
            if (r.getGuest().getFullName().toLowerCase().contains(lowerKeyword)) {
                results.add(r);
            }
        }
        return results;
    }

    /**
     * Fast O(log N) retrieval using TreeMap self-balancing index.
     */
    @Override
    public List<K2703047_Reservation> getReservationsByCheckInDate(LocalDate checkInDate) {
        if (checkInDate == null) {
            return Collections.emptyList();
        }
        List<K2703047_Reservation> list = checkInDateIndex.get(checkInDate);
        return (list != null) ? new ArrayList<>(list) : Collections.emptyList();
    }

    @Override
    public List<K2703047_Reservation> getActiveReservationsForRoom(String roomId) {
        List<K2703047_Reservation> results = new ArrayList<>();
        if (roomId == null) return results;
        for (K2703047_Reservation r : reservationMap.values()) {
            if (r.getStatus() == K2703047_ReservationStatus.CONFIRMED &&
                r.getRoom().getRoomId().equalsIgnoreCase(roomId)) {
                results.add(r);
            }
        }
        return results;
    }
```

```
    /**
     * Algorithm 3: Binary Search on sorted reservation list.
     * Time Complexity: O(log N)
     * Demonstrates knowledge of fundamental search algorithms.
     */
    @Override
    public K2703047_Reservation binarySearchByBookingId(String bookingId) {
        if (bookingId == null || bookingId.trim().isEmpty()) {
            return null;
        }
        String target = bookingId.trim().toUpperCase();

        // Ensure list is sorted by Booking ID for binary search
        List<K2703047_Reservation> sortedList = new ArrayList<>(reservationList);
        sortedList.sort(Comparator.comparing(K2703047_Reservation::getBookingId));

        int low = 0;
        int high = sortedList.size() - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;
            K2703047_Reservation midVal = sortedList.get(mid);
            int cmp = midVal.getBookingId().compareTo(target);

            if (cmp < 0) {
                low = mid + 1;
            } else if (cmp > 0) {
                high = mid - 1;
            } else {
                return midVal; // Found
            }
        }
        return null; // Not found
    }

    /**
     * Generates a detailed invoice for the given booking ID.
     */
    @Override
    public K2703047_Invoice generateInvoice(String bookingId) {
        K2703047_Reservation reservation = getReservationById(bookingId);
        if (reservation == null) {
            throw new IllegalArgumentException("No reservation found for Booking ID: " + bookingId);
        }
        return new K2703047_Invoice(reservation);
    }
}
```

```
package k2703047.smartstay.service;

import k2703047.smartstay.model.K2703047_Room;
import k2703047.smartstay.model.K2703047_RoomStatus;
import k2703047.smartstay.model.K2703047_RoomType;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
```

```
import java.util.stream.Collectors;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Implementation of Room Management functionalities for administrators.
 * Data Structure: LinkedHashMap provides O(1) search/insertion while preserving deterministic ordering.
 */
public class K2703047_RoomService implements K2703047_IRoomService {

    // Primary data structure for room records: Key = Room ID, Value = Room Object
    private final Map<String, K2703047_Room> roomRegistry;

    public K2703047_RoomService() {
        this.roomRegistry = new LinkedHashMap<>();
    }

    @Override
    public synchronized boolean addRoom(K2703047_Room room) {
        if (room == null || roomRegistry.containsKey(room.getRoomId())) {
            return false;
        }
        roomRegistry.put(room.getRoomId(), room);
        return true;
    }

    @Override
    public synchronized boolean updateRoom(String roomId, K2703047_Room updatedRoom) {
        if (roomId == null || updatedRoom == null) {
            return false;
        }
        String normalizedId = roomId.trim().toUpperCase();
        if (!roomRegistry.containsKey(normalizedId)) {
            return false;
        }
        roomRegistry.put(normalizedId, updatedRoom);
        return true;
    }

    @Override
    public synchronized boolean removeRoom(String roomId) {
        if (roomId == null) {
            return false;
        }
        String normalizedId = roomId.trim().toUpperCase();
        return roomRegistry.remove(normalizedId) != null;
    }

    @Override
    public synchronized boolean updateRoomStatus(String roomId, K2703047_RoomStatus newStatus) {
        if (roomId == null || newStatus == null) {
            return false;
        }
        String normalizedId = roomId.trim().toUpperCase();
        K2703047_Room room = roomRegistry.get(normalizedId);
        if (room != null) {
            room.setStatus(newStatus);
            return true;
        }
        return false;
```

```
    }

    @Override
    public K2703047_Room getRoomById(String roomId) {
        if (roomId == null) return null;
        return roomRegistry.get(roomId.trim().toUpperCase());
    }

    @Override
    public List<K2703047_Room> getAllRooms() {
        return new ArrayList<>(roomRegistry.values());
    }

    @Override
    public List<K2703047_Room> getRoomsByType(K2703047_RoomType roomType) {
        if (roomType == null) {
            return getAllRooms();
        }
        List<K2703047_Room> matchingRooms = new ArrayList<>();
        for (K2703047_Room room : roomRegistry.values()) {
            if (room.getRoomType() == roomType) {
                matchingRooms.add(room);
            }
        }
        return matchingRooms;
    }

    /**
     * Search Algorithm: Filters available rooms based on Room Type, current status,
     * and checks for date conflicts across existing reservations.
     */
    @Override
    public List<K2703047_Room> searchAvailableRooms(K2703047_RoomType roomType, LocalDate checkIn, LocalDate
checkOut,
                                                    K2703047_IReservationService reservationService) {
        List<K2703047_Room> candidateRooms = (roomType != null) ? getRoomsByType(roomType) : getAllRooms();
        List<K2703047_Room> availableRooms = new ArrayList<>();

        for (K2703047_Room room : candidateRooms) {
            // Business rule: Room status must be AVAILABLE (not UNDER_MAINTENANCE)
            if (room.getStatus() != K2703047_RoomStatus.AVAILABLE) {
                continue;
            }
            // Check if there is an overlapping reservation during the requested interval
            if (reservationService != null && checkIn != null && checkOut != null) {
                boolean hasConflict = reservationService.hasRoomDateConflict(room.getRoomId(), checkIn,
checkOut, null);
                if (hasConflict) {
                    continue;
                }
            }
            availableRooms.add(room);
        }
        return availableRooms;
    }

    @Override
    public boolean roomExists(String roomId) {
        if (roomId == null) return false;
        return roomRegistry.containsKey(roomId.trim().toUpperCase());
    }
```

}

```
package k2703047.smartstay.service;

import k2703047.smartstay.model.K2703047_Room;
import k2703047.smartstay.model.K2703047_RoomStatus;
import k2703047.smartstay.model.K2703047_RoomType;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Implementation of Room Management functionalities for administrators.
 * Data Structure: LinkedHashMap provides O(1) search/insertion while preserving deterministic ordering.
 */
public class K2703047_RoomService implements K2703047_IRoomService {

    // Primary data structure for room records: Key = Room ID, Value = Room Object
    private final Map<String, K2703047_Room> roomRegistry;

    public K2703047_RoomService() {
        this.roomRegistry = new LinkedHashMap<>();
    }

    @Override
    public synchronized boolean addRoom(K2703047_Room room) {
        if (room == null || roomRegistry.containsKey(room.getRoomId())) {
            return false;
        }
        roomRegistry.put(room.getRoomId(), room);
        return true;
    }

    @Override
    public synchronized boolean updateRoom(String roomId, K2703047_Room updatedRoom) {
        if (roomId == null || updatedRoom == null) {
            return false;
        }
        String normalizedId = roomId.trim().toUpperCase();
        if (!roomRegistry.containsKey(normalizedId)) {
            return false;
        }
        roomRegistry.put(normalizedId, updatedRoom);
        return true;
    }

    @Override
    public synchronized boolean removeRoom(String roomId) {
        if (roomId == null) {
            return false;
        }
        String normalizedId = roomId.trim().toUpperCase();
        return roomRegistry.remove(normalizedId) != null;
```

```
    }

    @Override
    public synchronized boolean updateRoomStatus(String roomId, K2703047_RoomStatus newStatus) {
        if (roomId == null || newStatus == null) {
            return false;
        }
        String normalizedId = roomId.trim().toUpperCase();
        K2703047_Room room = roomRegistry.get(normalizedId);
        if (room != null) {
            room.setStatus(newStatus);
            return true;
        }
        return false;
    }

    @Override
    public K2703047_Room getRoomById(String roomId) {
        if (roomId == null) return null;
        return roomRegistry.get(roomId.trim().toUpperCase());
    }

    @Override
    public List<K2703047_Room> getAllRooms() {
        return new ArrayList<>(roomRegistry.values());
    }

    @Override
    public List<K2703047_Room> getRoomsByType(K2703047_RoomType roomType) {
        if (roomType == null) {
            return getAllRooms();
        }
        List<K2703047_Room> matchingRooms = new ArrayList<>();
        for (K2703047_Room room : roomRegistry.values()) {
            if (room.getRoomType() == roomType) {
                matchingRooms.add(room);
            }
        }
        return matchingRooms;
    }

    /**
     * Search Algorithm: Filters available rooms based on Room Type, current status,
     * and checks for date conflicts across existing reservations.
     */
    @Override
    public List<K2703047_Room> searchAvailableRooms(K2703047_RoomType roomType, LocalDate checkIn, LocalDate
checkOut,
                                                    K2703047_IReservationService reservationService) {
        List<K2703047_Room> candidateRooms = (roomType != null) ? getRoomsByType(roomType) : getAllRooms();
        List<K2703047_Room> availableRooms = new ArrayList<>();

        for (K2703047_Room room : candidateRooms) {
            // Business rule: Room status must be AVAILABLE (not UNDER_MAINTENANCE)
            if (room.getStatus() != K2703047_RoomStatus.AVAILABLE) {
                continue;
            }
            // Check if there is an overlapping reservation during the requested interval
            if (reservationService != null && checkIn != null && checkOut != null) {
                boolean hasConflict = reservationService.hasRoomDateConflict(room.getRoomId(), checkIn,
checkOut, null);
                if (hasConflict) {
```

```
                    continue;
                }
            }
            availableRooms.add(room);
        }
        return availableRooms;
    }

    @Override
    public boolean roomExists(String roomId) {
        if (roomId == null) return false;
        return roomRegistry.containsKey(roomId.trim().toUpperCase());
    }
}
```

```
package k2703047.smartstay.test;

import k2703047.smartstay.model.K2703047_BedType;
import k2703047.smartstay.model.K2703047_FamilySuiteRoom;
import k2703047.smartstay.model.K2703047_Guest;
import k2703047.smartstay.model.K2703047_Invoice;
import k2703047.smartstay.model.K2703047_Reservation;
import k2703047.smartstay.model.K2703047_ReservationStatus;
import k2703047.smartstay.model.K2703047_Room;
import k2703047.smartstay.model.K2703047_RoomFactory;
import k2703047.smartstay.model.K2703047_RoomStatus;
import k2703047.smartstay.model.K2703047_RoomType;
import k2703047.smartstay.service.K2703047_HotelSystem;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Comprehensive Automated Test Suite validating Normal, Boundary, and Invalid Test Cases.
 * Aligned with Module Learning Outcomes (LO2, LO5, LO6).
 */
public class K2703047_TestSuite {

    public static class TestResult {
        public final String testId;
        public final String category;
        public final String description;
        public final boolean passed;
        public final String details;

        public TestResult(String testId, String category, String description, boolean passed, String details) {
            this.testId = testId;
            this.category = category;
            this.description = description;
            this.passed = passed;
            this.details = details;
        }
    }
```

```
    public static void main(String[] args) {
        runAllTests();
    }

    public static List<TestResult> runAllTests() {
        List<TestResult> results = new ArrayList<>();
        int passedCount = 0;


System.out.println("=============================================================================================")
;
        System.out.println("            SMARTSTAY HOTEL BOOKING SYSTEM - COMPREHENSIVE TEST SUITE
");
        System.out.println("       Kingston University CI6115 | Student ID: 2703047 | Module Learning Outcome
LO5     ");

System.out.println("=============================================================================================")
;

        // ========================================
        // SECTION 1: NORMAL TEST CASES
        // ========================================
        // TC01: Customer Registration
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            boolean ok = hs.registerGuest("987654321V", "Amara Silva", "+94712345678", "amara.silva@gmail.com");
            results.add(new TestResult("TC01", "Normal", "Guest registration with valid details", ok, "Guest
stored and retrievable"));
        } catch (Exception e) {
            results.add(new TestResult("TC01", "Normal", "Guest registration with valid details", false,
e.getMessage()));
        }

        // TC02: Make Reservation for Standard Room
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            K2703047_Reservation res = hs.makeReservation("199512345678", "R-101", today.plusDays(3),
today.plusDays(6), 2);
            boolean ok = res != null && res.getBookingId().startsWith("BK-") && res.getNumberOfNights() == 3;
            results.add(new TestResult("TC02", "Normal", "Create valid 3-night reservation", ok, "Booking
created with ID " + res.getBookingId()));
        } catch (Exception e) {
            results.add(new TestResult("TC02", "Normal", "Create valid 3-night reservation", false,
e.getMessage()));
        }

        // TC03: 5-Step Billing Calculation (Deluxe Room, 3 nights, no discount)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            // Room D-202 (Deluxe, 9000/night, capacity 2, tax 12%)
            K2703047_Reservation res = hs.makeReservation("199512345678", "D-202", today.plusDays(4),
today.plusDays(7), 2);
            K2703047_Invoice inv = hs.generateInvoice(res.getBookingId());

            // Expected calculations:
            // Base: 9,000 * 3 = 27,000
            // Extra: 0
            // Discount: 0 (< 5 nights)
            // Net: 27,000
            // Tax: 12% of 27,000 = 3,240
```

```
            // Total Gross: 30,240
            // Advance: 10,000
            // Final Payable: 20,240
            boolean ok = Math.abs(inv.getBaseRoomCharge() - 27000.0) < 0.01 &&
                         Math.abs(inv.getTaxAmount() - 3240.0) < 0.01 &&
                         Math.abs(inv.getFinalPayableAmount() - 20240.0) < 0.01;
            results.add(new TestResult("TC03", "Normal", "5-Step billing calculation for Deluxe Room", ok,
                    String.format("Base: LKR %,.2f | Tax: LKR %,.2f | Final: LKR %,.2f",
                            inv.getBaseRoomCharge(), inv.getTaxAmount(), inv.getFinalPayableAmount())));
        } catch (Exception e) {
            results.add(new TestResult("TC03", "Normal", "5-Step billing calculation for Deluxe Room", false,
e.getMessage()));
        }

        // TC04: Long-Stay Discount (>= 5 nights => 12% discount)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            // Standard Room R-102 (6000/night, tax 10%), 6 nights
            K2703047_Reservation res = hs.makeReservation("199512345678", "R-102", today.plusDays(3),
today.plusDays(9), 2);
            K2703047_Invoice inv = hs.generateInvoice(res.getBookingId());

            // Base = 6,000 * 6 = 36,000
            // Discount = 12% of 36,000 = 4,320
            // Net = 36,000 - 4,320 = 31,680
            // Tax = 10% of 31,680 = 3,168
            // Total = 34,848 - 10,000 = 24,848
            boolean ok = inv.isDiscountApplied() &&
                         Math.abs(inv.getDiscountAmount() - 4320.0) < 0.01 &&
                         Math.abs(inv.getFinalPayableAmount() - 24848.0) < 0.01;
            results.add(new TestResult("TC04", "Normal", "12% long-stay discount applied for 6 nights", ok,
                    String.format("Base: 36,000 | Discount: LKR %,.2f | Payable: LKR %,.2f",
inv.getDiscountAmount(), inv.getFinalPayableAmount())));
        } catch (Exception e) {
            results.add(new TestResult("TC04", "Normal", "12% long-stay discount applied for 6 nights", false,
e.getMessage()));
        }

        // TC05: Extra Guest Charge Calculation
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            // Suite Room S-301 (14,000/night, capacity 3, extra guest 2,000/night, tax 15%), 2 nights, 5 guests
(2 extra)
            K2703047_Reservation res = hs.makeReservation("199512345678", "S-301", today.plusDays(3),
today.plusDays(5), 5);
            K2703047_Invoice inv = hs.generateInvoice(res.getBookingId());

            // Base = 14,000 * 2 = 28,000
            // Extra = 2 extra guests * 2,000 * 2 nights = 8,000
            // Net = 28,000 + 8,000 = 36,000
            // Tax = 15% of 36,000 = 5,400
            // Total = 41,400 - 10,000 = 31,400
            boolean ok = inv.getExtraGuestCount() == 2 &&
                         Math.abs(inv.getExtraGuestCharges() - 8000.0) < 0.01 &&
                         Math.abs(inv.getFinalPayableAmount() - 31400.0) < 0.01;
            results.add(new TestResult("TC05", "Normal", "Extra guest charge calculation (2 extra guests)", ok,
                    String.format("Extra guests: %d | Extra charge: LKR %,.2f", inv.getExtraGuestCount(),
inv.getExtraGuestCharges())));
        } catch (Exception e) {
            results.add(new TestResult("TC05", "Normal", "Extra guest charge calculation (2 extra guests)",
```

```
false, e.getMessage()));
        }

        // TC06: Search Available Rooms by Room Type and Dates
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            List<K2703047_Room> suites = hs.searchAvailableRooms(K2703047_RoomType.SUITE, today.plusDays(3),
today.plusDays(5));
            boolean ok = !suites.isEmpty() && suites.get(0).getRoomType() == K2703047_RoomType.SUITE;
            results.add(new TestResult("TC06", "Normal", "Search available rooms by type and date", ok, "Found "
+ suites.size() + " suite(s)"));
        } catch (Exception e) {
            results.add(new TestResult("TC06", "Normal", "Search available rooms by type and date", false,
e.getMessage()));
        }

        // TC07: TreeMap Red-Black Tree Date Index Retrieval
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            LocalDate checkIn = today.plusDays(10);
            hs.makeReservation("199512345678", "R-101", checkIn, checkIn.plusDays(2), 2);
            List<K2703047_Reservation> onDate = hs.getReservationsByCheckInDate(checkIn);
            boolean ok = !onDate.isEmpty() && onDate.get(0).getCheckInDate().equals(checkIn);
            results.add(new TestResult("TC07", "Normal", "View reservations by check-in date (TreeMap)", ok,
"Indexed via TreeMap in O(log N)"));
        } catch (Exception e) {
            results.add(new TestResult("TC07", "Normal", "View reservations by check-in date (TreeMap)", false,
e.getMessage()));
        }

        // TC08: Binary Search Algorithm by Booking ID
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            K2703047_Reservation res = hs.makeReservation("199512345678", "FS-401", today.plusDays(7),
today.plusDays(10), 4);
            K2703047_Reservation found = hs.searchReservationBinary(res.getBookingId());
            boolean ok = found != null && found.getBookingId().equals(res.getBookingId());
            results.add(new TestResult("TC08", "Normal", "Binary Search on sorted reservations by ID", ok,
"Located " + res.getBookingId() + " in O(log N) time"));
        } catch (Exception e) {
            results.add(new TestResult("TC08", "Normal", "Binary Search on sorted reservations by ID", false,
e.getMessage()));
        }

        // ========================================
        // SECTION 2: BOUNDARY TEST CASES
        // ========================================
        // TC09: Advance Booking Boundary - Exactly 2 days in advance (Must SUCCEED)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            K2703047_Reservation res = hs.makeReservation("199512345678", "R-101", today.plusDays(2),
today.plusDays(4), 2);
            boolean ok = res != null;
            results.add(new TestResult("TC09", "Boundary", "Advance booking boundary: exactly 2 days ahead", ok,
"Accepted (boundary = 2 days)"));
        } catch (Exception e) {
            results.add(new TestResult("TC09", "Boundary", "Advance booking boundary: exactly 2 days ahead",
false, e.getMessage()));
```

```
        }

        // TC10: Advance Booking Boundary - 1 day in advance (Must FAIL)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            hs.makeReservation("199512345678", "R-101", today.plusDays(1), today.plusDays(3), 2);
            results.add(new TestResult("TC10", "Boundary", "Advance booking boundary: 1 day ahead
(insufficient)", false, "Should have failed"));
        } catch (IllegalArgumentException e) {
            results.add(new TestResult("TC10", "Boundary", "Advance booking boundary: 1 day ahead
(insufficient)", true, "Correctly rejected: " + e.getMessage()));
        } catch (Exception e) {
            results.add(new TestResult("TC10", "Boundary", "Advance booking boundary: 1 day ahead
(insufficient)", false, e.getMessage()));
        }

        // TC11: Discount Boundary - Exactly 5 nights stay (12% discount MUST apply)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            K2703047_Reservation res = hs.makeReservation("199512345678", "R-102", today.plusDays(3),
today.plusDays(8), 2); // 5 nights
            K2703047_Invoice inv = hs.generateInvoice(res.getBookingId());
            boolean ok = inv.getNumberOfNights() == 5 && inv.isDiscountApplied() && inv.getDiscountRate() ==
12.0;
            results.add(new TestResult("TC11", "Boundary", "Discount boundary: exactly 5 nights stay", ok, "12%
discount correctly triggered"));
        } catch (Exception e) {
            results.add(new TestResult("TC11", "Boundary", "Discount boundary: exactly 5 nights stay", false,
e.getMessage()));
        }

        // TC12: Discount Boundary - 4 nights stay (12% discount MUST NOT apply)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            K2703047_Reservation res = hs.makeReservation("199512345678", "R-102", today.plusDays(3),
today.plusDays(7), 2); // 4 nights
            K2703047_Invoice inv = hs.generateInvoice(res.getBookingId());
            boolean ok = inv.getNumberOfNights() == 4 && !inv.isDiscountApplied() && inv.getDiscountAmount() ==
0.0;
            results.add(new TestResult("TC12", "Boundary", "Discount boundary: 4 nights stay (no discount)", ok,
"0% discount correctly assigned"));
        } catch (Exception e) {
            results.add(new TestResult("TC12", "Boundary", "Discount boundary: 4 nights stay (no discount)",
false, e.getMessage()));
        }

        // TC13: Cancellation Boundary - Exactly 25 hours before check-in date (Must SUCCEED)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate checkIn = LocalDate.now().plusDays(5);
            K2703047_Reservation res = hs.makeReservation("199512345678", "R-102", checkIn, checkIn.plusDays(2),
2);

            // Simulation: Cancellation requested 25 hours before check-in date start
            LocalDateTime simTime = checkIn.atStartOfDay().minusHours(25);
            boolean cancelled = hs.cancelReservationWithCustomTime(res.getBookingId(), simTime);
            boolean ok = cancelled && res.getStatus() == K2703047_ReservationStatus.CANCELLED &&
res.isAdvanceRefunded();
            results.add(new TestResult("TC13", "Boundary", "Cancellation boundary: 25 hours before check-in",
```

```
ok, "Cancellation accepted and advance refunded"));
        } catch (Exception e) {
            results.add(new TestResult("TC13", "Boundary", "Cancellation boundary: 25 hours before check-in",
false, e.getMessage()));
        }

        // TC14: Cancellation Boundary - 23 hours before check-in date (Must FAIL)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate checkIn = LocalDate.now().plusDays(5);
            K2703047_Reservation res = hs.makeReservation("199512345678", "R-102", checkIn, checkIn.plusDays(2),
2);

            // Simulation: Cancellation requested 23 hours before check-in date start (within 24 hours!)
            LocalDateTime simTime = checkIn.atStartOfDay().minusHours(23);
            boolean cancelled = hs.cancelReservationWithCustomTime(res.getBookingId(), simTime);
            boolean ok = !cancelled && res.getStatus() == K2703047_ReservationStatus.CONFIRMED;
            results.add(new TestResult("TC14", "Boundary", "Cancellation boundary: 23 hours before check-in
(within 24h)", ok, "Cancellation correctly rejected"));
        } catch (Exception e) {
            results.add(new TestResult("TC14", "Boundary", "Cancellation boundary: 23 hours before check-in
(within 24h)", false, e.getMessage()));
        }

        // TC15: Capacity Boundary - Guest count equal to base capacity (0 extra guest charge)
        try {
            K2703047_Room room = K2703047_RoomFactory.createRoom("R-TEST", K2703047_RoomType.STANDARD,
K2703047_BedType.DOUBLE);
            double extra = room.calculateExtraGuestCharge(room.getBaseCapacity(), 3);
            boolean ok = (extra == 0.0);
            results.add(new TestResult("TC15", "Boundary", "Capacity boundary: guests = capacity (2 guests in
Standard)", ok, "Extra charge = LKR 0.00"));
        } catch (Exception e) {
            results.add(new TestResult("TC15", "Boundary", "Capacity boundary: guests = capacity", false,
e.getMessage()));
        }

        // TC16: Capacity Boundary - Guest count = base capacity + 1 (Exactly 1 extra guest charged)
        try {
            K2703047_Room room = K2703047_RoomFactory.createRoom("R-TEST", K2703047_RoomType.STANDARD,
K2703047_BedType.DOUBLE);
            // capacity is 2, 3 guests for 2 nights => 1 extra guest * 1000 * 2 = 2000
            double extra = room.calculateExtraGuestCharge(room.getBaseCapacity() + 1, 2);
            boolean ok = (extra == 2000.0);
            results.add(new TestResult("TC16", "Boundary", "Capacity boundary: guests = capacity + 1 (3 guests,
2 nights)", ok, "Extra charge = LKR 2,000.00"));
        } catch (Exception e) {
            results.add(new TestResult("TC16", "Boundary", "Capacity boundary: guests = capacity + 1", false,
e.getMessage()));
        }

        // ========================================
        // SECTION 3: INVALID / ERROR TEST CASES
        // ========================================
        // TC17: Booking room under maintenance (Must FAIL)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            // Room R-103 is seeded as UNDER_MAINTENANCE
            hs.makeReservation("199512345678", "R-103", today.plusDays(3), today.plusDays(5), 2);
            results.add(new TestResult("TC17", "Invalid", "Book room with status UNDER_MAINTENANCE", false,
"Should have been rejected"));
```

```
        } catch (IllegalStateException e) {
            results.add(new TestResult("TC17", "Invalid", "Book room with status UNDER_MAINTENANCE", true,
"Correctly rejected: " + e.getMessage()));
        } catch (Exception e) {
            results.add(new TestResult("TC17", "Invalid", "Book room with status UNDER_MAINTENANCE", false,
e.getMessage()));
        }

        // TC18: Double booking on overlapping dates (Must FAIL via Conflict Algorithm)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            LocalDate in1 = today.plusDays(5);
            LocalDate out1 = in1.plusDays(4); // Day 5 to Day 9
            hs.makeReservation("199512345678", "R-101", in1, out1, 2);

            // Attempt overlapping booking: Day 6 to Day 8
            hs.makeReservation("N87654321", "R-101", in1.plusDays(1), in1.plusDays(3), 2);
            results.add(new TestResult("TC18", "Invalid", "Double-booking overlapping date range", false,
"Should have rejected overlapping interval"));
        } catch (IllegalStateException e) {
            results.add(new TestResult("TC18", "Invalid", "Double-booking overlapping date range", true,
"Conflict detected and prevented: " + e.getMessage()));
        } catch (Exception e) {
            results.add(new TestResult("TC18", "Invalid", "Double-booking overlapping date range", false,
e.getMessage()));
        }

        // TC19: Check-out date before or equal to check-in date (Must FAIL)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            hs.makeReservation("199512345678", "R-101", today.plusDays(5), today.plusDays(4), 2);
            results.add(new TestResult("TC19", "Invalid", "Check-out date before check-in date", false, "Should
have been rejected"));
        } catch (IllegalArgumentException e) {
            results.add(new TestResult("TC19", "Invalid", "Check-out date before check-in date", true,
"Correctly rejected: " + e.getMessage()));
        } catch (Exception e) {
            results.add(new TestResult("TC19", "Invalid", "Check-out date before check-in date", false,
e.getMessage()));
        }

        // TC20: Invalid Customer Email format (Must FAIL)
        try {
            new K2703047_Guest("199512345678", "Test Guest", "+94771234567", "invalid-email-address");
            results.add(new TestResult("TC20", "Invalid", "Guest registration with malformed email", false,
"Should have failed email regex"));
        } catch (IllegalArgumentException e) {
            results.add(new TestResult("TC20", "Invalid", "Guest registration with malformed email", true,
"Regex caught invalid email format"));
        } catch (Exception e) {
            results.add(new TestResult("TC20", "Invalid", "Guest registration with malformed email", false,
e.getMessage()));
        }

        // TC21: Invalid Customer Phone format (Must FAIL)
        try {
            new K2703047_Guest("199512345678", "Test Guest", "123", "test@domain.com");
            results.add(new TestResult("TC21", "Invalid", "Guest registration with malformed phone (<9 digits)",
false, "Should have failed phone validation"));
        } catch (IllegalArgumentException e) {
```

```
            results.add(new TestResult("TC21", "Invalid", "Guest registration with malformed phone (<9 digits)",
true, "Phone validation caught invalid length"));
        } catch (Exception e) {
            results.add(new TestResult("TC21", "Invalid", "Guest registration with malformed phone (<9 digits)",
false, e.getMessage()));
        }

        // TC22: Removing room with active reservations (Must FAIL)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            hs.makeReservation("199512345678", "R-102", today.plusDays(4), today.plusDays(7), 2);
            hs.removeRoom("R-102"); // Should fail because active reservation exists
            results.add(new TestResult("TC22", "Invalid", "Remove room with active reservation", false, "Should
have thrown IllegalStateException"));
        } catch (IllegalStateException e) {
            results.add(new TestResult("TC22", "Invalid", "Remove room with active reservation", true,
"Safeguard triggered: " + e.getMessage()));
        } catch (Exception e) {
            results.add(new TestResult("TC22", "Invalid", "Remove room with active reservation", false,
e.getMessage()));
        }

        // Print Test Results Summary Table
        System.out.println(String.format("%-6s | %-9s | %-45s | %-8s | %s", "ID", "Category", "Description",
"Result", "Notes/Output"));
        System.out.println("-".repeat(110));
        for (TestResult tr : results) {
            if (tr.passed) passedCount++;
            String status = tr.passed ? "[PASS]" : "[FAIL]";
            System.out.println(String.format("%-6s | %-9s | %-45s | %-8s | %s",
                    tr.testId, tr.category, tr.description, status, tr.details));
        }
        System.out.println("=".repeat(110));
        System.out.println(String.format("TOTAL TESTS: %d | PASSED: %d | FAILED: %d | SUCCESS RATE: %.1f%%",
                results.size(), passedCount, (results.size() - passedCount), (passedCount * 100.0 /
results.size())));

System.out.println("==============================================================================================\n
");

        return results;
    }
}
```

```
package k2703047.smartstay.test;

import k2703047.smartstay.model.K2703047_BedType;
import k2703047.smartstay.model.K2703047_FamilySuiteRoom;
import k2703047.smartstay.model.K2703047_Guest;
import k2703047.smartstay.model.K2703047_Invoice;
import k2703047.smartstay.model.K2703047_Reservation;
import k2703047.smartstay.model.K2703047_ReservationStatus;
import k2703047.smartstay.model.K2703047_Room;
import k2703047.smartstay.model.K2703047_RoomFactory;
import k2703047.smartstay.model.K2703047_RoomStatus;
import k2703047.smartstay.model.K2703047_RoomType;
import k2703047.smartstay.service.K2703047_HotelSystem;

import java.time.LocalDate;
import java.time.LocalDateTime;
```

```
import java.util.ArrayList;
import java.util.List;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Comprehensive Automated Test Suite validating Normal, Boundary, and Invalid Test Cases.
 * Aligned with Module Learning Outcomes (LO2, LO5, LO6).
 */
public class K2703047_TestSuite {

    public static class TestResult {
        public final String testId;
        public final String category;
        public final String description;
        public final boolean passed;
        public final String details;

        public TestResult(String testId, String category, String description, boolean passed, String details) {
            this.testId = testId;
            this.category = category;
            this.description = description;
            this.passed = passed;
            this.details = details;
        }
    }

    public static void main(String[] args) {
        runAllTests();
    }

    public static List<TestResult> runAllTests() {
        List<TestResult> results = new ArrayList<>();
        int passedCount = 0;


System.out.println("============================================================================================")
;
        System.out.println("            SMARTSTAY HOTEL BOOKING SYSTEM - COMPREHENSIVE TEST SUITE
");
        System.out.println("       Kingston University CI6115 | Student ID: 2703047 | Module Learning Outcome
LO5     ");

System.out.println("============================================================================================")
;

        // ========================================
        // SECTION 1: NORMAL TEST CASES
        // ========================================
        // TC01: Customer Registration
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            boolean ok = hs.registerGuest("987654321V", "Amara Silva", "+94712345678", "amara.silva@gmail.com");
            results.add(new TestResult("TC01", "Normal", "Guest registration with valid details", ok, "Guest
stored and retrievable"));
        } catch (Exception e) {
            results.add(new TestResult("TC01", "Normal", "Guest registration with valid details", false,
e.getMessage()));
        }
```

```
        // TC02: Make Reservation for Standard Room
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            K2703047_Reservation res = hs.makeReservation("199512345678", "R-101", today.plusDays(3),
today.plusDays(6), 2);
            boolean ok = res != null && res.getBookingId().startsWith("BK-") && res.getNumberOfNights() == 3;
            results.add(new TestResult("TC02", "Normal", "Create valid 3-night reservation", ok, "Booking
created with ID " + res.getBookingId()));
        } catch (Exception e) {
            results.add(new TestResult("TC02", "Normal", "Create valid 3-night reservation", false,
e.getMessage()));
        }

        // TC03: 5-Step Billing Calculation (Deluxe Room, 3 nights, no discount)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            // Room D-202 (Deluxe, 9000/night, capacity 2, tax 12%)
            K2703047_Reservation res = hs.makeReservation("199512345678", "D-202", today.plusDays(4),
today.plusDays(7), 2);
            K2703047_Invoice inv = hs.generateInvoice(res.getBookingId());

            // Expected calculations:
            // Base: 9,000 * 3 = 27,000
            // Extra: 0
            // Discount: 0 (< 5 nights)
            // Net: 27,000
            // Tax: 12% of 27,000 = 3,240
            // Total Gross: 30,240
            // Advance: 10,000
            // Final Payable: 20,240
            boolean ok = Math.abs(inv.getBaseRoomCharge() - 27000.0) < 0.01 &&
                         Math.abs(inv.getTaxAmount() - 3240.0) < 0.01 &&
                         Math.abs(inv.getFinalPayableAmount() - 20240.0) < 0.01;
            results.add(new TestResult("TC03", "Normal", "5-Step billing calculation for Deluxe Room", ok,
                    String.format("Base: LKR %,.2f | Tax: LKR %,.2f | Final: LKR %,.2f",
                            inv.getBaseRoomCharge(), inv.getTaxAmount(), inv.getFinalPayableAmount())));
        } catch (Exception e) {
            results.add(new TestResult("TC03", "Normal", "5-Step billing calculation for Deluxe Room", false,
e.getMessage()));
        }

        // TC04: Long-Stay Discount (>= 5 nights => 12% discount)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            // Standard Room R-102 (6000/night, tax 10%), 6 nights
            K2703047_Reservation res = hs.makeReservation("199512345678", "R-102", today.plusDays(3),
today.plusDays(9), 2);
            K2703047_Invoice inv = hs.generateInvoice(res.getBookingId());

            // Base = 6,000 * 6 = 36,000
            // Discount = 12% of 36,000 = 4,320
            // Net = 36,000 - 4,320 = 31,680
            // Tax = 10% of 31,680 = 3,168
            // Total = 34,848 - 10,000 = 24,848
            boolean ok = inv.isDiscountApplied() &&
                         Math.abs(inv.getDiscountAmount() - 4320.0) < 0.01 &&
                         Math.abs(inv.getFinalPayableAmount() - 24848.0) < 0.01;
            results.add(new TestResult("TC04", "Normal", "12% long-stay discount applied for 6 nights", ok,
                    String.format("Base: 36,000 | Discount: LKR %,.2f | Payable: LKR %,.2f",
```

```
inv.getDiscountAmount(), inv.getFinalPayableAmount())));
        } catch (Exception e) {
            results.add(new TestResult("TC04", "Normal", "12% long-stay discount applied for 6 nights", false,
e.getMessage()));
        }

        // TC05: Extra Guest Charge Calculation
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            // Suite Room S-301 (14,000/night, capacity 3, extra guest 2,000/night, tax 15%), 2 nights, 5 guests
(2 extra)
            K2703047_Reservation res = hs.makeReservation("199512345678", "S-301", today.plusDays(3),
today.plusDays(5), 5);
            K2703047_Invoice inv = hs.generateInvoice(res.getBookingId());

            // Base = 14,000 * 2 = 28,000
            // Extra = 2 extra guests * 2,000 * 2 nights = 8,000
            // Net = 28,000 + 8,000 = 36,000
            // Tax = 15% of 36,000 = 5,400
            // Total = 41,400 - 10,000 = 31,400
            boolean ok = inv.getExtraGuestCount() == 2 &&
                         Math.abs(inv.getExtraGuestCharges() - 8000.0) < 0.01 &&
                         Math.abs(inv.getFinalPayableAmount() - 31400.0) < 0.01;
            results.add(new TestResult("TC05", "Normal", "Extra guest charge calculation (2 extra guests)", ok,
                    String.format("Extra guests: %d | Extra charge: LKR %,.2f", inv.getExtraGuestCount(),
inv.getExtraGuestCharges())));
        } catch (Exception e) {
            results.add(new TestResult("TC05", "Normal", "Extra guest charge calculation (2 extra guests)",
false, e.getMessage()));
        }

        // TC06: Search Available Rooms by Room Type and Dates
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            List<K2703047_Room> suites = hs.searchAvailableRooms(K2703047_RoomType.SUITE, today.plusDays(3),
today.plusDays(5));
            boolean ok = !suites.isEmpty() && suites.get(0).getRoomType() == K2703047_RoomType.SUITE;
            results.add(new TestResult("TC06", "Normal", "Search available rooms by type and date", ok, "Found "
+ suites.size() + " suite(s)"));
        } catch (Exception e) {
            results.add(new TestResult("TC06", "Normal", "Search available rooms by type and date", false,
e.getMessage()));
        }

        // TC07: TreeMap Red-Black Tree Date Index Retrieval
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            LocalDate checkIn = today.plusDays(10);
            hs.makeReservation("199512345678", "R-101", checkIn, checkIn.plusDays(2), 2);
            List<K2703047_Reservation> onDate = hs.getReservationsByCheckInDate(checkIn);
            boolean ok = !onDate.isEmpty() && onDate.get(0).getCheckInDate().equals(checkIn);
            results.add(new TestResult("TC07", "Normal", "View reservations by check-in date (TreeMap)", ok,
"Indexed via TreeMap in O(log N)"));
        } catch (Exception e) {
            results.add(new TestResult("TC07", "Normal", "View reservations by check-in date (TreeMap)", false,
e.getMessage()));
        }

        // TC08: Binary Search Algorithm by Booking ID
```

```
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            K2703047_Reservation res = hs.makeReservation("199512345678", "FS-401", today.plusDays(7),
today.plusDays(10), 4);
            K2703047_Reservation found = hs.searchReservationBinary(res.getBookingId());
            boolean ok = found != null && found.getBookingId().equals(res.getBookingId());
            results.add(new TestResult("TC08", "Normal", "Binary Search on sorted reservations by ID", ok,
"Located " + res.getBookingId() + " in O(log N) time"));
        } catch (Exception e) {
            results.add(new TestResult("TC08", "Normal", "Binary Search on sorted reservations by ID", false,
e.getMessage()));
        }

        // ========================================
        // SECTION 2: BOUNDARY TEST CASES
        // ========================================
        // TC09: Advance Booking Boundary - Exactly 2 days in advance (Must SUCCEED)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            K2703047_Reservation res = hs.makeReservation("199512345678", "R-101", today.plusDays(2),
today.plusDays(4), 2);
            boolean ok = res != null;
            results.add(new TestResult("TC09", "Boundary", "Advance booking boundary: exactly 2 days ahead", ok,
"Accepted (boundary = 2 days)"));
        } catch (Exception e) {
            results.add(new TestResult("TC09", "Boundary", "Advance booking boundary: exactly 2 days ahead",
false, e.getMessage()));
        }

        // TC10: Advance Booking Boundary - 1 day in advance (Must FAIL)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            hs.makeReservation("199512345678", "R-101", today.plusDays(1), today.plusDays(3), 2);
            results.add(new TestResult("TC10", "Boundary", "Advance booking boundary: 1 day ahead
(insufficient)", false, "Should have failed"));
        } catch (IllegalArgumentException e) {
            results.add(new TestResult("TC10", "Boundary", "Advance booking boundary: 1 day ahead
(insufficient)", true, "Correctly rejected: " + e.getMessage()));
        } catch (Exception e) {
            results.add(new TestResult("TC10", "Boundary", "Advance booking boundary: 1 day ahead
(insufficient)", false, e.getMessage()));
        }

        // TC11: Discount Boundary - Exactly 5 nights stay (12% discount MUST apply)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            K2703047_Reservation res = hs.makeReservation("199512345678", "R-102", today.plusDays(3),
today.plusDays(8), 2); // 5 nights
            K2703047_Invoice inv = hs.generateInvoice(res.getBookingId());
            boolean ok = inv.getNumberOfNights() == 5 && inv.isDiscountApplied() && inv.getDiscountRate() ==
12.0;
            results.add(new TestResult("TC11", "Boundary", "Discount boundary: exactly 5 nights stay", ok, "12%
discount correctly triggered"));
        } catch (Exception e) {
            results.add(new TestResult("TC11", "Boundary", "Discount boundary: exactly 5 nights stay", false,
e.getMessage()));
        }
```

```
        // TC12: Discount Boundary - 4 nights stay (12% discount MUST NOT apply)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            K2703047_Reservation res = hs.makeReservation("199512345678", "R-102", today.plusDays(3),
today.plusDays(7), 2); // 4 nights
            K2703047_Invoice inv = hs.generateInvoice(res.getBookingId());
            boolean ok = inv.getNumberOfNights() == 4 && !inv.isDiscountApplied() && inv.getDiscountAmount() ==
0.0;
            results.add(new TestResult("TC12", "Boundary", "Discount boundary: 4 nights stay (no discount)", ok,
"0% discount correctly assigned"));
        } catch (Exception e) {
            results.add(new TestResult("TC12", "Boundary", "Discount boundary: 4 nights stay (no discount)",
false, e.getMessage()));
        }

        // TC13: Cancellation Boundary - Exactly 25 hours before check-in date (Must SUCCEED)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate checkIn = LocalDate.now().plusDays(5);
            K2703047_Reservation res = hs.makeReservation("199512345678", "R-102", checkIn, checkIn.plusDays(2),
2);

            // Simulation: Cancellation requested 25 hours before check-in date start
            LocalDateTime simTime = checkIn.atStartOfDay().minusHours(25);
            boolean cancelled = hs.cancelReservationWithCustomTime(res.getBookingId(), simTime);
            boolean ok = cancelled && res.getStatus() == K2703047_ReservationStatus.CANCELLED &&
res.isAdvanceRefunded();
            results.add(new TestResult("TC13", "Boundary", "Cancellation boundary: 25 hours before check-in",
ok, "Cancellation accepted and advance refunded"));
        } catch (Exception e) {
            results.add(new TestResult("TC13", "Boundary", "Cancellation boundary: 25 hours before check-in",
false, e.getMessage()));
        }

        // TC14: Cancellation Boundary - 23 hours before check-in date (Must FAIL)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate checkIn = LocalDate.now().plusDays(5);
            K2703047_Reservation res = hs.makeReservation("199512345678", "R-102", checkIn, checkIn.plusDays(2),
2);

            // Simulation: Cancellation requested 23 hours before check-in date start (within 24 hours!)
            LocalDateTime simTime = checkIn.atStartOfDay().minusHours(23);
            boolean cancelled = hs.cancelReservationWithCustomTime(res.getBookingId(), simTime);
            boolean ok = !cancelled && res.getStatus() == K2703047_ReservationStatus.CONFIRMED;
            results.add(new TestResult("TC14", "Boundary", "Cancellation boundary: 23 hours before check-in
(within 24h)", ok, "Cancellation correctly rejected"));
        } catch (Exception e) {
            results.add(new TestResult("TC14", "Boundary", "Cancellation boundary: 23 hours before check-in
(within 24h)", false, e.getMessage()));
        }

        // TC15: Capacity Boundary - Guest count equal to base capacity (0 extra guest charge)
        try {
            K2703047_Room room = K2703047_RoomFactory.createRoom("R-TEST", K2703047_RoomType.STANDARD,
K2703047_BedType.DOUBLE);
            double extra = room.calculateExtraGuestCharge(room.getBaseCapacity(), 3);
            boolean ok = (extra == 0.0);
            results.add(new TestResult("TC15", "Boundary", "Capacity boundary: guests = capacity (2 guests in
Standard)", ok, "Extra charge = LKR 0.00"));
        } catch (Exception e) {
```

```
            results.add(new TestResult("TC15", "Boundary", "Capacity boundary: guests = capacity", false,
e.getMessage()));
        }

        // TC16: Capacity Boundary - Guest count = base capacity + 1 (Exactly 1 extra guest charged)
        try {
            K2703047_Room room = K2703047_RoomFactory.createRoom("R-TEST", K2703047_RoomType.STANDARD,
K2703047_BedType.DOUBLE);
            // capacity is 2, 3 guests for 2 nights => 1 extra guest * 1000 * 2 = 2000
            double extra = room.calculateExtraGuestCharge(room.getBaseCapacity() + 1, 2);
            boolean ok = (extra == 2000.0);
            results.add(new TestResult("TC16", "Boundary", "Capacity boundary: guests = capacity + 1 (3 guests,
2 nights)", ok, "Extra charge = LKR 2,000.00"));
        } catch (Exception e) {
            results.add(new TestResult("TC16", "Boundary", "Capacity boundary: guests = capacity + 1", false,
e.getMessage()));
        }

        // ========================================
        // SECTION 3: INVALID / ERROR TEST CASES
        // ========================================
        // TC17: Booking room under maintenance (Must FAIL)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            // Room R-103 is seeded as UNDER_MAINTENANCE
            hs.makeReservation("199512345678", "R-103", today.plusDays(3), today.plusDays(5), 2);
            results.add(new TestResult("TC17", "Invalid", "Book room with status UNDER_MAINTENANCE", false,
"Should have been rejected"));
        } catch (IllegalStateException e) {
            results.add(new TestResult("TC17", "Invalid", "Book room with status UNDER_MAINTENANCE", true,
"Correctly rejected: " + e.getMessage()));
        } catch (Exception e) {
            results.add(new TestResult("TC17", "Invalid", "Book room with status UNDER_MAINTENANCE", false,
e.getMessage()));
        }

        // TC18: Double booking on overlapping dates (Must FAIL via Conflict Algorithm)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            LocalDate in1 = today.plusDays(5);
            LocalDate out1 = in1.plusDays(4); // Day 5 to Day 9
            hs.makeReservation("199512345678", "R-101", in1, out1, 2);

            // Attempt overlapping booking: Day 6 to Day 8
            hs.makeReservation("N87654321", "R-101", in1.plusDays(1), in1.plusDays(3), 2);
            results.add(new TestResult("TC18", "Invalid", "Double-booking overlapping date range", false,
"Should have rejected overlapping interval"));
        } catch (IllegalStateException e) {
            results.add(new TestResult("TC18", "Invalid", "Double-booking overlapping date range", true,
"Conflict detected and prevented: " + e.getMessage()));
        } catch (Exception e) {
            results.add(new TestResult("TC18", "Invalid", "Double-booking overlapping date range", false,
e.getMessage()));
        }

        // TC19: Check-out date before or equal to check-in date (Must FAIL)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            hs.makeReservation("199512345678", "R-101", today.plusDays(5), today.plusDays(4), 2);
```

```
            results.add(new TestResult("TC19", "Invalid", "Check-out date before check-in date", false, "Should
have been rejected"));
        } catch (IllegalArgumentException e) {
            results.add(new TestResult("TC19", "Invalid", "Check-out date before check-in date", true,
"Correctly rejected: " + e.getMessage()));
        } catch (Exception e) {
            results.add(new TestResult("TC19", "Invalid", "Check-out date before check-in date", false,
e.getMessage()));
        }

        // TC20: Invalid Customer Email format (Must FAIL)
        try {
            new K2703047_Guest("199512345678", "Test Guest", "+94771234567", "invalid-email-address");
            results.add(new TestResult("TC20", "Invalid", "Guest registration with malformed email", false,
"Should have failed email regex"));
        } catch (IllegalArgumentException e) {
            results.add(new TestResult("TC20", "Invalid", "Guest registration with malformed email", true,
"Regex caught invalid email format"));
        } catch (Exception e) {
            results.add(new TestResult("TC20", "Invalid", "Guest registration with malformed email", false,
e.getMessage()));
        }

        // TC21: Invalid Customer Phone format (Must FAIL)
        try {
            new K2703047_Guest("199512345678", "Test Guest", "123", "test@domain.com");
            results.add(new TestResult("TC21", "Invalid", "Guest registration with malformed phone (<9 digits)",
false, "Should have failed phone validation"));
        } catch (IllegalArgumentException e) {
            results.add(new TestResult("TC21", "Invalid", "Guest registration with malformed phone (<9 digits)",
true, "Phone validation caught invalid length"));
        } catch (Exception e) {
            results.add(new TestResult("TC21", "Invalid", "Guest registration with malformed phone (<9 digits)",
false, e.getMessage()));
        }

        // TC22: Removing room with active reservations (Must FAIL)
        try {
            K2703047_HotelSystem hs = new K2703047_HotelSystem();
            LocalDate today = LocalDate.now();
            hs.makeReservation("199512345678", "R-102", today.plusDays(4), today.plusDays(7), 2);
            hs.removeRoom("R-102"); // Should fail because active reservation exists
            results.add(new TestResult("TC22", "Invalid", "Remove room with active reservation", false, "Should
have thrown IllegalStateException"));
        } catch (IllegalStateException e) {
            results.add(new TestResult("TC22", "Invalid", "Remove room with active reservation", true,
"Safeguard triggered: " + e.getMessage()));
        } catch (Exception e) {
            results.add(new TestResult("TC22", "Invalid", "Remove room with active reservation", false,
e.getMessage()));
        }

        // Print Test Results Summary Table
        System.out.println(String.format("%-6s | %-9s | %-45s | %-8s | %s", "ID", "Category", "Description",
"Result", "Notes/Output"));
        System.out.println("-".repeat(110));
        for (TestResult tr : results) {
            if (tr.passed) passedCount++;
            String status = tr.passed ? "[PASS]" : "[FAIL]";
            System.out.println(String.format("%-6s | %-9s | %-45s | %-8s | %s",
                    tr.testId, tr.category, tr.description, status, tr.details));
        }
```

```
        System.out.println("=".repeat(110));
        System.out.println(String.format("TOTAL TESTS: %d | PASSED: %d | FAILED: %d | SUCCESS RATE: %.1f%%",
                results.size(), passedCount, (results.size() - passedCount), (passedCount * 100.0 /
results.size())));

System.out.println("==========================================================================================\n
");

        return results;
    }
}
```

```
package k2703047.smartstay.ui;

import k2703047.smartstay.model.K2703047_BedType;
import k2703047.smartstay.model.K2703047_Guest;
import k2703047.smartstay.model.K2703047_Invoice;
import k2703047.smartstay.model.K2703047_Reservation;
import k2703047.smartstay.model.K2703047_Room;
import k2703047.smartstay.model.K2703047_RoomFactory;
import k2703047.smartstay.model.K2703047_RoomStatus;
import k2703047.smartstay.model.K2703047_RoomType;
import k2703047.smartstay.service.K2703047_HotelSystem;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.util.List;
import java.util.Scanner;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Interactive Command Line Interface (CLI) for SmartStay Hotel Booking System.
 * Usability features:
 * - Colorized ANSI output headers and status messages
 * - Robust input validation and error feedback
 * - Clear table formats for rooms, guests, reservations, and invoices
 */
public class K2703047_CLIView {

    private final K2703047_HotelSystem hotelSystem;
    private final Scanner scanner;
    private final DateTimeFormatter dateFormatter = DateTimeFormatter.ofPattern("yyyy-MM-dd");

    public K2703047_CLIView(K2703047_HotelSystem hotelSystem) {
        this.hotelSystem = hotelSystem;
        this.scanner = new Scanner(System.in);
    }

    public void start() {
        boolean running = true;
        while (running) {
            printHeader("SMARTSTAY HOTEL BOOKING SYSTEM - MAIN PORTAL");
            System.out.println(" 1. Administrator Portal (Manage Rooms & Status)");
            System.out.println(" 2. Guest Portal (Register, Search, Book, Cancel, Invoices)");
            System.out.println(" 3. View All Reservations & Invoices");
            System.out.println(" 4. Launch Desktop GUI Interface");
```

```
        System.out.println(" 5. Exit System");
        System.out.print("\n Select an option [1-5]: ");

        String choice = scanner.nextLine().trim();
        switch (choice) {
            case "1":
                handleAdminMenu();
                break;
            case "2":
                handleGuestMenu();
                break;
            case "3":
                handleViewReservationsAndInvoices();
                break;
            case "4":
                launchGui();
                break;
            case "5":
                System.out.println("\nExiting SmartStay Hotel Booking System. Goodbye!");
                running = false;
                break;
            default:
                System.out.println("Invalid selection. Please choose an option from 1 to 5.");
        }
    }
}

// ========================================
// ADMINISTRATOR SUB-MENU
// ========================================
private void handleAdminMenu() {
    boolean back = false;
    while (!back) {
        printHeader("ADMINISTRATOR PORTAL");
        System.out.println(" 1. View All Room Records");
        System.out.println(" 2. Add New Room Record");
        System.out.println(" 3. Update Room Record");
        System.out.println(" 4. Manage Room Availability Status");
        System.out.println(" 5. Remove Room Record");
        System.out.println(" 6. Return to Main Menu");
        System.out.print("\n Select admin operation [1-6]: ");

        String choice = scanner.nextLine().trim();
        switch (choice) {
            case "1":
                displayAllRooms();
                break;
            case "2":
                handleAddNewRoom();
                break;
            case "3":
                handleUpdateRoom();
                break;
            case "4":
                handleManageRoomStatus();
                break;
            case "5":
                handleRemoveRoom();
                break;
            case "6":
                back = true;
                break;
```

```
                default:
                    System.out.println("Invalid selection. Please try again.");
            }
        }
    }

    private void displayAllRooms() {
        printHeader("ALL ROOM RECORDS");
        List<K2703047_Room> rooms = hotelSystem.getAllRooms();
        if (rooms.isEmpty()) {
            System.out.println("No rooms registered in the system.");
            return;
        }
        printRoomTable(rooms);
    }

    private void handleAddNewRoom() {
        printHeader("ADD NEW ROOM RECORD");
        try {
            System.out.print("Enter Room ID (e.g. R-105, D-205, S-305): ");
            String roomId = scanner.nextLine().trim().toUpperCase();
            if (hotelSystem.getRoomById(roomId) != null) {
                System.out.println("[Error] A room with ID " + roomId + " already exists.");
                return;
            }

            System.out.println("Select Room Type:");
            System.out.println("1. Standard (LKR 6,000/night, No Breakfast, Extra: LKR 1,000, Tax: 10%)");
            System.out.println("2. Deluxe   (LKR 9,000/night, Breakfast Yes, Extra: LKR 1,500, Tax: 12%)");
            System.out.println("3. Suite    (LKR 14,000/night, Breakfast Yes, Extra: LKR 2,000, Tax: 15%)");
            System.out.println("4. Family Suite (LKR 18,000/night, Breakfast Yes, Extra: LKR 2,500, Tax: 15%)");
            System.out.print("Choice [1-4]: ");
            String tChoice = scanner.nextLine().trim();
            K2703047_RoomType type;
            switch (tChoice) {
                case "1": type = K2703047_RoomType.STANDARD; break;
                case "2": type = K2703047_RoomType.DELUXE; break;
                case "3": type = K2703047_RoomType.SUITE; break;
                case "4": type = K2703047_RoomType.FAMILY_SUITE; break;
                default:
                    System.out.println("[Error] Invalid room type selected.");
                    return;
            }

            System.out.println("Select Bed Type:");
            System.out.println("1. Single Bed\n2. Double Bed\n3. Queen Bed\n4. King Bed");
            System.out.print("Choice [1-4]: ");
            String bChoice = scanner.nextLine().trim();
            K2703047_BedType bedType;
            switch (bChoice) {
                case "1": bedType = K2703047_BedType.SINGLE; break;
                case "2": bedType = K2703047_BedType.DOUBLE; break;
                case "3": bedType = K2703047_BedType.QUEEN; break;
                case "4": bedType = K2703047_BedType.KING; break;
                default:
                    System.out.println("[Error] Invalid bed type selected.");
                    return;
            }

            System.out.print("Use default daily rate of LKR " + String.format("%,.0f", type.getDailyRate()) + "?
(Y/N): ");
            String useDef = scanner.nextLine().trim().toUpperCase();
```

```
            double rate = type.getDailyRate();
            if (useDef.equals("N")) {
                System.out.print("Enter custom daily rate (LKR): ");
                rate = Double.parseDouble(scanner.nextLine().trim());
            }

            K2703047_Room newRoom = K2703047_RoomFactory.createRoom(
                    roomId, type, bedType, rate, K2703047_RoomStatus.AVAILABLE, type.getStandardCapacity());
            boolean added = hotelSystem.addRoom(newRoom);
            if (added) {
                System.out.println("[Success] Room " + roomId + " successfully added to the system!");
            } else {
                System.out.println("[Error] Failed to add room.");
            }
        } catch (Exception e) {
            System.out.println("[Error] " + e.getMessage());
        }
    }

    private void handleUpdateRoom() {
        printHeader("UPDATE ROOM RECORD");
        displayAllRooms();
        System.out.print("Enter Room ID to update: ");
        String roomId = scanner.nextLine().trim().toUpperCase();
        K2703047_Room existing = hotelSystem.getRoomById(roomId);
        if (existing == null) {
            System.out.println("[Error] Room not found.");
            return;
        }

        try {
            System.out.print("Enter new Daily Rate (Current: LKR " + existing.getDailyRate() + ") or press Enter
to skip: ");
            String rInput = scanner.nextLine().trim();
            if (!rInput.isEmpty()) {
                existing.setDailyRate(Double.parseDouble(rInput));
            }

            System.out.println("Select new Bed Type (Current: " + existing.getBedType() + "):");
            System.out.println("1. Single Bed | 2. Double Bed | 3. Queen Bed | 4. King Bed | Enter to skip");
            String bInput = scanner.nextLine().trim();
            if (!bInput.isEmpty()) {
                switch (bInput) {
                    case "1": existing.setBedType(K2703047_BedType.SINGLE); break;
                    case "2": existing.setBedType(K2703047_BedType.DOUBLE); break;
                    case "3": existing.setBedType(K2703047_BedType.QUEEN); break;
                    case "4": existing.setBedType(K2703047_BedType.KING); break;
                }
            }

            System.out.println("[Success] Room " + roomId + " updated successfully!");
        } catch (Exception e) {
            System.out.println("[Error] Could not update room: " + e.getMessage());
        }
    }

    private void handleManageRoomStatus() {
        printHeader("MANAGE ROOM AVAILABILITY STATUS");
        displayAllRooms();
        System.out.print("Enter Room ID to update status: ");
        String roomId = scanner.nextLine().trim().toUpperCase();
        K2703047_Room room = hotelSystem.getRoomById(roomId);
```

```
        if (room == null) {
            System.out.println("[Error] Room not found.");
            return;
        }

        System.out.println("Current Status: " + room.getStatus().getDisplayName());
        System.out.println("Select New Status:");
        System.out.println(" 1. Available");
        System.out.println(" 2. Reserved");
        System.out.println(" 3. Under Maintenance");
        System.out.print("Choice [1-3]: ");
        String choice = scanner.nextLine().trim();

        K2703047_RoomStatus newStatus = null;
        switch (choice) {
            case "1": newStatus = K2703047_RoomStatus.AVAILABLE; break;
            case "2": newStatus = K2703047_RoomStatus.RESERVED; break;
            case "3": newStatus = K2703047_RoomStatus.UNDER_MAINTENANCE; break;
            default:
                System.out.println("[Error] Invalid status option.");
                return;
        }

        hotelSystem.updateRoomStatus(roomId, newStatus);
        System.out.println("[Success] Room " + roomId + " status updated to " + newStatus.getDisplayName() +
"!");
    }

    private void handleRemoveRoom() {
        printHeader("REMOVE ROOM RECORD");
        displayAllRooms();
        System.out.print("Enter Room ID to remove: ");
        String roomId = scanner.nextLine().trim().toUpperCase();
        try {
            boolean removed = hotelSystem.removeRoom(roomId);
            if (removed) {
                System.out.println("[Success] Room " + roomId + " removed successfully!");
            } else {
                System.out.println("[Error] Room not found.");
            }
        } catch (IllegalStateException e) {
            System.out.println("[Error] " + e.getMessage());
        }
    }

    // ==========================================
    // GUEST SUB-MENU
    // ==========================================
    private void handleGuestMenu() {
        boolean back = false;
        while (!back) {
            printHeader("GUEST / CUSTOMER PORTAL");
            System.out.println(" 1. Register as Customer");
            System.out.println(" 2. Search Available Rooms by Room Type and Date");
            System.out.println(" 3. Make Room Reservation");
            System.out.println(" 4. Search Reservations by Guest Name or Booking ID");
            System.out.println(" 5. View Reservations by Check-In Date");
            System.out.println(" 6. Cancel Eligible Reservation");
            System.out.println(" 7. Return to Main Menu");
            System.out.print("\n Select guest operation [1-7]: ");

            String choice = scanner.nextLine().trim();
```

```
        switch (choice) {
            case "1":
                handleRegisterGuest();
                break;
            case "2":
                handleSearchRooms();
                break;
            case "3":
                handleMakeReservation();
                break;
            case "4":
                handleSearchReservations();
                break;
            case "5":
                handleViewByCheckInDate();
                break;
            case "6":
                handleCancelReservation();
                break;
            case "7":
                back = true;
                break;
            default:
                System.out.println("Invalid selection. Please try again.");
        }
    }
}

private void handleRegisterGuest() {
    printHeader("CUSTOMER REGISTRATION");
    try {
        System.out.print("Enter NIC or Passport Number: ");
        String nic = scanner.nextLine().trim().toUpperCase();
        if (hotelSystem.getGuest(nic) != null) {
            System.out.println("[Info] Customer is already registered: " + hotelSystem.getGuest(nic));
            return;
        }

        System.out.print("Enter Full Name: ");
        String name = scanner.nextLine().trim();

        System.out.print("Enter Contact Number (e.g. +94771234567): ");
        String phone = scanner.nextLine().trim();

        System.out.print("Enter Email Address: ");
        String email = scanner.nextLine().trim();

        boolean registered = hotelSystem.registerGuest(nic, name, phone, email);
        if (registered) {
            System.out.println("[Success] Customer registered successfully!");
        }
    } catch (Exception e) {
        System.out.println("[Error] Registration failed: " + e.getMessage());
    }
}

private void handleSearchRooms() {
    printHeader("SEARCH AVAILABLE ROOMS");
    try {
        System.out.println("Filter by Room Type:");
        System.out.println("0. All Types");
        System.out.println("1. Standard | 2. Deluxe | 3. Suite | 4. Family Suite");
```

```
            System.out.print("Choice [0-4]: ");
            String tc = scanner.nextLine().trim();
            K2703047_RoomType type = null;
            if (tc.equals("1")) type = K2703047_RoomType.STANDARD;
            else if (tc.equals("2")) type = K2703047_RoomType.DELUXE;
            else if (tc.equals("3")) type = K2703047_RoomType.SUITE;
            else if (tc.equals("4")) type = K2703047_RoomType.FAMILY_SUITE;

            LocalDate today = LocalDate.now();
            System.out.printf("Enter Check-In Date (YYYY-MM-DD, minimum %s): ", today.plusDays(2));
            String inStr = scanner.nextLine().trim();
            LocalDate checkIn = LocalDate.parse(inStr, dateFormatter);

            System.out.print("Enter Check-Out Date (YYYY-MM-DD): ");
            String outStr = scanner.nextLine().trim();
            LocalDate checkOut = LocalDate.parse(outStr, dateFormatter);

            List<K2703047_Room> available = hotelSystem.searchAvailableRooms(type, checkIn, checkOut);
            if (available.isEmpty()) {
                System.out.println("\n[Notice] No rooms are available for the selected dates and criteria.");
            } else {
                System.out.println("\n[Success] Found " + available.size() + " available room(s):");
                printRoomTable(available);
            }
        } catch (DateTimeParseException e) {
            System.out.println("[Error] Invalid date format. Please use YYYY-MM-DD.");
        } catch (Exception e) {
            System.out.println("[Error] " + e.getMessage());
        }
    }

    private void handleMakeReservation() {
        printHeader("MAKE ROOM RESERVATION");
        try {
            System.out.print("Enter Guest NIC/Passport: ");
            String nic = scanner.nextLine().trim().toUpperCase();
            K2703047_Guest guest = hotelSystem.getGuest(nic);
            if (guest == null) {
                System.out.println("[Error] Customer not registered. Please register first (Option 1).");
                return;
            }
            System.out.println("Guest: " + guest.getFullName() + " (" + guest.getEmailAddress() + ")");

            LocalDate today = LocalDate.now();
            System.out.printf("Enter Check-In Date (YYYY-MM-DD, at least 2 days ahead, min: %s): ",
today.plusDays(2));
            LocalDate checkIn = LocalDate.parse(scanner.nextLine().trim(), dateFormatter);

            System.out.print("Enter Check-Out Date (YYYY-MM-DD): ");
            LocalDate checkOut = LocalDate.parse(scanner.nextLine().trim(), dateFormatter);

            System.out.print("Enter Number of Guests: ");
            int guests = Integer.parseInt(scanner.nextLine().trim());

            // Show available rooms for these dates
            List<K2703047_Room> available = hotelSystem.searchAvailableRooms(null, checkIn, checkOut);
            if (available.isEmpty()) {
                System.out.println("[Error] No rooms are available for the selected dates.");
                return;
            }
            System.out.println("\nAvailable Rooms for selected dates:");
            printRoomTable(available);
```

```
            System.out.print("Enter Room ID to reserve: ");
            String roomId = scanner.nextLine().trim().toUpperCase();

            // Confirmation of LKR 10,000 advance
            System.out.println("\n[Business Policy] A refundable booking advance of LKR 10,000 is charged
now.");
            System.out.print("Confirm reservation and charge advance? (Y/N): ");
            String confirm = scanner.nextLine().trim().toUpperCase();
            if (!confirm.equals("Y")) {
                System.out.println("Reservation cancelled by user.");
                return;
            }

            K2703047_Reservation reservation = hotelSystem.makeReservation(nic, roomId, checkIn, checkOut,
guests);
            System.out.println("\n[SUCCESS] Reservation Confirmed!");
            System.out.println("Booking ID: " + reservation.getBookingId());

            // Generate invoice immediately
            K2703047_Invoice invoice = hotelSystem.generateInvoice(reservation.getBookingId());
            System.out.println(invoice.generateInvoiceText());

            // Auto-export invoice
            String invoicePath = "invoices/" + reservation.getBookingId() + ".txt";
            invoice.exportToFile(invoicePath);
            System.out.println("[Notice] Invoice automatically saved to: " + invoicePath);

        } catch (DateTimeParseException e) {
            System.out.println("[Error] Invalid date format. Please use YYYY-MM-DD.");
        } catch (Exception e) {
            System.out.println("[Error] Reservation failed: " + e.getMessage());
        }
    }

    private void handleSearchReservations() {
        printHeader("SEARCH RESERVATIONS");
        System.out.println(" 1. Search by Booking ID (Demonstrates O(log N) Binary Search)");
        System.out.println(" 2. Search by Guest Name");
        System.out.print("Select search type [1-2]: ");
        String opt = scanner.nextLine().trim();

        if (opt.equals("1")) {
            System.out.print("Enter Booking ID (e.g. BK-1001): ");
            String bid = scanner.nextLine().trim().toUpperCase();
            K2703047_Reservation r = hotelSystem.searchReservationBinary(bid);
            if (r != null) {
                System.out.println("\n[Found via Binary Search Algorithm]:");
                System.out.println(r);
                showInvoiceOption(r.getBookingId());
            } else {
                System.out.println("[Not Found] No reservation exists with Booking ID: " + bid);
            }
        } else if (opt.equals("2")) {
            System.out.print("Enter Guest Name or keyword: ");
            String name = scanner.nextLine().trim();
            List<K2703047_Reservation> results = hotelSystem.searchReservationsByGuestName(name);
            if (results.isEmpty()) {
                System.out.println("No reservations found for guest matching '" + name + "'.");
            } else {
                printReservationTable(results);
                System.out.print("Enter Booking ID to view invoice (or Enter to skip): ");
```

```
                String bid = scanner.nextLine().trim().toUpperCase();
                if (!bid.isEmpty()) {
                    showInvoiceOption(bid);
                }
            }
        }
    }

    private void handleViewByCheckInDate() {
        printHeader("VIEW RESERVATIONS BY CHECK-IN DATE");
        try {
            System.out.print("Enter Check-In Date (YYYY-MM-DD): ");
            LocalDate date = LocalDate.parse(scanner.nextLine().trim(), dateFormatter);
            List<K2703047_Reservation> list = hotelSystem.getReservationsByCheckInDate(date);
            if (list.isEmpty()) {
                System.out.println("No reservations scheduled to check in on " + date);
            } else {
                System.out.println("[Indexed via TreeMap Red-Black Tree] Found " + list.size() + "
reservation(s):");
                printReservationTable(list);
            }
        } catch (Exception e) {
            System.out.println("[Error] " + e.getMessage());
        }
    }

    private void handleCancelReservation() {
        printHeader("CANCEL RESERVATION");
        System.out.print("Enter Booking ID to cancel: ");
        String bid = scanner.nextLine().trim().toUpperCase();
        K2703047_Reservation r = hotelSystem.getReservation(bid);
        if (r == null) {
            System.out.println("[Error] Reservation not found.");
            return;
        }

        System.out.println("Reservation Details:");
        System.out.println(r);
        System.out.println("\n[Business Policy Reminder] Reservations cannot be cancelled within 24 hours of
check-in.");
        System.out.print("Are you sure you want to cancel this booking? (Y/N): ");
        if (!scanner.nextLine().trim().equalsIgnoreCase("Y")) {
            System.out.println("Cancellation aborted.");
            return;
        }

        boolean cancelled = hotelSystem.cancelReservation(bid);
        if (cancelled) {
            System.out.println("\n[Success] Reservation " + bid + " cancelled successfully!");
            System.out.println("[Notice] Refund of LKR 10,000 booking advance processed to " +
r.getGuest().getFullName());
        } else {
            System.out.println("\n[Error] Cancellation Rejected: Reservation cannot be cancelled within 24 hours
of check-in date or is already cancelled.");
        }
    }

    private void handleViewReservationsAndInvoices() {
        printHeader("ALL SYSTEM RESERVATIONS");
        List<K2703047_Reservation> all = hotelSystem.getAllReservations();
        if (all.isEmpty()) {
            System.out.println("No reservations recorded yet.");
```

```
            return;
        }
        printReservationTable(all);

        System.out.print("\nEnter Booking ID to view detailed invoice (or Enter to return): ");
        String bid = scanner.nextLine().trim().toUpperCase();
        if (!bid.isEmpty()) {
            showInvoiceOption(bid);
        }
    }

    private void showInvoiceOption(String bookingId) {
        try {
            K2703047_Invoice invoice = hotelSystem.generateInvoice(bookingId);
            System.out.println(invoice.generateInvoiceText());
            System.out.print("Export invoice to file? (Y/N): ");
            if (scanner.nextLine().trim().equalsIgnoreCase("Y")) {
                String path = "invoices/" + bookingId + ".txt";
                invoice.exportToFile(path);
                System.out.println("Invoice saved to: " + path);
            }
        } catch (Exception e) {
            System.out.println("[Error] " + e.getMessage());
        }
    }

    private void launchGui() {
        System.out.println("\nLaunching SmartStay Desktop GUI...");
        new Thread(() -> {
            try {
                K2703047_GUIView.launch(hotelSystem);
            } catch (Exception e) {
                System.out.println("GUI error: " + e.getMessage());
            }
        }).start();
    }

    // Table printing helper utilities
    private void printRoomTable(List<K2703047_Room> rooms) {
        System.out.println("+----------+---------------+------------+-----------------+------------------+-----
----------+");
        System.out.println("| Room ID  | Room Type     | Bed Type   | Daily Rate(LKR) | Breakfast Included|
Status        |");
        System.out.println("+----------+---------------+------------+-----------------+------------------+-----
----------+");
        for (K2703047_Room r : rooms) {
            System.out.printf("| %-8s | %-13s | %-10s | LKR %11.2f | %-17s | %-13s |\n",
                    r.getRoomId(),
                    r.getRoomType().getDisplayName(),
                    r.getBedType().getDisplayName(),
                    r.getDailyRate(),
                    r.hasComplimentaryBreakfast() ? "Yes" : "No",
                    r.getStatus().getDisplayName());
        }
        System.out.println("+----------+---------------+------------+-----------------+------------------+-----
----------+");
    }

    private void printReservationTable(List<K2703047_Reservation> list) {
        System.out.println("+------------+--------------------+---------+--------------+--------------+--------
+-------------+");
        System.out.println("| Booking ID | Guest Name         | Room ID | Check-In     | Check-Out    | Guests |
```

```
Status      |");
        System.out.println("+------------+--------------------+---------+-------------+-------------+--------
+-------------+");
        for (K2703047_Reservation r : list) {
            System.out.printf("| %-10s | %-18s | %-7s | %-12s | %-12s | %-6d | %-11s |\n",
                    r.getBookingId(),
                    r.getGuest().getFullName(),
                    r.getRoom().getRoomId(),
                    r.getCheckInDate(),
                    r.getCheckOutDate(),
                    r.getNumberOfGuests(),
                    r.getStatus().getDisplayName());
        }
        System.out.println("+------------+--------------------+---------+-------------+-------------+--------
+-------------+");
    }

    private void printHeader(String title) {
        System.out.println("\n" + "=".repeat(80));
        System.out.println(" " + title);
        System.out.println("=".repeat(80));
    }
}
```

```
package k2703047.smartstay.ui;

import k2703047.smartstay.model.K2703047_BedType;
import k2703047.smartstay.model.K2703047_Guest;
import k2703047.smartstay.model.K2703047_Invoice;
import k2703047.smartstay.model.K2703047_Reservation;
import k2703047.smartstay.model.K2703047_Room;
import k2703047.smartstay.model.K2703047_RoomFactory;
import k2703047.smartstay.model.K2703047_RoomStatus;
import k2703047.smartstay.model.K2703047_RoomType;
import k2703047.smartstay.service.K2703047_HotelSystem;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.util.List;
import java.util.Scanner;

/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Interactive Command Line Interface (CLI) for SmartStay Hotel Booking System.
 * Usability features:
 * - Colorized ANSI output headers and status messages
 * - Robust input validation and error feedback
 * - Clear table formats for rooms, guests, reservations, and invoices
 */
public class K2703047_CLIView {

    private final K2703047_HotelSystem hotelSystem;
    private final Scanner scanner;
    private final DateTimeFormatter dateFormatter = DateTimeFormatter.ofPattern("yyyy-MM-dd");

    public K2703047_CLIView(K2703047_HotelSystem hotelSystem) {
        this.hotelSystem = hotelSystem;
```

```
        this.scanner = new Scanner(System.in);
    }

    public void start() {
        boolean running = true;
        while (running) {
            printHeader("SMARTSTAY HOTEL BOOKING SYSTEM - MAIN PORTAL");
            System.out.println(" 1. Administrator Portal (Manage Rooms & Status)");
            System.out.println(" 2. Guest Portal (Register, Search, Book, Cancel, Invoices)");
            System.out.println(" 3. View All Reservations & Invoices");
            System.out.println(" 4. Launch Desktop GUI Interface");
            System.out.println(" 5. Exit System");
            System.out.print("\n Select an option [1-5]: ");

            String choice = scanner.nextLine().trim();
            switch (choice) {
                case "1":
                    handleAdminMenu();
                    break;
                case "2":
                    handleGuestMenu();
                    break;
                case "3":
                    handleViewReservationsAndInvoices();
                    break;
                case "4":
                    launchGui();
                    break;
                case "5":
                    System.out.println("\nExiting SmartStay Hotel Booking System. Goodbye!");
                    running = false;
                    break;
                default:
                    System.out.println("Invalid selection. Please choose an option from 1 to 5.");
            }
        }
    }

    // ========================================
    // ADMINISTRATOR SUB-MENU
    // ========================================
    private void handleAdminMenu() {
        boolean back = false;
        while (!back) {
            printHeader("ADMINISTRATOR PORTAL");
            System.out.println(" 1. View All Room Records");
            System.out.println(" 2. Add New Room Record");
            System.out.println(" 3. Update Room Record");
            System.out.println(" 4. Manage Room Availability Status");
            System.out.println(" 5. Remove Room Record");
            System.out.println(" 6. Return to Main Menu");
            System.out.print("\n Select admin operation [1-6]: ");

            String choice = scanner.nextLine().trim();
            switch (choice) {
                case "1":
                    displayAllRooms();
                    break;
                case "2":
                    handleAddNewRoom();
                    break;
                case "3":
```

```
                    handleUpdateRoom();
                    break;
                case "4":
                    handleManageRoomStatus();
                    break;
                case "5":
                    handleRemoveRoom();
                    break;
                case "6":
                    back = true;
                    break;
                default:
                    System.out.println("Invalid selection. Please try again.");
            }
        }
    }

    private void displayAllRooms() {
        printHeader("ALL ROOM RECORDS");
        List<K2703047_Room> rooms = hotelSystem.getAllRooms();
        if (rooms.isEmpty()) {
            System.out.println("No rooms registered in the system.");
            return;
        }
        printRoomTable(rooms);
    }

    private void handleAddNewRoom() {
        printHeader("ADD NEW ROOM RECORD");
        try {
            System.out.print("Enter Room ID (e.g. R-105, D-205, S-305): ");
            String roomId = scanner.nextLine().trim().toUpperCase();
            if (hotelSystem.getRoomById(roomId) != null) {
                System.out.println("[Error] A room with ID " + roomId + " already exists.");
                return;
            }

            System.out.println("Select Room Type:");
            System.out.println("1. Standard (LKR 6,000/night, No Breakfast, Extra: LKR 1,000, Tax: 10%)");
            System.out.println("2. Deluxe   (LKR 9,000/night, Breakfast Yes, Extra: LKR 1,500, Tax: 12%)");
            System.out.println("3. Suite    (LKR 14,000/night, Breakfast Yes, Extra: LKR 2,000, Tax: 15%)");
            System.out.println("4. Family Suite (LKR 18,000/night, Breakfast Yes, Extra: LKR 2,500, Tax:
15%)");
            System.out.print("Choice [1-4]: ");
            String tChoice = scanner.nextLine().trim();
            K2703047_RoomType type;
            switch (tChoice) {
                case "1": type = K2703047_RoomType.STANDARD; break;
                case "2": type = K2703047_RoomType.DELUXE; break;
                case "3": type = K2703047_RoomType.SUITE; break;
                case "4": type = K2703047_RoomType.FAMILY_SUITE; break;
                default:
                    System.out.println("[Error] Invalid room type selected.");
                    return;
            }

            System.out.println("Select Bed Type:");
            System.out.println("1. Single Bed\n2. Double Bed\n3. Queen Bed\n4. King Bed");
            System.out.print("Choice [1-4]: ");
            String bChoice = scanner.nextLine().trim();
            K2703047_BedType bedType;
            switch (bChoice) {
```

```
                case "1": bedType = K2703047_BedType.SINGLE; break;
                case "2": bedType = K2703047_BedType.DOUBLE; break;
                case "3": bedType = K2703047_BedType.QUEEN; break;
                case "4": bedType = K2703047_BedType.KING; break;
                default:
                    System.out.println("[Error] Invalid bed type selected.");
                    return;
            }

            System.out.print("Use default daily rate of LKR " + String.format("%,.0f", type.getDailyRate()) +
"? (Y/N): ");
            String useDef = scanner.nextLine().trim().toUpperCase();
            double rate = type.getDailyRate();
            if (useDef.equals("N")) {
                System.out.print("Enter custom daily rate (LKR): ");
                rate = Double.parseDouble(scanner.nextLine().trim());
            }

            K2703047_Room newRoom = K2703047_RoomFactory.createRoom(
                    roomId, type, bedType, rate, K2703047_RoomStatus.AVAILABLE, type.getStandardCapacity());
            boolean added = hotelSystem.addRoom(newRoom);
            if (added) {
                System.out.println("[Success] Room " + roomId + " successfully added to the system!");
            } else {
                System.out.println("[Error] Failed to add room.");
            }
        } catch (Exception e) {
            System.out.println("[Error] " + e.getMessage());
        }
    }

    private void handleUpdateRoom() {
        printHeader("UPDATE ROOM RECORD");
        displayAllRooms();
        System.out.print("Enter Room ID to update: ");
        String roomId = scanner.nextLine().trim().toUpperCase();
        K2703047_Room existing = hotelSystem.getRoomById(roomId);
        if (existing == null) {
            System.out.println("[Error] Room not found.");
            return;
        }

        try {
            System.out.print("Enter new Daily Rate (Current: LKR " + existing.getDailyRate() + ") or press
Enter to skip: ");
            String rInput = scanner.nextLine().trim();
            if (!rInput.isEmpty()) {
                existing.setDailyRate(Double.parseDouble(rInput));
            }

            System.out.println("Select new Bed Type (Current: " + existing.getBedType() + "):");
            System.out.println("1. Single Bed | 2. Double Bed | 3. Queen Bed | 4. King Bed | Enter to skip");
            String bInput = scanner.nextLine().trim();
            if (!bInput.isEmpty()) {
                switch (bInput) {
                    case "1": existing.setBedType(K2703047_BedType.SINGLE); break;
                    case "2": existing.setBedType(K2703047_BedType.DOUBLE); break;
                    case "3": existing.setBedType(K2703047_BedType.QUEEN); break;
                    case "4": existing.setBedType(K2703047_BedType.KING); break;
                }
            }
```

```
            System.out.println("[Success] Room " + roomId + " updated successfully!");
        } catch (Exception e) {
            System.out.println("[Error] Could not update room: " + e.getMessage());
        }
    }

    private void handleManageRoomStatus() {
        printHeader("MANAGE ROOM AVAILABILITY STATUS");
        displayAllRooms();
        System.out.print("Enter Room ID to update status: ");
        String roomId = scanner.nextLine().trim().toUpperCase();
        K2703047_Room room = hotelSystem.getRoomById(roomId);
        if (room == null) {
            System.out.println("[Error] Room not found.");
            return;
        }

        System.out.println("Current Status: " + room.getStatus().getDisplayName());
        System.out.println("Select New Status:");
        System.out.println(" 1. Available");
        System.out.println(" 2. Reserved");
        System.out.println(" 3. Under Maintenance");
        System.out.print("Choice [1-3]: ");
        String choice = scanner.nextLine().trim();

        K2703047_RoomStatus newStatus = null;
        switch (choice) {
            case "1": newStatus = K2703047_RoomStatus.AVAILABLE; break;
            case "2": newStatus = K2703047_RoomStatus.RESERVED; break;
            case "3": newStatus = K2703047_RoomStatus.UNDER_MAINTENANCE; break;
            default:
                System.out.println("[Error] Invalid status option.");
                return;
        }

        hotelSystem.updateRoomStatus(roomId, newStatus);
        System.out.println("[Success] Room " + roomId + " status updated to " + newStatus.getDisplayName() +
"!");
    }

    private void handleRemoveRoom() {
        printHeader("REMOVE ROOM RECORD");
        displayAllRooms();
        System.out.print("Enter Room ID to remove: ");
        String roomId = scanner.nextLine().trim().toUpperCase();
        try {
            boolean removed = hotelSystem.removeRoom(roomId);
            if (removed) {
                System.out.println("[Success] Room " + roomId + " removed successfully!");
            } else {
                System.out.println("[Error] Room not found.");
            }
        } catch (IllegalStateException e) {
            System.out.println("[Error] " + e.getMessage());
        }
    }

    // ========================================
    // GUEST SUB-MENU
    // ========================================
    private void handleGuestMenu() {
        boolean back = false;
```

```
        while (!back) {
            printHeader("GUEST / CUSTOMER PORTAL");
            System.out.println(" 1. Register as Customer");
            System.out.println(" 2. Search Available Rooms by Room Type and Date");
            System.out.println(" 3. Make Room Reservation");
            System.out.println(" 4. Search Reservations by Guest Name or Booking ID");
            System.out.println(" 5. View Reservations by Check-In Date");
            System.out.println(" 6. Cancel Eligible Reservation");
            System.out.println(" 7. Return to Main Menu");
            System.out.print("\n Select guest operation [1-7]: ");

            String choice = scanner.nextLine().trim();
            switch (choice) {
                case "1":
                    handleRegisterGuest();
                    break;
                case "2":
                    handleSearchRooms();
                    break;
                case "3":
                    handleMakeReservation();
                    break;
                case "4":
                    handleSearchReservations();
                    break;
                case "5":
                    handleViewByCheckInDate();
                    break;
                case "6":
                    handleCancelReservation();
                    break;
                case "7":
                    back = true;
                    break;
                default:
                    System.out.println("Invalid selection. Please try again.");
            }
        }
    }

    private void handleRegisterGuest() {
        printHeader("CUSTOMER REGISTRATION");
        try {
            System.out.print("Enter NIC or Passport Number: ");
            String nic = scanner.nextLine().trim().toUpperCase();
            if (hotelSystem.getGuest(nic) != null) {
                System.out.println("[Info] Customer is already registered: " + hotelSystem.getGuest(nic));
                return;
            }

            System.out.print("Enter Full Name: ");
            String name = scanner.nextLine().trim();

            System.out.print("Enter Contact Number (e.g. +94771234567): ");
            String phone = scanner.nextLine().trim();

            System.out.print("Enter Email Address: ");
            String email = scanner.nextLine().trim();

            boolean registered = hotelSystem.registerGuest(nic, name, phone, email);
            if (registered) {
                System.out.println("[Success] Customer registered successfully!");
```

```
            }
        } catch (Exception e) {
            System.out.println("[Error] Registration failed: " + e.getMessage());
        }
    }

    private void handleSearchRooms() {
        printHeader("SEARCH AVAILABLE ROOMS");
        try {
            System.out.println("Filter by Room Type:");
            System.out.println("0. All Types");
            System.out.println("1. Standard | 2. Deluxe | 3. Suite | 4. Family Suite");
            System.out.print("Choice [0-4]: ");
            String tc = scanner.nextLine().trim();
            K2703047_RoomType type = null;
            if (tc.equals("1")) type = K2703047_RoomType.STANDARD;
            else if (tc.equals("2")) type = K2703047_RoomType.DELUXE;
            else if (tc.equals("3")) type = K2703047_RoomType.SUITE;
            else if (tc.equals("4")) type = K2703047_RoomType.FAMILY_SUITE;

            LocalDate today = LocalDate.now();
            System.out.printf("Enter Check-In Date (YYYY-MM-DD, minimum %s): ", today.plusDays(2));
            String inStr = scanner.nextLine().trim();
            LocalDate checkIn = LocalDate.parse(inStr, dateFormatter);

            System.out.print("Enter Check-Out Date (YYYY-MM-DD): ");
            String outStr = scanner.nextLine().trim();
            LocalDate checkOut = LocalDate.parse(outStr, dateFormatter);

            List<K2703047_Room> available = hotelSystem.searchAvailableRooms(type, checkIn, checkOut);
            if (available.isEmpty()) {
                System.out.println("\n[Notice] No rooms are available for the selected dates and criteria.");
            } else {
                System.out.println("\n[Success] Found " + available.size() + " available room(s):");
                printRoomTable(available);
            }
        } catch (DateTimeParseException e) {
            System.out.println("[Error] Invalid date format. Please use YYYY-MM-DD.");
        } catch (Exception e) {
            System.out.println("[Error] " + e.getMessage());
        }
    }

    private void handleMakeReservation() {
        printHeader("MAKE ROOM RESERVATION");
        try {
            System.out.print("Enter Guest NIC/Passport: ");
            String nic = scanner.nextLine().trim().toUpperCase();
            K2703047_Guest guest = hotelSystem.getGuest(nic);
            if (guest == null) {
                System.out.println("[Error] Customer not registered. Please register first (Option 1).");
                return;
            }
            System.out.println("Guest: " + guest.getFullName() + " (" + guest.getEmailAddress() + ")");

            LocalDate today = LocalDate.now();
            System.out.printf("Enter Check-In Date (YYYY-MM-DD, at least 2 days ahead, min: %s): ",
today.plusDays(2));
            LocalDate checkIn = LocalDate.parse(scanner.nextLine().trim(), dateFormatter);

            System.out.print("Enter Check-Out Date (YYYY-MM-DD): ");
            LocalDate checkOut = LocalDate.parse(scanner.nextLine().trim(), dateFormatter);
```

```
            System.out.print("Enter Number of Guests: ");
            int guests = Integer.parseInt(scanner.nextLine().trim());

            // Show available rooms for these dates
            List<K2703047_Room> available = hotelSystem.searchAvailableRooms(null, checkIn, checkOut);
            if (available.isEmpty()) {
                System.out.println("[Error] No rooms are available for the selected dates.");
                return;
            }
            System.out.println("\nAvailable Rooms for selected dates:");
            printRoomTable(available);

            System.out.print("Enter Room ID to reserve: ");
            String roomId = scanner.nextLine().trim().toUpperCase();

            // Confirmation of LKR 10,000 advance
            System.out.println("\n[Business Policy] A refundable booking advance of LKR 10,000 is charged
now.");
            System.out.print("Confirm reservation and charge advance? (Y/N): ");
            String confirm = scanner.nextLine().trim().toUpperCase();
            if (!confirm.equals("Y")) {
                System.out.println("Reservation cancelled by user.");
                return;
            }

            K2703047_Reservation reservation = hotelSystem.makeReservation(nic, roomId, checkIn, checkOut,
guests);
            System.out.println("\n[SUCCESS] Reservation Confirmed!");
            System.out.println("Booking ID: " + reservation.getBookingId());

            // Generate invoice immediately
            K2703047_Invoice invoice = hotelSystem.generateInvoice(reservation.getBookingId());
            System.out.println(invoice.generateInvoiceText());

            // Auto-export invoice
            String invoicePath = "invoices/" + reservation.getBookingId() + ".txt";
            invoice.exportToFile(invoicePath);
            System.out.println("[Notice] Invoice automatically saved to: " + invoicePath);

        } catch (DateTimeParseException e) {
            System.out.println("[Error] Invalid date format. Please use YYYY-MM-DD.");
        } catch (Exception e) {
            System.out.println("[Error] Reservation failed: " + e.getMessage());
        }
    }

    private void handleSearchReservations() {
        printHeader("SEARCH RESERVATIONS");
        System.out.println(" 1. Search by Booking ID (Demonstrates O(log N) Binary Search)");
        System.out.println(" 2. Search by Guest Name");
        System.out.print("Select search type [1-2]: ");
        String opt = scanner.nextLine().trim();

        if (opt.equals("1")) {
            System.out.print("Enter Booking ID (e.g. BK-1001): ");
            String bid = scanner.nextLine().trim().toUpperCase();
            K2703047_Reservation r = hotelSystem.searchReservationBinary(bid);
            if (r != null) {
                System.out.println("\n[Found via Binary Search Algorithm]:");
                System.out.println(r);
                showInvoiceOption(r.getBookingId());
```

```
            } else {
                System.out.println("[Not Found] No reservation exists with Booking ID: " + bid);
            }
        } else if (opt.equals("2")) {
            System.out.print("Enter Guest Name or keyword: ");
            String name = scanner.nextLine().trim();
            List<K2703047_Reservation> results = hotelSystem.searchReservationsByGuestName(name);
            if (results.isEmpty()) {
                System.out.println("No reservations found for guest matching '" + name + "'.");
            } else {
                printReservationTable(results);
                System.out.print("Enter Booking ID to view invoice (or Enter to skip): ");
                String bid = scanner.nextLine().trim().toUpperCase();
                if (!bid.isEmpty()) {
                    showInvoiceOption(bid);
                }
            }
        }
    }

    private void handleViewByCheckInDate() {
        printHeader("VIEW RESERVATIONS BY CHECK-IN DATE");
        try {
            System.out.print("Enter Check-In Date (YYYY-MM-DD): ");
            LocalDate date = LocalDate.parse(scanner.nextLine().trim(), dateFormatter);
            List<K2703047_Reservation> list = hotelSystem.getReservationsByCheckInDate(date);
            if (list.isEmpty()) {
                System.out.println("No reservations scheduled to check in on " + date);
            } else {
                System.out.println("[Indexed via TreeMap Red-Black Tree] Found " + list.size() + "
reservation(s):");
                printReservationTable(list);
            }
        } catch (Exception e) {
            System.out.println("[Error] " + e.getMessage());
        }
    }

    private void handleCancelReservation() {
        printHeader("CANCEL RESERVATION");
        System.out.print("Enter Booking ID to cancel: ");
        String bid = scanner.nextLine().trim().toUpperCase();
        K2703047_Reservation r = hotelSystem.getReservation(bid);
        if (r == null) {
            System.out.println("[Error] Reservation not found.");
            return;
        }

        System.out.println("Reservation Details:");
        System.out.println(r);
        System.out.println("\n[Business Policy Reminder] Reservations cannot be cancelled within 24 hours of
check-in.");
        System.out.print("Are you sure you want to cancel this booking? (Y/N): ");
        if (!scanner.nextLine().trim().equalsIgnoreCase("Y")) {
            System.out.println("Cancellation aborted.");
            return;
        }

        boolean cancelled = hotelSystem.cancelReservation(bid);
        if (cancelled) {
            System.out.println("\n[Success] Reservation " + bid + " cancelled successfully!");
            System.out.println("[Notice] Refund of LKR 10,000 booking advance processed to " +
```

```
r.getGuest().getFullName());
        } else {
            System.out.println("\n[Error] Cancellation Rejected: Reservation cannot be cancelled within 24
hours of check-in date or is already cancelled.");
        }
    }

    private void handleViewReservationsAndInvoices() {
        printHeader("ALL SYSTEM RESERVATIONS");
        List<K2703047_Reservation> all = hotelSystem.getAllReservations();
        if (all.isEmpty()) {
            System.out.println("No reservations recorded yet.");
            return;
        }
        printReservationTable(all);

        System.out.print("\nEnter Booking ID to view detailed invoice (or Enter to return): ");
        String bid = scanner.nextLine().trim().toUpperCase();
        if (!bid.isEmpty()) {
            showInvoiceOption(bid);
        }
    }

    private void showInvoiceOption(String bookingId) {
        try {
            K2703047_Invoice invoice = hotelSystem.generateInvoice(bookingId);
            System.out.println(invoice.generateInvoiceText());
            System.out.print("Export invoice to file? (Y/N): ");
            if (scanner.nextLine().trim().equalsIgnoreCase("Y")) {
                String path = "invoices/" + bookingId + ".txt";
                invoice.exportToFile(path);
                System.out.println("Invoice saved to: " + path);
            }
        } catch (Exception e) {
            System.out.println("[Error] " + e.getMessage());
        }
    }

    private void launchGui() {
        System.out.println("\nLaunching SmartStay Desktop GUI...");
        new Thread(() -> {
            try {
                K2703047_GUIView.launch(hotelSystem);
            } catch (Exception e) {
                System.out.println("GUI error: " + e.getMessage());
            }
        }).start();
    }

    // Table printing helper utilities
    private void printRoomTable(List<K2703047_Room> rooms) {
        System.out.println("+----------+---------------+------------+-----------------+------------------+--
-------------+");
        System.out.println("| Room ID  | Room Type     | Bed Type   | Daily Rate(LKR) | Breakfast Included|
Status        |");
        System.out.println("+----------+---------------+------------+-----------------+------------------+--
-------------+");
        for (K2703047_Room r : rooms) {
            System.out.printf("| %-8s | %-13s | %-10s | LKR %11.2f | %-17s | %-13s |\n",
                    r.getRoomId(),
                    r.getRoomType().getDisplayName(),
                    r.getBedType().getDisplayName(),
```

```
                    r.getDailyRate(),
                    r.hasComplimentaryBreakfast() ? "Yes" : "No",
                    r.getStatus().getDisplayName());
        }
        System.out.println("+----------+---------------+------------+-----------------+-------------------+--
-------------+");
    }

    private void printReservationTable(List<K2703047_Reservation> list) {
        System.out.println("+------------+--------------------+---------+-------------+-------------+------
--+-------------+");
        System.out.println("| Booking ID | Guest Name         | Room ID | Check-In    | Check-Out   |
Guests | Status      |");
        System.out.println("+------------+--------------------+---------+-------------+-------------+------
--+-------------+");
        for (K2703047_Reservation r : list) {
            System.out.printf("| %-10s | %-18s | %-7s | %-12s | %-12s | %-6d | %-11s |\n",
                    r.getBookingId(),
                    r.getGuest().getFullName(),
                    r.getRoom().getRoomId(),
                    r.getCheckInDate(),
                    r.getCheckOutDate(),
                    r.getNumberOfGuests(),
                    r.getStatus().getDisplayName());
        }
        System.out.println("+------------+--------------------+---------+-------------+-------------+------
--+-------------+");
    }

    private void printHeader(String title) {
        System.out.println("\n" + "=".repeat(80));
        System.out.println(" " + title);
        System.out.println("=".repeat(80));
    }
}
```

```
package k2703047.smartstay.ui;

import k2703047.smartstay.model.K2703047_BedType;
import k2703047.smartstay.model.K2703047_Guest;
import k2703047.smartstay.model.K2703047_Invoice;
import k2703047.smartstay.model.K2703047_Reservation;
import k2703047.smartstay.model.K2703047_ReservationStatus;
import k2703047.smartstay.model.K2703047_Room;
import k2703047.smartstay.model.K2703047_RoomFactory;
import k2703047.smartstay.model.K2703047_RoomStatus;
import k2703047.smartstay.model.K2703047_RoomType;
import k2703047.smartstay.service.K2703047_HotelSystem;

import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.border.TitledBorder;
import javax.swing.table.DefaultTableCellRenderer;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.datatransfer.StringSelection;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.List;
```

```
/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Desktop Graphical User Interface (Java Swing) for SmartStay Hotel Booking System.
 * Evaluates UI/UX design:
 * - Professional layout with tabs, tables, status indicators, and responsive panels
 * - Form validation and clear feedback dialogs
 * - Live billing estimator and printable/exportable invoice preview
 */
public class K2703047_GUIView extends JFrame {

    private final K2703047_HotelSystem hotelSystem;
    private final DateTimeFormatter dateFormatter = DateTimeFormatter.ofPattern("yyyy-MM-dd");

    // Table Models
    private DefaultTableModel roomTableModel;
    private DefaultTableModel guestTableModel;
    private DefaultTableModel reservationTableModel;
    private DefaultTableModel searchResultsTableModel;

    // Component references
    private JTable roomTable;
    private JTable guestTable;
    private JTable reservationTable;
    private JTable searchResultsTable;
    private JTextArea invoiceTextArea;
    private JComboBox<String> guestSelectorCombo;
    private JComboBox<String> invoiceSelectorCombo;

    public K2703047_GUIView(K2703047_HotelSystem hotelSystem) {
        super("SmartStay Hotel Chain - Room Reservation System | Student ID: 2703047");
        this.hotelSystem = hotelSystem;
        initializeUI();
        refreshAllData();
    }

    public static void launch(K2703047_HotelSystem hotelSystem) {
        SwingUtilities.invokeLater(() -> {
            try {
                UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
            } catch (Exception ignored) {}
            K2703047_GUIView frame = new K2703047_GUIView(hotelSystem);
            frame.setVisible(true);
        });
    }

    private void initializeUI() {
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        setSize(1100, 750);
        setMinimumSize(new Dimension(950, 650));
        setLocationRelativeTo(null);

        JPanel mainPanel = new JPanel(new BorderLayout(10, 10));
        mainPanel.setBorder(new EmptyBorder(12, 12, 12, 12));

        // Top Header
        JPanel headerPanel = new JPanel(new BorderLayout());
        headerPanel.setBackground(new Color(24, 43, 73));
        headerPanel.setBorder(new EmptyBorder(14, 18, 14, 18));
```

```
        JLabel titleLabel = new JLabel("SMARTSTAY HOTEL CHAIN - SRI LANKA");
        titleLabel.setFont(new Font("Segoe UI", Font.BOLD, 20));
        titleLabel.setForeground(Color.WHITE);

        JLabel subLabel = new JLabel("CI6115 Programming III | Patterns & Algorithms | Student ID: 2703047");
        subLabel.setFont(new Font("Segoe UI", Font.PLAIN, 12));
        subLabel.setForeground(new Color(200, 220, 255));

        headerPanel.add(titleLabel, BorderLayout.NORTH);
        headerPanel.add(subLabel, BorderLayout.SOUTH);
        mainPanel.add(headerPanel, BorderLayout.NORTH);

        // Tabbed Pane
        JTabbedPane tabbedPane = new JTabbedPane();
        tabbedPane.setFont(new Font("Segoe UI", Font.PLAIN, 13));

        tabbedPane.addTab("  Rooms (Admin)  ", createRoomsPanel());
        tabbedPane.addTab("  Guest Registration  ", createGuestPanel());
        tabbedPane.addTab("  Search & Book  ", createBookingPanel());
        tabbedPane.addTab("  Reservations & Cancellation  ", createReservationManagementPanel());
        tabbedPane.addTab("  Detailed Invoices  ", createInvoicePanel());

        mainPanel.add(tabbedPane, BorderLayout.CENTER);

        // Bottom Status Bar
        JPanel statusBar = new JPanel(new BorderLayout());
        statusBar.setBorder(new EmptyBorder(5, 5, 5, 5));
        JLabel statusLabel = new JLabel("Ready. System loaded with initial rooms, packages, and registered
guests.");
        statusLabel.setFont(new Font("Segoe UI", Font.ITALIC, 11));
        statusBar.add(statusLabel, BorderLayout.WEST);

        mainPanel.add(statusBar, BorderLayout.SOUTH);

        setContentPane(mainPanel);
    }

    // ========================================
    // TAB 1: ROOMS MANAGEMENT PANEL (ADMIN)
    // ========================================
    private JPanel createRoomsPanel() {
        JPanel panel = new JPanel(new BorderLayout(8, 8));
        panel.setBorder(new EmptyBorder(10, 10, 10, 10));

        // Table
        String[] cols = {"Room ID", "Room Type", "Bed Type", "Daily Rate (LKR)", "Breakfast", "Capacity",
"Status"};
        roomTableModel = new DefaultTableModel(cols, 0) {
            @Override
            public boolean isCellEditable(int r, int c) { return false; }
        };
        roomTable = new JTable(roomTableModel);
        roomTable.setRowHeight(26);
        roomTable.setFont(new Font("Segoe UI", Font.PLAIN, 13));
        roomTable.getTableHeader().setFont(new Font("Segoe UI", Font.BOLD, 13));
        alignTableColumns(roomTable, 0, 1, 2, 4, 5, 6);

        JScrollPane scrollPane = new JScrollPane(roomTable);
        panel.add(scrollPane, BorderLayout.CENTER);

        // Toolbar
        JPanel toolbar = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 6));
```

```
        JButton btnAdd = new JButton("Add Room");
        JButton btnUpdateRate = new JButton("Update Rate");
        JButton btnStatus = new JButton("Change Status");
        JButton btnRemove = new JButton("Remove Room");
        JButton btnRefresh = new JButton("Refresh");

        toolbar.add(btnAdd);
        toolbar.add(btnUpdateRate);
        toolbar.add(btnStatus);
        toolbar.add(btnRemove);
        toolbar.add(btnRefresh);

        panel.add(toolbar, BorderLayout.SOUTH);

        // Action Listeners
        btnAdd.addActionListener(e -> showAddRoomDialog());
        btnUpdateRate.addActionListener(e -> showUpdateRoomDialog());
        btnStatus.addActionListener(e -> showChangeStatusDialog());
        btnRemove.addActionListener(e -> handleRemoveRoomGUI());
        btnRefresh.addActionListener(e -> refreshRoomsTable());

        return panel;
    }

    private void showAddRoomDialog() {
        JTextField idField = new JTextField(10);
        JComboBox<K2703047_RoomType> typeCombo = new JComboBox<>(K2703047_RoomType.values());
        JComboBox<K2703047_BedType> bedCombo = new JComboBox<>(K2703047_BedType.values());
        JTextField rateField = new JTextField(10);

        typeCombo.addActionListener(e -> {
            K2703047_RoomType t = (K2703047_RoomType) typeCombo.getSelectedItem();
            if (t != null) rateField.setText(String.valueOf(t.getDailyRate()));
        });
        typeCombo.setSelectedIndex(0);

        JPanel form = new JPanel(new GridLayout(4, 2, 8, 8));
        form.add(new JLabel("Room ID (e.g. R-105):"));
        form.add(idField);
        form.add(new JLabel("Room Type:"));
        form.add(typeCombo);
        form.add(new JLabel("Bed Type:"));
        form.add(bedCombo);
        form.add(new JLabel("Daily Rate (LKR):"));
        form.add(rateField);

        int res = JOptionPane.showConfirmDialog(this, form, "Add New Room", JOptionPane.OK_CANCEL_OPTION);
        if (res == JOptionPane.OK_OPTION) {
            try {
                String id = idField.getText().trim().toUpperCase();
                K2703047_RoomType type = (K2703047_RoomType) typeCombo.getSelectedItem();
                K2703047_BedType bed = (K2703047_BedType) bedCombo.getSelectedItem();
                double rate = Double.parseDouble(rateField.getText().trim());

                if (hotelSystem.getRoomById(id) != null) {
                    JOptionPane.showMessageDialog(this, "Room ID already exists!", "Error",
JOptionPane.ERROR_MESSAGE);
                    return;
                }
                K2703047_Room room = K2703047_RoomFactory.createRoom(id, type, bed, rate,
K2703047_RoomStatus.AVAILABLE, type.getStandardCapacity());
```

```
                hotelSystem.addRoom(room);
                refreshRoomsTable();
                JOptionPane.showMessageDialog(this, "Room " + id + " added successfully!");
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Error adding room: " + ex.getMessage(), "Input Error",
JOptionPane.ERROR_MESSAGE);
            }
        }
    }

    private void showUpdateRoomDialog() {
        int row = roomTable.getSelectedRow();
        if (row < 0) {
            JOptionPane.showMessageDialog(this, "Please select a room to update.", "Selection Required",
JOptionPane.WARNING_MESSAGE);
            return;
        }
        String roomId = (String) roomTableModel.getValueAt(row, 0);
        K2703047_Room room = hotelSystem.getRoomById(roomId);
        if (room == null) return;

        String input = JOptionPane.showInputDialog(this, "Enter new Daily Rate (LKR):", room.getDailyRate());
        if (input != null && !input.trim().isEmpty()) {
            try {
                double newRate = Double.parseDouble(input.trim());
                room.setDailyRate(newRate);
                refreshRoomsTable();
                JOptionPane.showMessageDialog(this, "Rate updated for room " + roomId);
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Invalid rate: " + ex.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
            }
        }
    }

    private void showChangeStatusDialog() {
        int row = roomTable.getSelectedRow();
        if (row < 0) {
            JOptionPane.showMessageDialog(this, "Please select a room to modify status.", "Selection Required",
JOptionPane.WARNING_MESSAGE);
            return;
        }
        String roomId = (String) roomTableModel.getValueAt(row, 0);
        K2703047_RoomStatus current = (K2703047_RoomStatus) roomTableModel.getValueAt(row, 6);

        K2703047_RoomStatus selected = (K2703047_RoomStatus) JOptionPane.showInputDialog(
                this, "Select new availability status for " + roomId + ":",
                "Change Room Status", JOptionPane.QUESTION_MESSAGE, null,
                K2703047_RoomStatus.values(), current);

        if (selected != null) {
            hotelSystem.updateRoomStatus(roomId, selected);
            refreshRoomsTable();
            JOptionPane.showMessageDialog(this, "Room " + roomId + " status set to " + selected);
        }
    }

    private void handleRemoveRoomGUI() {
        int row = roomTable.getSelectedRow();
        if (row < 0) {
            JOptionPane.showMessageDialog(this, "Please select a room to remove.", "Selection Required",
JOptionPane.WARNING_MESSAGE);
```

```
            return;
        }
        String roomId = (String) roomTableModel.getValueAt(row, 0);
        int confirm = JOptionPane.showConfirmDialog(this, "Are you sure you want to remove room " + roomId +
"?", "Confirm Removal", JOptionPane.YES_NO_OPTION);
        if (confirm == JOptionPane.YES_OPTION) {
            try {
                hotelSystem.removeRoom(roomId);
                refreshRoomsTable();
                JOptionPane.showMessageDialog(this, "Room " + roomId + " removed.");
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, ex.getMessage(), "Cannot Remove",
JOptionPane.ERROR_MESSAGE);
            }
        }
    }

    // ========================================
    // TAB 2: GUEST MANAGEMENT PANEL
    // ========================================
    private JPanel createGuestPanel() {
        JPanel panel = new JPanel(new BorderLayout(10, 10));
        panel.setBorder(new EmptyBorder(10, 10, 10, 10));

        // Top Form
        JPanel formPanel = new JPanel(new GridLayout(2, 4, 8, 8));
        formPanel.setBorder(BorderFactory.createTitledBorder("Register New Customer"));

        JTextField nicField = new JTextField();
        JTextField nameField = new JTextField();
        JTextField phoneField = new JTextField();
        JTextField emailField = new JTextField();

        formPanel.add(new JLabel("NIC / Passport:"));
        formPanel.add(new JLabel("Full Name:"));
        formPanel.add(new JLabel("Contact Number:"));
        formPanel.add(new JLabel("Email Address:"));

        formPanel.add(nicField);
        formPanel.add(nameField);
        formPanel.add(phoneField);
        formPanel.add(emailField);

        JPanel formWrapper = new JPanel(new BorderLayout());
        formWrapper.add(formPanel, BorderLayout.CENTER);

        JButton btnRegister = new JButton("Register Customer");
        btnRegister.setFont(new Font("Segoe UI", Font.BOLD, 12));
        JPanel btnWrapper = new JPanel(new FlowLayout(FlowLayout.RIGHT));
        btnWrapper.add(btnRegister);
        formWrapper.add(btnWrapper, BorderLayout.SOUTH);

        panel.add(formWrapper, BorderLayout.NORTH);

        // Guest Table
        String[] cols = {"NIC / Passport", "Full Name", "Contact Number", "Email Address"};
        guestTableModel = new DefaultTableModel(cols, 0) {
            @Override
            public boolean isCellEditable(int r, int c) { return false; }
        };
        guestTable = new JTable(guestTableModel);
        guestTable.setRowHeight(24);
```

```
        panel.add(new JScrollPane(guestTable), BorderLayout.CENTER);

        btnRegister.addActionListener(e -> {
            try {
                String nic = nicField.getText().trim().toUpperCase();
                String name = nameField.getText().trim();
                String phone = phoneField.getText().trim();
                String email = emailField.getText().trim();

                boolean ok = hotelSystem.registerGuest(nic, name, phone, email);
                if (ok) {
                    JOptionPane.showMessageDialog(this, "Guest registered successfully!");
                    nicField.setText("");
                    nameField.setText("");
                    phoneField.setText("");
                    emailField.setText("");
                    refreshGuestsTable();
                } else {
                    JOptionPane.showMessageDialog(this, "Guest already registered with this NIC/Passport.",
"Warning", JOptionPane.WARNING_MESSAGE);
                }
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Validation Error: " + ex.getMessage(), "Input Error",
JOptionPane.ERROR_MESSAGE);
            }
        });

        return panel;
    }

    // ========================================
    // TAB 3: SEARCH & BOOKING PANEL
    // ========================================
    private JPanel createBookingPanel() {
        JPanel panel = new JPanel(new BorderLayout(10, 10));
        panel.setBorder(new EmptyBorder(10, 10, 10, 10));

        // Search Criteria Panel
        JPanel searchPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 12, 8));
        searchPanel.setBorder(BorderFactory.createTitledBorder("Search Available Rooms"));

        JComboBox<String> typeFilterCombo = new JComboBox<>(new String[]{"All Room Types", "Standard", "Deluxe",
"Suite", "Family Suite"});
        LocalDate minCheckIn = LocalDate.now().plusDays(2);
        JTextField inDateField = new JTextField(minCheckIn.format(dateFormatter), 9);
        JTextField outDateField = new JTextField(minCheckIn.plusDays(3).format(dateFormatter), 9);
        JButton btnSearch = new JButton("Find Available Rooms");
        btnSearch.setFont(new Font("Segoe UI", Font.BOLD, 12));

        searchPanel.add(new JLabel("Room Type:"));
        searchPanel.add(typeFilterCombo);
        searchPanel.add(new JLabel("Check-In (Min 2 days advance):"));
        searchPanel.add(inDateField);
        searchPanel.add(new JLabel("Check-Out:"));
        searchPanel.add(outDateField);
        searchPanel.add(btnSearch);

        panel.add(searchPanel, BorderLayout.NORTH);

        // Search Results Table
        String[] cols = {"Room ID", "Type", "Bed", "Daily Rate (LKR)", "Breakfast Included", "Base Capacity"};
        searchResultsTableModel = new DefaultTableModel(cols, 0) {
```

```
            @Override
            public boolean isCellEditable(int r, int c) { return false; }
        };
        searchResultsTable = new JTable(searchResultsTableModel);
        searchResultsTable.setRowHeight(24);
        panel.add(new JScrollPane(searchResultsTable), BorderLayout.CENTER);

        // Booking Action Panel at Bottom
        JPanel actionPanel = new JPanel(new GridLayout(2, 1, 6, 6));
        actionPanel.setBorder(BorderFactory.createTitledBorder("Complete Room Reservation"));

        JPanel inputRow = new JPanel(new FlowLayout(FlowLayout.LEFT, 15, 6));
        guestSelectorCombo = new JComboBox<>();
        JSpinner guestCountSpinner = new JSpinner(new SpinnerNumberModel(2, 1, 10, 1));
        JButton btnBook = new JButton("Confirm Booking & Pay LKR 10,000 Advance");
        btnBook.setFont(new Font("Segoe UI", Font.BOLD, 13));
        btnBook.setBackground(new Color(24, 134, 75));
        btnBook.setForeground(Color.WHITE);

        inputRow.add(new JLabel("Select Registered Guest:"));
        inputRow.add(guestSelectorCombo);
        inputRow.add(new JLabel("Number of Guests:"));
        inputRow.add(guestCountSpinner);
        inputRow.add(btnBook);

        JLabel policyLabel = new JLabel("  * Refundable advance of LKR 10,000 is charged at reservation. Free
cancellation until 24h prior to check-in.");
        policyLabel.setFont(new Font("Segoe UI", Font.ITALIC, 11));
        policyLabel.setForeground(new Color(80, 80, 80));

        actionPanel.add(inputRow);
        actionPanel.add(policyLabel);

        panel.add(actionPanel, BorderLayout.SOUTH);

        // Action Listener for Search
        btnSearch.addActionListener(e -> {
            try {
                LocalDate inDate = LocalDate.parse(inDateField.getText().trim(), dateFormatter);
                LocalDate outDate = LocalDate.parse(outDateField.getText().trim(), dateFormatter);

                int typeIdx = typeFilterCombo.getSelectedIndex();
                K2703047_RoomType filterType = null;
                if (typeIdx == 1) filterType = K2703047_RoomType.STANDARD;
                else if (typeIdx == 2) filterType = K2703047_RoomType.DELUXE;
                else if (typeIdx == 3) filterType = K2703047_RoomType.SUITE;
                else if (typeIdx == 4) filterType = K2703047_RoomType.FAMILY_SUITE;

                List<K2703047_Room> results = hotelSystem.searchAvailableRooms(filterType, inDate, outDate);
                searchResultsTableModel.setRowCount(0);
                for (K2703047_Room r : results) {
                    searchResultsTableModel.addRow(new Object[]{
                            r.getRoomId(),
                            r.getRoomType().getDisplayName(),
                            r.getBedType().getDisplayName(),
                            String.format("LKR %,.2f", r.getDailyRate()),
                            r.hasComplimentaryBreakfast() ? "Yes" : "No",
                            r.getBaseCapacity()
                    });
                }
                if (results.isEmpty()) {
                    JOptionPane.showMessageDialog(this, "No available rooms for the selected dates.", "Search
```

```
Results", JOptionPane.INFORMATION_MESSAGE);
                }
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Search error: " + ex.getMessage(), "Input Error",
JOptionPane.ERROR_MESSAGE);
            }
        });

        // Action Listener for Booking
        btnBook.addActionListener(e -> {
            int row = searchResultsTable.getSelectedRow();
            if (row < 0) {
                JOptionPane.showMessageDialog(this, "Please select an available room from the search results
table.", "Selection Required", JOptionPane.WARNING_MESSAGE);
                return;
            }
            if (guestSelectorCombo.getSelectedIndex() < 0) {
                JOptionPane.showMessageDialog(this, "Please select or register a guest first.", "Guest
Required", JOptionPane.WARNING_MESSAGE);
                return;
            }

            try {
                String roomId = (String) searchResultsTableModel.getValueAt(row, 0);
                String selectedGuest = (String) guestSelectorCombo.getSelectedItem();
                String nic = selectedGuest.substring(selectedGuest.indexOf("[") + 1,
selectedGuest.indexOf("]"));

                LocalDate inDate = LocalDate.parse(inDateField.getText().trim(), dateFormatter);
                LocalDate outDate = LocalDate.parse(outDateField.getText().trim(), dateFormatter);
                int guestCount = (int) guestCountSpinner.getValue();

                K2703047_Reservation res = hotelSystem.makeReservation(nic, roomId, inDate, outDate,
guestCount);
                refreshAllData();

                JOptionPane.showMessageDialog(this,
                        "Reservation confirmed successfully!\n" +
                        "Booking ID: " + res.getBookingId() + "\n" +
                        "Refundable Advance Charged: LKR 10,000\n\n" +
                        "View detailed invoice in the 'Detailed Invoices' tab.",
                        "Reservation Confirmed", JOptionPane.INFORMATION_MESSAGE);

                // Auto-show invoice
                displayInvoiceForBooking(res.getBookingId());

            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Booking Failed: " + ex.getMessage(), "Reservation Error",
JOptionPane.ERROR_MESSAGE);
            }
        });

        return panel;
    }

    // ========================================
    // TAB 4: RESERVATIONS & CANCELLATION PANEL
    // ========================================
    private JPanel createReservationManagementPanel() {
        JPanel panel = new JPanel(new BorderLayout(10, 10));
        panel.setBorder(new EmptyBorder(10, 10, 10, 10));
```

```
        // Filter / Search Toolbar
        JPanel searchBar = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 6));
        searchBar.setBorder(BorderFactory.createTitledBorder("Search & Filter Reservations"));

        JTextField searchField = new JTextField(12);
        JButton btnSearchName = new JButton("Search Guest Name");
        JButton btnBinarySearchId = new JButton("Binary Search ID");
        JTextField checkInDateField = new JTextField(9);
        JButton btnFilterDate = new JButton("Filter by Check-In Date");
        JButton btnShowAll = new JButton("Show All");

        searchBar.add(new JLabel("Keyword / ID:"));
        searchBar.add(searchField);
        searchBar.add(btnSearchName);
        searchBar.add(btnBinarySearchId);
        searchBar.add(new JLabel("Check-In:"));
        searchBar.add(checkInDateField);
        searchBar.add(btnFilterDate);
        searchBar.add(btnShowAll);

        panel.add(searchBar, BorderLayout.NORTH);

        // Reservations Table
        String[] cols = {"Booking ID", "Guest Name", "Room ID", "Check-In", "Check-Out", "Nights", "Guests",
"Status"};
        reservationTableModel = new DefaultTableModel(cols, 0) {
            @Override
            public boolean isCellEditable(int r, int c) { return false; }
        };
        reservationTable = new JTable(reservationTableModel);
        reservationTable.setRowHeight(24);
        panel.add(new JScrollPane(reservationTable), BorderLayout.CENTER);

        // Bottom Actions
        JPanel bottomBar = new JPanel(new FlowLayout(FlowLayout.RIGHT, 12, 6));
        JButton btnCancel = new JButton("Cancel Selected Reservation");
        btnCancel.setForeground(Color.RED);
        btnCancel.setFont(new Font("Segoe UI", Font.BOLD, 12));

        JButton btnViewInvoice = new JButton("View Detailed Invoice");
        btnViewInvoice.setFont(new Font("Segoe UI", Font.BOLD, 12));

        bottomBar.add(btnCancel);
        bottomBar.add(btnViewInvoice);
        panel.add(bottomBar, BorderLayout.SOUTH);

        // Action Listeners
        btnSearchName.addActionListener(e -> {
            String kw = searchField.getText().trim();
            List<K2703047_Reservation> list = hotelSystem.searchReservationsByGuestName(kw);
            populateReservationTable(list);
        });

        btnBinarySearchId.addActionListener(e -> {
            String bid = searchField.getText().trim().toUpperCase();
            K2703047_Reservation r = hotelSystem.searchReservationBinary(bid);
            if (r != null) {
                populateReservationTable(List.of(r));
                JOptionPane.showMessageDialog(this, "Found reservation via O(log N) Binary Search Algorithm!",
"Binary Search Result", JOptionPane.INFORMATION_MESSAGE);
            } else {
                JOptionPane.showMessageDialog(this, "No reservation found with Booking ID: " + bid, "Not Found",
```

```
JOptionPane.WARNING_MESSAGE);
            }
        });

        btnFilterDate.addActionListener(e -> {
            try {
                LocalDate date = LocalDate.parse(checkInDateField.getText().trim(), dateFormatter);
                List<K2703047_Reservation> list = hotelSystem.getReservationsByCheckInDate(date);
                populateReservationTable(list);
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Invalid date format. Use YYYY-MM-DD", "Error",
JOptionPane.ERROR_MESSAGE);
            }
        });

        btnShowAll.addActionListener(e -> refreshReservationsTable());

        btnCancel.addActionListener(e -> {
            int row = reservationTable.getSelectedRow();
            if (row < 0) {
                JOptionPane.showMessageDialog(this, "Please select a reservation to cancel.", "Selection
Required", JOptionPane.WARNING_MESSAGE);
                return;
            }
            String bid = (String) reservationTableModel.getValueAt(row, 0);
            int confirm = JOptionPane.showConfirmDialog(this,
                    "Are you sure you want to cancel booking " + bid + "?\n" +
                    "Notice: Cancellation is strictly prohibited within 24 hours of check-in.",
                    "Confirm Cancellation", JOptionPane.YES_NO_OPTION);

            if (confirm == JOptionPane.YES_OPTION) {
                boolean ok = hotelSystem.cancelReservation(bid);
                if (ok) {
                    JOptionPane.showMessageDialog(this,
                            "Reservation " + bid + " cancelled successfully.\n" +
                            "Refund of LKR 10,000 booking advance processed.",
                            "Cancellation Approved", JOptionPane.INFORMATION_MESSAGE);
                    refreshAllData();
                } else {
                    JOptionPane.showMessageDialog(this,
                            "Cancellation Rejected!\n" +
                            "Business rule violation: Reservations cannot be modified or cancelled within 24
hours of check-in.",
                            "Cancellation Policy Violation", JOptionPane.ERROR_MESSAGE);
                }
            }
        });

        btnViewInvoice.addActionListener(e -> {
            int row = reservationTable.getSelectedRow();
            if (row < 0) {
                JOptionPane.showMessageDialog(this, "Please select a reservation to view its invoice.",
"Selection Required", JOptionPane.WARNING_MESSAGE);
                return;
            }
            String bid = (String) reservationTableModel.getValueAt(row, 0);
            displayInvoiceForBooking(bid);
        });

        return panel;
    }
```

```
    // ========================================
    // TAB 5: INVOICE VIEWER PANEL
    // ========================================
    private JPanel createInvoicePanel() {
        JPanel panel = new JPanel(new BorderLayout(10, 10));
        panel.setBorder(new EmptyBorder(10, 10, 10, 10));

        // Selector Bar
        JPanel topBar = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 6));
        invoiceSelectorCombo = new JComboBox<>();
        JButton btnLoad = new JButton("Load Invoice");
        JButton btnSaveFile = new JButton("Export Invoice to Text File");
        JButton btnCopy = new JButton("Copy to Clipboard");

        topBar.add(new JLabel("Select Booking:"));
        topBar.add(invoiceSelectorCombo);
        topBar.add(btnLoad);
        topBar.add(btnSaveFile);
        topBar.add(btnCopy);

        panel.add(topBar, BorderLayout.NORTH);

        // Monospace Text Area for Invoice
        invoiceTextArea = new JTextArea();
        invoiceTextArea.setEditable(false);
        invoiceTextArea.setFont(new Font("Consolas", Font.PLAIN, 13));
        invoiceTextArea.setBackground(new Color(250, 252, 255));
        invoiceTextArea.setBorder(new EmptyBorder(10, 10, 10, 10));

        panel.add(new JScrollPane(invoiceTextArea), BorderLayout.CENTER);

        btnLoad.addActionListener(e -> {
            String selected = (String) invoiceSelectorCombo.getSelectedItem();
            if (selected != null && selected.contains(" ")) {
                String bid = selected.split(" ")[0];
                displayInvoiceForBooking(bid);
            }
        });

        btnSaveFile.addActionListener(e -> {
            String selected = (String) invoiceSelectorCombo.getSelectedItem();
            if (selected != null && selected.contains(" ")) {
                String bid = selected.split(" ")[0];
                try {
                    K2703047_Invoice inv = hotelSystem.generateInvoice(bid);
                    String path = "invoices/" + bid + ".txt";
                    inv.exportToFile(path);
                    JOptionPane.showMessageDialog(this, "Invoice exported successfully to:\n" + path, "Export
Complete", JOptionPane.INFORMATION_MESSAGE);
                } catch (Exception ex) {
                    JOptionPane.showMessageDialog(this, "Export failed: " + ex.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
                }
            }
        });

        btnCopy.addActionListener(e -> {
            String text = invoiceTextArea.getText();
            if (!text.isEmpty()) {
                Toolkit.getDefaultToolkit().getSystemClipboard().setContents(new StringSelection(text), null);
                JOptionPane.showMessageDialog(this, "Invoice text copied to clipboard!");
            }
```

```
    });

    return panel;
}

private void displayInvoiceForBooking(String bookingId) {
    try {
        K2703047_Invoice inv = hotelSystem.generateInvoice(bookingId);
        invoiceTextArea.setText(inv.generateInvoiceText());
        invoiceTextArea.setCaretPosition(0);
    } catch (Exception e) {
        invoiceTextArea.setText("Error generating invoice: " + e.getMessage());
    }
}

// Refresh Data Helpers
public void refreshAllData() {
    refreshRoomsTable();
    refreshGuestsTable();
    refreshReservationsTable();
    refreshCombos();
}

private void refreshRoomsTable() {
    roomTableModel.setRowCount(0);
    for (K2703047_Room r : hotelSystem.getAllRooms()) {
        roomTableModel.addRow(new Object[]{
                r.getRoomId(),
                r.getRoomType().getDisplayName(),
                r.getBedType().getDisplayName(),
                String.format("%,.2f", r.getDailyRate()),
                r.hasComplimentaryBreakfast() ? "Yes" : "No",
                r.getBaseCapacity(),
                r.getStatus()
        });
    }
}

private void refreshGuestsTable() {
    guestTableModel.setRowCount(0);
    for (K2703047_Guest g : hotelSystem.getAllGuests()) {
        guestTableModel.addRow(new Object[]{
                g.getNicOrPassport(),
                g.getFullName(),
                g.getContactNumber(),
                g.getEmailAddress()
        });
    }
}

private void refreshReservationsTable() {
    populateReservationTable(hotelSystem.getAllReservations());
}

private void populateReservationTable(List<K2703047_Reservation> list) {
    reservationTableModel.setRowCount(0);
    for (K2703047_Reservation r : list) {
        reservationTableModel.addRow(new Object[]{
                r.getBookingId(),
                r.getGuest().getFullName(),
                r.getRoom().getRoomId(),
                r.getCheckInDate(),
```

```
                    r.getCheckOutDate(),
                    r.getNumberOfNights(),
                    r.getNumberOfGuests(),
                    r.getStatus().getDisplayName()
            });
        }
    }

    private void refreshCombos() {
        if (guestSelectorCombo != null) {
            guestSelectorCombo.removeAllItems();
            for (K2703047_Guest g : hotelSystem.getAllGuests()) {
                guestSelectorCombo.addItem(g.getFullName() + " [" + g.getNicOrPassport() + "]");
            }
        }
        if (invoiceSelectorCombo != null) {
            invoiceSelectorCombo.removeAllItems();
            for (K2703047_Reservation r : hotelSystem.getAllReservations()) {
                invoiceSelectorCombo.addItem(r.getBookingId() + " - " + r.getGuest().getFullName() + " (" +
r.getRoom().getRoomId() + ")");
            }
            if (invoiceSelectorCombo.getItemCount() > 0 && invoiceTextArea.getText().isEmpty()) {
                String first = invoiceSelectorCombo.getItemAt(0);
                displayInvoiceForBooking(first.split(" ")[0]);
            }
        }
    }

    private void alignTableColumns(JTable table, int... centerCols) {
        DefaultTableCellRenderer center = new DefaultTableCellRenderer();
        center.setHorizontalAlignment(SwingConstants.CENTER);
        for (int c : centerCols) {
            table.getColumnModel().getColumn(c).setCellRenderer(center);
        }
    }
}
```

```
package k2703047.smartstay.ui;

import k2703047.smartstay.model.K2703047_BedType;
import k2703047.smartstay.model.K2703047_Guest;
import k2703047.smartstay.model.K2703047_Invoice;
import k2703047.smartstay.model.K2703047_Reservation;
import k2703047.smartstay.model.K2703047_ReservationStatus;
import k2703047.smartstay.model.K2703047_Room;
import k2703047.smartstay.model.K2703047_RoomFactory;
import k2703047.smartstay.model.K2703047_RoomStatus;
import k2703047.smartstay.model.K2703047_RoomType;
import k2703047.smartstay.service.K2703047_HotelSystem;

import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.border.TitledBorder;
import javax.swing.table.DefaultTableCellRenderer;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.datatransfer.StringSelection;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.List;
```

```java
/**
 * Kingston University Coursework - CI6115 Programming III
 * Student ID: 2703047
 * Module: Patterns and Algorithms
 *
 * Desktop Graphical User Interface (Java Swing) for SmartStay Hotel Booking System.
 * Evaluates UI/UX design:
 * - Professional layout with tabs, tables, status indicators, and responsive panels
 * - Form validation and clear feedback dialogs
 * - Live billing estimator and printable/exportable invoice preview
 */
public class K2703047_GUIView extends JFrame {

    private final K2703047_HotelSystem hotelSystem;
    private final DateTimeFormatter dateFormatter = DateTimeFormatter.ofPattern("yyyy-MM-dd");

    // Table Models
    private DefaultTableModel roomTableModel;
    private DefaultTableModel guestTableModel;
    private DefaultTableModel reservationTableModel;
    private DefaultTableModel searchResultsTableModel;

    // Component references
    private JTable roomTable;
    private JTable guestTable;
    private JTable reservationTable;
    private JTable searchResultsTable;
    private JTextArea invoiceTextArea;
    private JComboBox<String> guestSelectorCombo;
    private JComboBox<String> invoiceSelectorCombo;

    public K2703047_GUIView(K2703047_HotelSystem hotelSystem) {
        super("SmartStay Hotel Chain - Room Reservation System | Student ID: 2703047");
        this.hotelSystem = hotelSystem;
        initializeUI();
        refreshAllData();
    }

    public static void launch(K2703047_HotelSystem hotelSystem) {
        SwingUtilities.invokeLater(() -> {
            try {
                UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
            } catch (Exception ignored) {}
            K2703047_GUIView frame = new K2703047_GUIView(hotelSystem);
            frame.setVisible(true);
        });
    }

    private void initializeUI() {
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        setSize(1100, 750);
        setMinimumSize(new Dimension(950, 650));
        setLocationRelativeTo(null);

        JPanel mainPanel = new JPanel(new BorderLayout(10, 10));
        mainPanel.setBorder(new EmptyBorder(12, 12, 12, 12));

        // Top Header
        JPanel headerPanel = new JPanel(new BorderLayout());
        headerPanel.setBackground(new Color(24, 43, 73));
        headerPanel.setBorder(new EmptyBorder(14, 18, 14, 18));
```

```
        JLabel titleLabel = new JLabel("SMARTSTAY HOTEL CHAIN - SRI LANKA");
        titleLabel.setFont(new Font("Segoe UI", Font.BOLD, 20));
        titleLabel.setForeground(Color.WHITE);

        JLabel subLabel = new JLabel("CI6115 Programming III | Patterns & Algorithms | Student ID: 2703047");
        subLabel.setFont(new Font("Segoe UI", Font.PLAIN, 12));
        subLabel.setForeground(new Color(200, 220, 255));

        headerPanel.add(titleLabel, BorderLayout.NORTH);
        headerPanel.add(subLabel, BorderLayout.SOUTH);
        mainPanel.add(headerPanel, BorderLayout.NORTH);

        // Tabbed Pane
        JTabbedPane tabbedPane = new JTabbedPane();
        tabbedPane.setFont(new Font("Segoe UI", Font.PLAIN, 13));

        tabbedPane.addTab("  Rooms (Admin)  ", createRoomsPanel());
        tabbedPane.addTab("  Guest Registration  ", createGuestPanel());
        tabbedPane.addTab("  Search & Book  ", createBookingPanel());
        tabbedPane.addTab("  Reservations & Cancellation  ", createReservationManagementPanel());
        tabbedPane.addTab("  Detailed Invoices  ", createInvoicePanel());

        mainPanel.add(tabbedPane, BorderLayout.CENTER);

        // Bottom Status Bar
        JPanel statusBar = new JPanel(new BorderLayout());
        statusBar.setBorder(new EmptyBorder(5, 5, 5, 5));
        JLabel statusLabel = new JLabel("Ready. System loaded with initial rooms, packages, and registered
guests.");
        statusLabel.setFont(new Font("Segoe UI", Font.ITALIC, 11));
        statusBar.add(statusLabel, BorderLayout.WEST);

        mainPanel.add(statusBar, BorderLayout.SOUTH);

        setContentPane(mainPanel);
    }

    // ========================================
    // TAB 1: ROOMS MANAGEMENT PANEL (ADMIN)
    // ========================================
    private JPanel createRoomsPanel() {
        JPanel panel = new JPanel(new BorderLayout(8, 8));
        panel.setBorder(new EmptyBorder(10, 10, 10, 10));

        // Table
        String[] cols = {"Room ID", "Room Type", "Bed Type", "Daily Rate (LKR)", "Breakfast", "Capacity",
"Status"};
        roomTableModel = new DefaultTableModel(cols, 0) {
            @Override
            public boolean isCellEditable(int r, int c) { return false; }
        };
        roomTable = new JTable(roomTableModel);
        roomTable.setRowHeight(26);
        roomTable.setFont(new Font("Segoe UI", Font.PLAIN, 13));
        roomTable.getTableHeader().setFont(new Font("Segoe UI", Font.BOLD, 13));
        alignTableColumns(roomTable, 0, 1, 2, 4, 5, 6);

        JScrollPane scrollPane = new JScrollPane(roomTable);
        panel.add(scrollPane, BorderLayout.CENTER);

        // Toolbar
        JPanel toolbar = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 6));
```

```
        JButton btnAdd = new JButton("Add Room");
        JButton btnUpdateRate = new JButton("Update Rate");
        JButton btnStatus = new JButton("Change Status");
        JButton btnRemove = new JButton("Remove Room");
        JButton btnRefresh = new JButton("Refresh");

        toolbar.add(btnAdd);
        toolbar.add(btnUpdateRate);
        toolbar.add(btnStatus);
        toolbar.add(btnRemove);
        toolbar.add(btnRefresh);

        panel.add(toolbar, BorderLayout.SOUTH);

        // Action Listeners
        btnAdd.addActionListener(e -> showAddRoomDialog());
        btnUpdateRate.addActionListener(e -> showUpdateRoomDialog());
        btnStatus.addActionListener(e -> showChangeStatusDialog());
        btnRemove.addActionListener(e -> handleRemoveRoomGUI());
        btnRefresh.addActionListener(e -> refreshRoomsTable());

        return panel;
    }

    private void showAddRoomDialog() {
        JTextField idField = new JTextField(10);
        JComboBox<K2703047_RoomType> typeCombo = new JComboBox<>(K2703047_RoomType.values());
        JComboBox<K2703047_BedType> bedCombo = new JComboBox<>(K2703047_BedType.values());
        JTextField rateField = new JTextField(10);

        typeCombo.addActionListener(e -> {
            K2703047_RoomType t = (K2703047_RoomType) typeCombo.getSelectedItem();
            if (t != null) rateField.setText(String.valueOf(t.getDailyRate()));
        });
        typeCombo.setSelectedIndex(0);

        JPanel form = new JPanel(new GridLayout(4, 2, 8, 8));
        form.add(new JLabel("Room ID (e.g. R-105):"));
        form.add(idField);
        form.add(new JLabel("Room Type:"));
        form.add(typeCombo);
        form.add(new JLabel("Bed Type:"));
        form.add(bedCombo);
        form.add(new JLabel("Daily Rate (LKR):"));
        form.add(rateField);

        int res = JOptionPane.showConfirmDialog(this, form, "Add New Room", JOptionPane.OK_CANCEL_OPTION);
        if (res == JOptionPane.OK_OPTION) {
            try {
                String id = idField.getText().trim().toUpperCase();
                K2703047_RoomType type = (K2703047_RoomType) typeCombo.getSelectedItem();
                K2703047_BedType bed = (K2703047_BedType) bedCombo.getSelectedItem();
                double rate = Double.parseDouble(rateField.getText().trim());

                if (hotelSystem.getRoomById(id) != null) {
                    JOptionPane.showMessageDialog(this, "Room ID already exists!", "Error",
JOptionPane.ERROR_MESSAGE);
                    return;
                }
                K2703047_Room room = K2703047_RoomFactory.createRoom(id, type, bed, rate,
K2703047_RoomStatus.AVAILABLE, type.getStandardCapacity());
```

```
                hotelSystem.addRoom(room);
                refreshRoomsTable();
                JOptionPane.showMessageDialog(this, "Room " + id + " added successfully!");
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Error adding room: " + ex.getMessage(), "Input Error",
JOptionPane.ERROR_MESSAGE);
            }
        }
    }

    private void showUpdateRoomDialog() {
        int row = roomTable.getSelectedRow();
        if (row < 0) {
            JOptionPane.showMessageDialog(this, "Please select a room to update.", "Selection Required",
JOptionPane.WARNING_MESSAGE);
            return;
        }
        String roomId = (String) roomTableModel.getValueAt(row, 0);
        K2703047_Room room = hotelSystem.getRoomById(roomId);
        if (room == null) return;

        String input = JOptionPane.showInputDialog(this, "Enter new Daily Rate (LKR):", room.getDailyRate());
        if (input != null && !input.trim().isEmpty()) {
            try {
                double newRate = Double.parseDouble(input.trim());
                room.setDailyRate(newRate);
                refreshRoomsTable();
                JOptionPane.showMessageDialog(this, "Rate updated for room " + roomId);
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Invalid rate: " + ex.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
            }
        }
    }

    private void showChangeStatusDialog() {
        int row = roomTable.getSelectedRow();
        if (row < 0) {
            JOptionPane.showMessageDialog(this, "Please select a room to modify status.", "Selection
Required", JOptionPane.WARNING_MESSAGE);
            return;
        }
        String roomId = (String) roomTableModel.getValueAt(row, 0);
        K2703047_RoomStatus current = (K2703047_RoomStatus) roomTableModel.getValueAt(row, 6);

        K2703047_RoomStatus selected = (K2703047_RoomStatus) JOptionPane.showInputDialog(
                this, "Select new availability status for " + roomId + ":",
                "Change Room Status", JOptionPane.QUESTION_MESSAGE, null,
                K2703047_RoomStatus.values(), current);

        if (selected != null) {
            hotelSystem.updateRoomStatus(roomId, selected);
            refreshRoomsTable();
            JOptionPane.showMessageDialog(this, "Room " + roomId + " status set to " + selected);
        }
    }

    private void handleRemoveRoomGUI() {
        int row = roomTable.getSelectedRow();
        if (row < 0) {
            JOptionPane.showMessageDialog(this, "Please select a room to remove.", "Selection Required",
JOptionPane.WARNING_MESSAGE);
```

```
            return;
        }
        String roomId = (String) roomTableModel.getValueAt(row, 0);
        int confirm = JOptionPane.showConfirmDialog(this, "Are you sure you want to remove room " + roomId +
"?", "Confirm Removal", JOptionPane.YES_NO_OPTION);
        if (confirm == JOptionPane.YES_OPTION) {
            try {
                hotelSystem.removeRoom(roomId);
                refreshRoomsTable();
                JOptionPane.showMessageDialog(this, "Room " + roomId + " removed.");
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, ex.getMessage(), "Cannot Remove",
JOptionPane.ERROR_MESSAGE);
            }
        }
    }

    // ========================================
    // TAB 2: GUEST MANAGEMENT PANEL
    // ========================================
    private JPanel createGuestPanel() {
        JPanel panel = new JPanel(new BorderLayout(10, 10));
        panel.setBorder(new EmptyBorder(10, 10, 10, 10));

        // Top Form
        JPanel formPanel = new JPanel(new GridLayout(2, 4, 8, 8));
        formPanel.setBorder(BorderFactory.createTitledBorder("Register New Customer"));

        JTextField nicField = new JTextField();
        JTextField nameField = new JTextField();
        JTextField phoneField = new JTextField();
        JTextField emailField = new JTextField();

        formPanel.add(new JLabel("NIC / Passport:"));
        formPanel.add(new JLabel("Full Name:"));
        formPanel.add(new JLabel("Contact Number:"));
        formPanel.add(new JLabel("Email Address:"));

        formPanel.add(nicField);
        formPanel.add(nameField);
        formPanel.add(phoneField);
        formPanel.add(emailField);

        JPanel formWrapper = new JPanel(new BorderLayout());
        formWrapper.add(formPanel, BorderLayout.CENTER);

        JButton btnRegister = new JButton("Register Customer");
        btnRegister.setFont(new Font("Segoe UI", Font.BOLD, 12));
        JPanel btnWrapper = new JPanel(new FlowLayout(FlowLayout.RIGHT));
        btnWrapper.add(btnRegister);
        formWrapper.add(btnWrapper, BorderLayout.SOUTH);

        panel.add(formWrapper, BorderLayout.NORTH);

        // Guest Table
        String[] cols = {"NIC / Passport", "Full Name", "Contact Number", "Email Address"};
        guestTableModel = new DefaultTableModel(cols, 0) {
            @Override
            public boolean isCellEditable(int r, int c) { return false; }
        };
        guestTable = new JTable(guestTableModel);
        guestTable.setRowHeight(24);
```

```
        panel.add(new JScrollPane(guestTable), BorderLayout.CENTER);

        btnRegister.addActionListener(e -> {
            try {
                String nic = nicField.getText().trim().toUpperCase();
                String name = nameField.getText().trim();
                String phone = phoneField.getText().trim();
                String email = emailField.getText().trim();

                boolean ok = hotelSystem.registerGuest(nic, name, phone, email);
                if (ok) {
                    JOptionPane.showMessageDialog(this, "Guest registered successfully!");
                    nicField.setText("");
                    nameField.setText("");
                    phoneField.setText("");
                    emailField.setText("");
                    refreshGuestsTable();
                } else {
                    JOptionPane.showMessageDialog(this, "Guest already registered with this NIC/Passport.",
"Warning", JOptionPane.WARNING_MESSAGE);
                }
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Validation Error: " + ex.getMessage(), "Input Error",
JOptionPane.ERROR_MESSAGE);
            }
        });

        return panel;
    }

    // ========================================
    // TAB 3: SEARCH & BOOKING PANEL
    // ========================================
    private JPanel createBookingPanel() {
        JPanel panel = new JPanel(new BorderLayout(10, 10));
        panel.setBorder(new EmptyBorder(10, 10, 10, 10));

        // Search Criteria Panel
        JPanel searchPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 12, 8));
        searchPanel.setBorder(BorderFactory.createTitledBorder("Search Available Rooms"));

        JComboBox<String> typeFilterCombo = new JComboBox<>(new String[]{"All Room Types", "Standard",
"Deluxe", "Suite", "Family Suite"});
        LocalDate minCheckIn = LocalDate.now().plusDays(2);
        JTextField inDateField = new JTextField(minCheckIn.format(dateFormatter), 9);
        JTextField outDateField = new JTextField(minCheckIn.plusDays(3).format(dateFormatter), 9);
        JButton btnSearch = new JButton("Find Available Rooms");
        btnSearch.setFont(new Font("Segoe UI", Font.BOLD, 12));

        searchPanel.add(new JLabel("Room Type:"));
        searchPanel.add(typeFilterCombo);
        searchPanel.add(new JLabel("Check-In (Min 2 days advance):"));
        searchPanel.add(inDateField);
        searchPanel.add(new JLabel("Check-Out:"));
        searchPanel.add(outDateField);
        searchPanel.add(btnSearch);

        panel.add(searchPanel, BorderLayout.NORTH);

        // Search Results Table
        String[] cols = {"Room ID", "Type", "Bed", "Daily Rate (LKR)", "Breakfast Included", "Base
Capacity"};
```

```
        searchResultsTableModel = new DefaultTableModel(cols, 0) {
            @Override
            public boolean isCellEditable(int r, int c) { return false; }
        };
        searchResultsTable = new JTable(searchResultsTableModel);
        searchResultsTable.setRowHeight(24);
        panel.add(new JScrollPane(searchResultsTable), BorderLayout.CENTER);

        // Booking Action Panel at Bottom
        JPanel actionPanel = new JPanel(new GridLayout(2, 1, 6, 6));
        actionPanel.setBorder(BorderFactory.createTitledBorder("Complete Room Reservation"));

        JPanel inputRow = new JPanel(new FlowLayout(FlowLayout.LEFT, 15, 6));
        guestSelectorCombo = new JComboBox<>();
        JSpinner guestCountSpinner = new JSpinner(new SpinnerNumberModel(2, 1, 10, 1));
        JButton btnBook = new JButton("Confirm Booking & Pay LKR 10,000 Advance");
        btnBook.setFont(new Font("Segoe UI", Font.BOLD, 13));
        btnBook.setBackground(new Color(24, 134, 75));
        btnBook.setForeground(Color.WHITE);

        inputRow.add(new JLabel("Select Registered Guest:"));
        inputRow.add(guestSelectorCombo);
        inputRow.add(new JLabel("Number of Guests:"));
        inputRow.add(guestCountSpinner);
        inputRow.add(btnBook);

        JLabel policyLabel = new JLabel("  * Refundable advance of LKR 10,000 is charged at reservation. Free
cancellation until 24h prior to check-in.");
        policyLabel.setFont(new Font("Segoe UI", Font.ITALIC, 11));
        policyLabel.setForeground(new Color(80, 80, 80));

        actionPanel.add(inputRow);
        actionPanel.add(policyLabel);

        panel.add(actionPanel, BorderLayout.SOUTH);

        // Action Listener for Search
        btnSearch.addActionListener(e -> {
            try {
                LocalDate inDate = LocalDate.parse(inDateField.getText().trim(), dateFormatter);
                LocalDate outDate = LocalDate.parse(outDateField.getText().trim(), dateFormatter);

                int typeIdx = typeFilterCombo.getSelectedIndex();
                K2703047_RoomType filterType = null;
                if (typeIdx == 1) filterType = K2703047_RoomType.STANDARD;
                else if (typeIdx == 2) filterType = K2703047_RoomType.DELUXE;
                else if (typeIdx == 3) filterType = K2703047_RoomType.SUITE;
                else if (typeIdx == 4) filterType = K2703047_RoomType.FAMILY_SUITE;

                List<K2703047_Room> results = hotelSystem.searchAvailableRooms(filterType, inDate, outDate);
                searchResultsTableModel.setRowCount(0);
                for (K2703047_Room r : results) {
                    searchResultsTableModel.addRow(new Object[]{
                            r.getRoomId(),
                            r.getRoomType().getDisplayName(),
                            r.getBedType().getDisplayName(),
                            String.format("LKR %,.2f", r.getDailyRate()),
                            r.hasComplimentaryBreakfast() ? "Yes" : "No",
                            r.getBaseCapacity()
                    });
                }
                if (results.isEmpty()) {
```

```
                    JOptionPane.showMessageDialog(this, "No available rooms for the selected dates.", "Search
Results", JOptionPane.INFORMATION_MESSAGE);
                }
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Search error: " + ex.getMessage(), "Input Error",
JOptionPane.ERROR_MESSAGE);
            }
        });

        // Action Listener for Booking
        btnBook.addActionListener(e -> {
            int row = searchResultsTable.getSelectedRow();
            if (row < 0) {
                JOptionPane.showMessageDialog(this, "Please select an available room from the search results
table.", "Selection Required", JOptionPane.WARNING_MESSAGE);
                return;
            }
            if (guestSelectorCombo.getSelectedIndex() < 0) {
                JOptionPane.showMessageDialog(this, "Please select or register a guest first.", "Guest
Required", JOptionPane.WARNING_MESSAGE);
                return;
            }

            try {
                String roomId = (String) searchResultsTableModel.getValueAt(row, 0);
                String selectedGuest = (String) guestSelectorCombo.getSelectedItem();
                String nic = selectedGuest.substring(selectedGuest.indexOf("[") + 1,
selectedGuest.indexOf("]"));

                LocalDate inDate = LocalDate.parse(inDateField.getText().trim(), dateFormatter);
                LocalDate outDate = LocalDate.parse(outDateField.getText().trim(), dateFormatter);
                int guestCount = (int) guestCountSpinner.getValue();

                K2703047_Reservation res = hotelSystem.makeReservation(nic, roomId, inDate, outDate,
guestCount);
                refreshAllData();

                JOptionPane.showMessageDialog(this,
                        "Reservation confirmed successfully!\n" +
                        "Booking ID: " + res.getBookingId() + "\n" +
                        "Refundable Advance Charged: LKR 10,000\n\n" +
                        "View detailed invoice in the 'Detailed Invoices' tab.",
                        "Reservation Confirmed", JOptionPane.INFORMATION_MESSAGE);

                // Auto-show invoice
                displayInvoiceForBooking(res.getBookingId());

            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Booking Failed: " + ex.getMessage(), "Reservation
Error", JOptionPane.ERROR_MESSAGE);
            }
        });

        return panel;
    }

    // ========================================
    // TAB 4: RESERVATIONS & CANCELLATION PANEL
    // ========================================
    private JPanel createReservationManagementPanel() {
        JPanel panel = new JPanel(new BorderLayout(10, 10));
        panel.setBorder(new EmptyBorder(10, 10, 10, 10));
```

```
        // Filter / Search Toolbar
        JPanel searchBar = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 6));
        searchBar.setBorder(BorderFactory.createTitledBorder("Search & Filter Reservations"));

        JTextField searchField = new JTextField(12);
        JButton btnSearchName = new JButton("Search Guest Name");
        JButton btnBinarySearchId = new JButton("Binary Search ID");
        JTextField checkInDateField = new JTextField(9);
        JButton btnFilterDate = new JButton("Filter by Check-In Date");
        JButton btnShowAll = new JButton("Show All");

        searchBar.add(new JLabel("Keyword / ID:"));
        searchBar.add(searchField);
        searchBar.add(btnSearchName);
        searchBar.add(btnBinarySearchId);
        searchBar.add(new JLabel("Check-In:"));
        searchBar.add(checkInDateField);
        searchBar.add(btnFilterDate);
        searchBar.add(btnShowAll);

        panel.add(searchBar, BorderLayout.NORTH);

        // Reservations Table
        String[] cols = {"Booking ID", "Guest Name", "Room ID", "Check-In", "Check-Out", "Nights", "Guests",
"Status"};
        reservationTableModel = new DefaultTableModel(cols, 0) {
            @Override
            public boolean isCellEditable(int r, int c) { return false; }
        };
        reservationTable = new JTable(reservationTableModel);
        reservationTable.setRowHeight(24);
        panel.add(new JScrollPane(reservationTable), BorderLayout.CENTER);

        // Bottom Actions
        JPanel bottomBar = new JPanel(new FlowLayout(FlowLayout.RIGHT, 12, 6));
        JButton btnCancel = new JButton("Cancel Selected Reservation");
        btnCancel.setForeground(Color.RED);
        btnCancel.setFont(new Font("Segoe UI", Font.BOLD, 12));

        JButton btnViewInvoice = new JButton("View Detailed Invoice");
        btnViewInvoice.setFont(new Font("Segoe UI", Font.BOLD, 12));

        bottomBar.add(btnCancel);
        bottomBar.add(btnViewInvoice);
        panel.add(bottomBar, BorderLayout.SOUTH);

        // Action Listeners
        btnSearchName.addActionListener(e -> {
            String kw = searchField.getText().trim();
            List<K2703047_Reservation> list = hotelSystem.searchReservationsByGuestName(kw);
            populateReservationTable(list);
        });

        btnBinarySearchId.addActionListener(e -> {
            String bid = searchField.getText().trim().toUpperCase();
            K2703047_Reservation r = hotelSystem.searchReservationBinary(bid);
            if (r != null) {
                populateReservationTable(List.of(r));
                JOptionPane.showMessageDialog(this, "Found reservation via O(log N) Binary Search
Algorithm!", "Binary Search Result", JOptionPane.INFORMATION_MESSAGE);
            } else {
```

```
                JOptionPane.showMessageDialog(this, "No reservation found with Booking ID: " + bid, "Not
Found", JOptionPane.WARNING_MESSAGE);
            }
        });

        btnFilterDate.addActionListener(e -> {
            try {
                LocalDate date = LocalDate.parse(checkInDateField.getText().trim(), dateFormatter);
                List<K2703047_Reservation> list = hotelSystem.getReservationsByCheckInDate(date);
                populateReservationTable(list);
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Invalid date format. Use YYYY-MM-DD", "Error",
JOptionPane.ERROR_MESSAGE);
            }
        });

        btnShowAll.addActionListener(e -> refreshReservationsTable());

        btnCancel.addActionListener(e -> {
            int row = reservationTable.getSelectedRow();
            if (row < 0) {
                JOptionPane.showMessageDialog(this, "Please select a reservation to cancel.", "Selection
Required", JOptionPane.WARNING_MESSAGE);
                return;
            }
            String bid = (String) reservationTableModel.getValueAt(row, 0);
            int confirm = JOptionPane.showConfirmDialog(this,
                    "Are you sure you want to cancel booking " + bid + "?\n" +
                    "Notice: Cancellation is strictly prohibited within 24 hours of check-in.",
                    "Confirm Cancellation", JOptionPane.YES_NO_OPTION);

            if (confirm == JOptionPane.YES_OPTION) {
                boolean ok = hotelSystem.cancelReservation(bid);
                if (ok) {
                    JOptionPane.showMessageDialog(this,
                            "Reservation " + bid + " cancelled successfully.\n" +
                            "Refund of LKR 10,000 booking advance processed.",
                            "Cancellation Approved", JOptionPane.INFORMATION_MESSAGE);
                    refreshAllData();
                } else {
                    JOptionPane.showMessageDialog(this,
                            "Cancellation Rejected!\n" +
                            "Business rule violation: Reservations cannot be modified or cancelled within 24
hours of check-in.",
                            "Cancellation Policy Violation", JOptionPane.ERROR_MESSAGE);
                }
            }
        });

        btnViewInvoice.addActionListener(e -> {
            int row = reservationTable.getSelectedRow();
            if (row < 0) {
                JOptionPane.showMessageDialog(this, "Please select a reservation to view its invoice.",
"Selection Required", JOptionPane.WARNING_MESSAGE);
                return;
            }
            String bid = (String) reservationTableModel.getValueAt(row, 0);
            displayInvoiceForBooking(bid);
        });

        return panel;
    }
```

```
    // ========================================
    // TAB 5: INVOICE VIEWER PANEL
    // ========================================
    private JPanel createInvoicePanel() {
        JPanel panel = new JPanel(new BorderLayout(10, 10));
        panel.setBorder(new EmptyBorder(10, 10, 10, 10));

        // Selector Bar
        JPanel topBar = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 6));
        invoiceSelectorCombo = new JComboBox<>();
        JButton btnLoad = new JButton("Load Invoice");
        JButton btnSaveFile = new JButton("Export Invoice to Text File");
        JButton btnCopy = new JButton("Copy to Clipboard");

        topBar.add(new JLabel("Select Booking:"));
        topBar.add(invoiceSelectorCombo);
        topBar.add(btnLoad);
        topBar.add(btnSaveFile);
        topBar.add(btnCopy);

        panel.add(topBar, BorderLayout.NORTH);

        // Monospace Text Area for Invoice
        invoiceTextArea = new JTextArea();
        invoiceTextArea.setEditable(false);
        invoiceTextArea.setFont(new Font("Consolas", Font.PLAIN, 13));
        invoiceTextArea.setBackground(new Color(250, 252, 255));
        invoiceTextArea.setBorder(new EmptyBorder(10, 10, 10, 10));

        panel.add(new JScrollPane(invoiceTextArea), BorderLayout.CENTER);

        btnLoad.addActionListener(e -> {
            String selected = (String) invoiceSelectorCombo.getSelectedItem();
            if (selected != null && selected.contains(" ")) {
                String bid = selected.split(" ")[0];
                displayInvoiceForBooking(bid);
            }
        });

        btnSaveFile.addActionListener(e -> {
            String selected = (String) invoiceSelectorCombo.getSelectedItem();
            if (selected != null && selected.contains(" ")) {
                String bid = selected.split(" ")[0];
                try {
                    K2703047_Invoice inv = hotelSystem.generateInvoice(bid);
                    String path = "invoices/" + bid + ".txt";
                    inv.exportToFile(path);
                    JOptionPane.showMessageDialog(this, "Invoice exported successfully to:\n" + path, "Export
Complete", JOptionPane.INFORMATION_MESSAGE);
                } catch (Exception ex) {
                    JOptionPane.showMessageDialog(this, "Export failed: " + ex.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
                }
            }
        });

        btnCopy.addActionListener(e -> {
            String text = invoiceTextArea.getText();
            if (!text.isEmpty()) {
                Toolkit.getDefaultToolkit().getSystemClipboard().setContents(new StringSelection(text),
null);
```

```
            JOptionPane.showMessageDialog(this, "Invoice text copied to clipboard!");
        }
    });

    return panel;
}

private void displayInvoiceForBooking(String bookingId) {
    try {
        K2703047_Invoice inv = hotelSystem.generateInvoice(bookingId);
        invoiceTextArea.setText(inv.generateInvoiceText());
        invoiceTextArea.setCaretPosition(0);
    } catch (Exception e) {
        invoiceTextArea.setText("Error generating invoice: " + e.getMessage());
    }
}

// Refresh Data Helpers
public void refreshAllData() {
    refreshRoomsTable();
    refreshGuestsTable();
    refreshReservationsTable();
    refreshCombos();
}

private void refreshRoomsTable() {
    roomTableModel.setRowCount(0);
    for (K2703047_Room r : hotelSystem.getAllRooms()) {
        roomTableModel.addRow(new Object[]{
                r.getRoomId(),
                r.getRoomType().getDisplayName(),
                r.getBedType().getDisplayName(),
                String.format("%,.2f", r.getDailyRate()),
                r.hasComplimentaryBreakfast() ? "Yes" : "No",
                r.getBaseCapacity(),
                r.getStatus()
        });
    }
}

private void refreshGuestsTable() {
    guestTableModel.setRowCount(0);
    for (K2703047_Guest g : hotelSystem.getAllGuests()) {
        guestTableModel.addRow(new Object[]{
                g.getNicOrPassport(),
                g.getFullName(),
                g.getContactNumber(),
                g.getEmailAddress()
        });
    }
}

private void refreshReservationsTable() {
    populateReservationTable(hotelSystem.getAllReservations());
}

private void populateReservationTable(List<K2703047_Reservation> list) {
    reservationTableModel.setRowCount(0);
    for (K2703047_Reservation r : list) {
        reservationTableModel.addRow(new Object[]{
                r.getBookingId(),
                r.getGuest().getFullName(),
```

```
                    r.getRoom().getRoomId(),
                    r.getCheckInDate(),
                    r.getCheckOutDate(),
                    r.getNumberOfNights(),
                    r.getNumberOfGuests(),
                    r.getStatus().getDisplayName()
            });
        }
    }

    private void refreshCombos() {
        if (guestSelectorCombo != null) {
            guestSelectorCombo.removeAllItems();
            for (K2703047_Guest g : hotelSystem.getAllGuests()) {
                guestSelectorCombo.addItem(g.getFullName() + " [" + g.getNicOrPassport() + "]");
            }
        }
        if (invoiceSelectorCombo != null) {
            invoiceSelectorCombo.removeAllItems();
            for (K2703047_Reservation r : hotelSystem.getAllReservations()) {
                invoiceSelectorCombo.addItem(r.getBookingId() + " - " + r.getGuest().getFullName() + " (" +
r.getRoom().getRoomId() + ")");
            }
            if (invoiceSelectorCombo.getItemCount() > 0 && invoiceTextArea.getText().isEmpty()) {
                String first = invoiceSelectorCombo.getItemAt(0);
                displayInvoiceForBooking(first.split(" ")[0]);
            }
        }
    }

    private void alignTableColumns(JTable table, int... centerCols) {
        DefaultTableCellRenderer center = new DefaultTableCellRenderer();
        center.setHorizontalAlignment(SwingConstants.CENTER);
        for (int c : centerCols) {
            table.getColumnModel().getColumn(c).setCellRenderer(center);
        }
    }
}
```